Circuit Codex

User manual

What each tool does, what its terms mean, and where its numbers come from. Tap a tool to go to its section.

Index

PASSIVE COMPONENTS

Resistor color code	Resistors
SMD resistor code	Resistors
Resistor power rating	Resistors
E-series values	Resistors
Series/parallel (R)	Resistors
Voltage divider	Resistors
Current divider	Resistors
Wheatstone bridge	Resistors
Delta-Y transform	Resistors
NTC/PTC thermistor	Resistors
Ceramic cap code	Capacitors
Film capacitor code	Capacitors
SMD capacitor code	Capacitors
Series/parallel (C)	Capacitors
RC charge and discharge	Capacitors
Capacitor energy	Capacitors
Inductor color code	Inductors
Inductor SMD code	Inductors
RC filter	Passive filters
RL filter	Passive filters
SMD package sizes	Reference

ACTIVE & SEMICONDUCTOR DEVICES

Diode forward voltage	Diodes
LED series resistor	Diodes
BJT biasing	Transistors (BJT)
BJT as a switch	Transistors (BJT)
BJT currents	Transistors (BJT)
MOSFET biasing	MOSFET / IGBT
MOSFET switching	MOSFET / IGBT
Half-wave rectifier	Rectifiers
Bridge rectifier	Rectifiers
Center-tap rectifier	Rectifiers
Rectifier ripple	Rectifiers
AC phase control	Thyristors (SCR, TRIAC) & MOSFET
Inverting	Op-amps
Non-inverting	Op-amps
Buffer	Op-amps
Comparators	Op-amps
Integrator	Op-amps
Differentiator	Op-amps
Summing	Op-amps
Differential	Op-amps
Photocell / LDR	Optoelectronics
Optocoupler	Optoelectronics

Wavelength ↔ color	Optoelectronics

DIGITAL & LOGIC CIRCUITS

Logic gates	Logic gates
Karnaugh map	Logic gates
Flip-flops	Sequential logic
Multivibrators	Sequential logic
555 timer	Sequential logic
Number bases	Number systems
ASCII table	Number systems
PWM (duty cycle)	Timing & interfaces
Switch debounce	Timing & interfaces
I2C pull-up resistor	Timing & interfaces
UART baud rate	Timing & interfaces
Crystal load	Timing & interfaces
Oscillator stability	Timing & interfaces
PLL settings	Timing & interfaces
ADC resolution	Data conversion
DAC resolution	Data conversion
SNR estimation	Data conversion

RF & MICROWAVE

Radio wavelength	Antennas & propagation

Resistor color code



What it computes

The resistance, tolerance and, on 6-band parts, temperature coefficient that a resistor's colour bands stand for — and the other way round: type a value and it shows the bands. It also says whether the value is a standard E-series value and, if not, the nearest one.

Source

IEC 60062:2016, *Marking codes for resistors and capacitors*. Its Table 1 gives each colour a digit, a multiplier, a tolerance and a temperature coefficient; clause 3 says how the bands are laid out. Both were read in the official preview of the standard, which runs to clause 4.2.2 — all this tool needs.

What the bands mean

A resistor is too small to print a number on, so the value is painted as rings of colour, one digit per ring. Each colour stands for a digit from 0 to 9: black 0, brown 1, red 2, orange 3, yellow 4, green 5, blue 6, violet 7, grey 8, white 9.

- **Digit bands** — the first two (4-band parts) or three (5- and 6-band parts) give the significant figures.
- **Multiplier** — the next band says how many zeros follow, as a power of ten: red is ×100, orange ×1000. Gold (×0.1), silver (×0.01) and pink (×0.001, new in the 2016 edition) make values below 10 Ω: yellow–violet–pink is 47 mΩ.
- **Tolerance** — how far the actual resistance may be from the marked value: gold ±5% means a 1 kΩ part measures somewhere from 950 Ω to 1050 Ω. Leaving the band off means ±20%.
- **Temperature coefficient** (6-band only) — how much the resistance drifts per degree of temperature change, in parts per million per kelvin: brown is 100 ppm/K, so 0.01% per degree.

Brown–black–red–gold is therefore 1, 0, ×100, ±5%: 1 kΩ ±5%.

Which end to start from. The tolerance band stands apart from the others, on the right, and the standard makes it at least 1.5 times as wide as the others; the digits are bunched on the left, starting as near the end as possible. Gold and silver are never digits, so a part with gold at one end is read from the other. On 6-band parts the temperature band comes last but is not the wide one: the 2016 edition changed that, because a wide sixth band was mistaken for the tolerance.

Standard values. Resistors are made only in preferred values, the E-series, spaced so that neighbouring values just overlap within their tolerance. The tolerance band picks the one series the value is checked against — gold, ±5%, means E24; brown, ±1%, means E96 — and when the value is not in it, the tool gives the nearest one that is. A 1% 4.7 kΩ part therefore reads "Not in E96", though makers do sell it (see the E-series tool).

How the value is worked out

```
4 bands:  Value = (10 × D1 + D2) × Multiplier
5, 6:     Value = (100 × D1 + 10 × D2 + D3) × Multiplier
```

Typing a value runs it backwards: the value is split into as many significant digits as the band count allows, rounded, and the remaining power of ten picks the multiplier colour. The smallest value a code can show is 10 mΩ on 4 bands (brown–black–pink) and 0.1 Ω on 5 or 6; the largest is 99 GΩ and 999 GΩ. Outside that there is no band for it, and the tool says so.

Assumptions and limits

- **The 2016 tolerance colours.** IEC 60062:2016 made orange ±0.05%, yellow ±0.02% and grey ±0.01%. Before that edition some manufacturers used grey for a non-standard ±0.05%, so a grey tolerance band on an old part may mean that.
- **Colours are only as good as the light.** Red, orange and brown are easily confused on a real part, and so are violet and grey. When in doubt, measure.
- **Zero-ohm links** are marked with a single black band and are not a colour code as such.

What it deliberately does not do

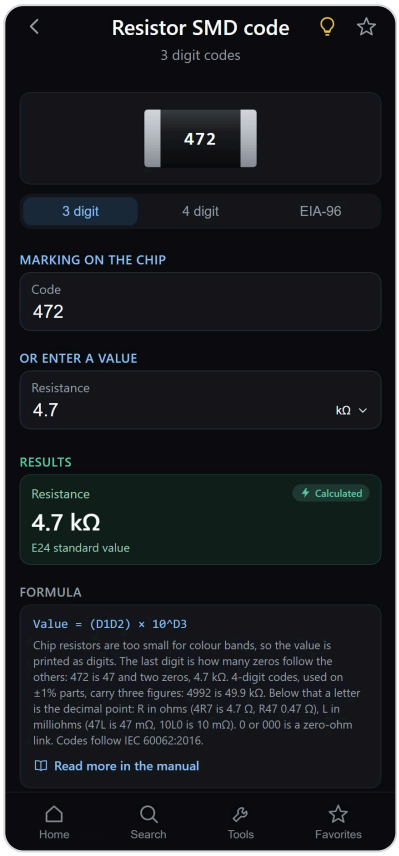
- **Capacitor and inductor colour codes** have their own tools, since the same colours mean different units and tolerances there.
- **SMD resistor codes** (the three- and four-digit and EIA-96 markings) are the SMD code tool.

How it was checked

Against the IEC 60062:2016 table: digits, multipliers, tolerances and temperature coefficients for every colour. This check found the app had grey as ±0.05%, the pre-2016 non-standard value, and no tolerance for orange or yellow; all three now follow the standard. A second reading of the official text found two more gaps, now fixed: the pink ×0.001 multiplier was missing, and the drawing made the tolerance band no wider than the others.

Brown–black–red–gold reads 1 kΩ ±5%, 950 Ω to 1.05 kΩ, an E24 value. 47 mΩ gives yellow–violet–pink. At grey's ±0.01% the range reads 999.9 Ω to 1.0001 kΩ — at four figures it had printed as "1 kΩ", which the range now shows to enough figures to see. Typing 4.7 k gives yellow–violet–red; 0.22 Ω gives red–red–silver.

SMD resistor code



What it computes

The resistance printed on a surface-mount (chip) resistor, from its marking — 3-digit, 4-digit or EIA-96 — and the other way round: the marking a value would carry. It says whether the value is a standard E-series value.

Source

IEC 60062:2016, *Marking codes for resistors and capacitors*, which defines the three-character code (clause 4.2.2), the four-character code (4.2.3), R as the decimal point, and in its Annex A a special three-character code for values with three figures — what the industry calls EIA-96.

What was read in the standard itself, and what was not: the official 2016 preview runs to clause 4.2.2, far enough to confirm the three-character code and its limit — it "is applicable to values from an E series up to E24", since it carries only two figures. The three- and four-character tables were read in **GOST IEC 60062-2014**, the official Russian edition of IEC 60062:2004, declared identical to it (IDT) and published free by Standartinform; its tables 3 and 4 match the official IEC 2004 preview word for word, and the 2016 versions of both tables in ROHM's application note "List of Nominal Resistance Values" (2024), which reproduces them from IEC 60062:2016. Annex A (EIA-96) is new in 2016, lies beyond the preview and is not in the ROHM note; it was checked against published examples only.

What the marking means

A chip resistor is far too small for colour bands, so the value is printed as characters, three or four of them.

- **3-digit code** — usual on ±5% parts. The first two digits are the value's figures, the last is how many zeros follow them. 472 is 47 followed by two zeros: 4700 Ω, 4.7 kΩ. 334 is 330 kΩ; 100 is 10 Ω, not 100 Ω.
- **4-digit code** — usual on ±1% parts, which need a third figure. The first three are the figures, the last the number of zeros. 1001 is 1.00 kΩ, 4992 is 49.9 kΩ.
- **R and L** — below 10 Ω (3-digit) or 100 Ω (4-digit) there is no zero count small enough, so a letter stands where the decimal point is: R in ohms, down to 0.1 Ω, then L in milliohms. 4R7 is 4.7 Ω, R47 0.47 Ω, 47R0 47.0 Ω; 47L is 47 mΩ, 1L0 1 mΩ, 10L0 10 mΩ, 12L7 12.7 mΩ — the milliohm values are current-sense resistors. Some makers print R010 for 10 mΩ instead of 10L0; it reads the same, and the tool reads it.
- **0, 000 or 0000** — a zero-ohm link, used as a jumper that a pick-and-place machine can fit.
- **EIA-96** — on small 1% parts (0603 and below) even four characters do not fit. Two digits and a letter do: the digits are not the value but its position among the 96 values of the E96 series, 01 for 100 up to

96 for 976, and the letter is the multiplier. 01C is the first E96 value, 100, times 100: 10 kΩ. 68X is 499 × 0.1 = 49.9 Ω.

How the value is worked out

```
3-digit:  Value = D1D2 × 10^D3
4-digit:  Value = D1D2D3 × 10^D4
EIA-96:   Value = E96[code] × multiplier
```

EIA-96 multiplier letters: Z ×0.001, Y or R ×0.01, X or S ×0.1, A ×1, B or H ×10, C ×100, D ×1000, E $\times 10^4$, F $\times 10^5$. Where two letters share a multiplier the tool reads both and prints the first.

Typing a value runs it backwards. A marking holds only two or three figures, so a value with more is rounded to what the part could carry — 12 345 Ω becomes 1232, 12.3 kΩ — and the tool says so. In EIA-96 only E96 values exist, so a value is moved to the nearest one: 4.7 kΩ becomes 66B, 4.75 kΩ.

Assumptions and limits

- **The marking does not say the tolerance.** A 3-digit code is usually ±5% and a 4-digit or EIA-96 code ±1%, but only the datasheet or the reel label says.
- **Standard value.** The value is checked against the series of that usual tolerance: E24 for a 3-digit code, E96 for 4-digit and EIA-96. So 472 is an E24 value, and 4701 reads "Not in E96 — nearest is 4.75 kΩ": E96 has no 4.7. A 3-digit code carries two figures, enough for every E6 to E24 value; the three figures of E48 to E192 need 4 digits or EIA-96.
- **Which scheme a part uses** shows in the marking: a letter other than R or L at the end means EIA-96, and digits only are the 3- or 4-digit code, told apart by how many there are. Pick the pill that matches.
- **Small parts may not be marked at all.** 0402 and smaller usually carry no marking; the value is only on the reel.
- Some makers use their own schemes for current-sense resistors (such as a lowercase m for milliohms); those are not covered.

What it deliberately does not do

- **Colour bands** are the Color code tool.
- **SMD capacitor and inductor markings** have their own tools, since the same characters mean picofarads or microhenries there.

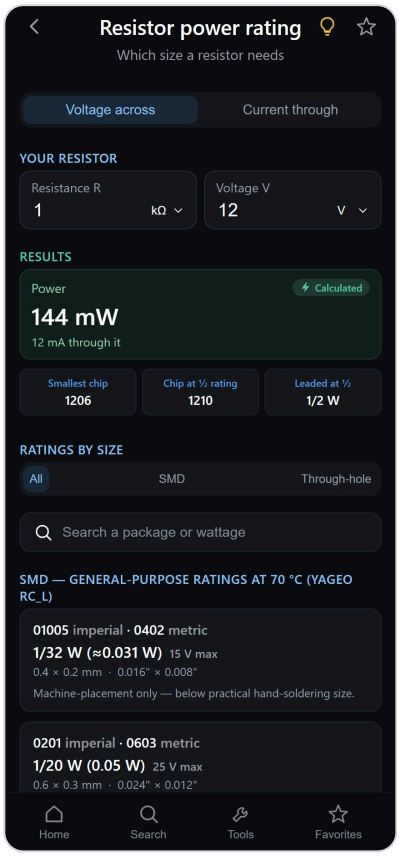
How it was checked

Against published examples of the codes: 334 is 330 kΩ, 222 is 2.2 kΩ, 1001 is 1.00 kΩ, 4992 is 49.9 kΩ, R300 is 0.30 Ω, 01C is 10 kΩ; also 68X is 49.9 Ω and 66B 4.75 kΩ.

The review found four faults, now fixed. 47 mΩ on the 3-digit pill printed "R047", four characters; 10 mΩ on the 4-digit pill printed "R0100", five characters, and typing R010 showed R0100 on the chip. A marking of just "0" for a zero-ohm link was rejected. And 12 345 Ω showed the code 1232 beside a resistance of 12.35 kΩ, with nothing to say the code means 12.3 kΩ.

Reading the standard's tables (in ROHM's note) then showed that the first fix was itself wrong: milliohms take the letter L, not R. The tool had printed 47 mΩ as R05, rounding it to 50 mΩ, and 12.7 mΩ as R013; the standard writes 47L and 12L7, exactly. It now writes the L forms — 47L, 1L0, L12, 10L0, 12L7 — and still reads R010.

Resistor power rating



What it computes

Which size a resistor needs. Enter its resistance and the voltage across it, or the current through it, and the tool gives the power it dissipates, the smallest chip size that takes it, the smallest at half its rating — the usual design margin — and the through-hole wattage class at half its rating. A chip must also be rated for the voltage across it.

Below that is the reference table: how much power each common resistor size can dissipate, with its dimensions. Surface-mount parts are listed by package code, with their maximum working voltage; through-hole parts by their wattage class, with a typical body size. It can be filtered to SMD or through-hole and searched by package or wattage.

Source

- **SMD** — Yageo's RC_L general-purpose thick-film chip resistor datasheet (V.11): rated power at 70 °C, maximum working voltage and operating temperature range for each size from 01005 to 2512. These are the conventional figures most tables repeat.
- **Vishay D/CRCW e3** (revision April 2026) as the counter-example: the same sizes rated higher, 0.125 W for 0603 and 0.25 W for 0805.
- **Through-hole** body sizes are typical ranges from distributor listings, not a standard.

What it means

When current flows through a resistor, the resistor turns electrical power into heat. The power rating is the most heat it can get rid of, continuously, without being damaged or drifting out of tolerance. Exceed it and the part runs hot, its value shifts, and eventually it fails.

The power a resistor actually has to handle comes from what is across it or through it:

$$P = V^2 / R = I^2 \times R$$

A 1 kΩ resistor with 5 V across it dissipates 25 / 1000 = 25 mW, well inside an 0603's 100 mW. The same resistor at 12 V dissipates 144 mW: more than an 0805's 125 mW, so a 1206 at least, and a 1210 with the half-rating rule below.

Temperature. The rating holds only while the air around the part is at 70 °C or cooler. Above that the part can shed less heat, and the allowed power falls in a straight line to nothing at 155 °C, the maximum film temperature — 125 °C for 0201 and 01005. A common rule is to use no more than half the rating, which leaves room for a warm enclosure and for the board around the part heating it too.

Voltage. Each size also has a maximum working voltage, set by the gap between its terminals. For high resistance values this limit is reached before the power one:

```
V = √(P × R), or the maximum working voltage, whichever is lower
```

An 0603 (1/10 W, 75 V) at 1 MΩ would reach its power rating at 316 V, but may only see 75 V, which is 75^2 / 1 MΩ = 5.6 mW. For high-voltage dividers the voltage rating, not the wattage, sets the size — or several resistors in series share the voltage.

Assumptions and limits

- **Ratings depend on the series, not only the size.** The table gives the conventional general-purpose figures; anti-surge, high-power and some newer thick-film series rate the same package two or four times higher. The part's own datasheet has the last word.
- **The board matters.** An SMD rating assumes the copper pads and board carry heat away as in the maker's test. Small pads, thin copper or neighbouring hot parts lower it.
- **Through-hole sizes vary by maker and technology**: a metal-film 1/4 W part and a carbon-film one are not the same size. The ranges are for recognising a part, not for a footprint.
- Pulses are a separate question: a short pulse can exceed the continuous rating, within limits the datasheet gives as a pulse curve.
- **The size check uses the table's figures**: the Yageo ratings for chips, each with its maximum working voltage, and the wattage class for leaded parts, whose voltage rating is not in the table and depends on the part. "Half its rating" is the usual margin for a warm enclosure and hot neighbours; the full rating holds only up to 70 °C around the part, on the maker's pad layout.
- Above 200 V no chip in the table will do; several in series share the voltage, each taking its part.

What it deliberately does not do

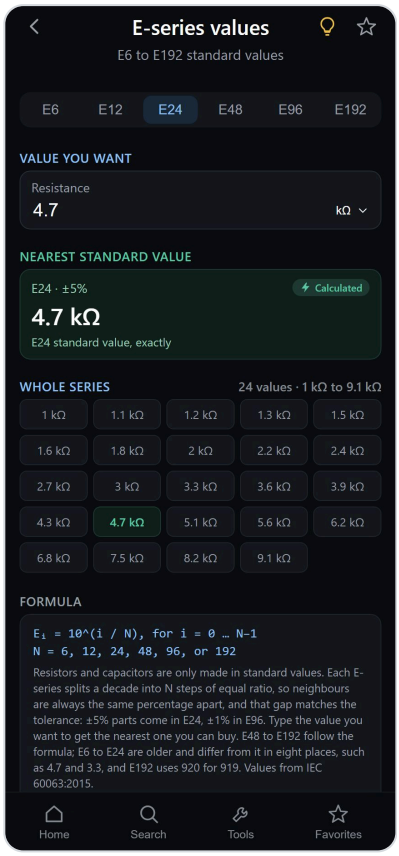
- **Package dimensions in detail** (pad sizes, heights) are the SMD package sizes tool.
- **Temperature rise** from a given power and board is a thermal calculation, not a rating lookup.

How it was checked

Every SMD power rating, working voltage and the 125 °C / 155 °C limits against the Yageo RC_L datasheet. The review corrected the 1210's size in inches, which read 0.12″ × 0.125″, and the 0805's width to 1.25 mm; the SMD package sizes review later aligned all the sizes to makers' datasheets (1210: 0.126″ × 0.098″); it also withdrew the claim that SMD ratings are standard across manufacturers, which the Vishay datasheet contradicts. The maximum working voltages were added then.

The size check was added in the re-walk. Its reference cases: 1 kΩ at 12 V is 144 mW, a 1206 at full rating, a 1210 or a 1/2 W leaded part at half; a 0.1 Ω shunt at 2 A is 0.4 W, a 1210 or at half a 2512; 1 MΩ at 100 V is only 10 mW but needs an 0805, the first chip rated over 75 V. The re-walk also corrected this section's own example, which had said 144 mW fits "an 0805 at least" when an 0805 is rated 125 mW.

E-series values



What it computes

The standard value nearest to the one you want, in the series you choose (E6 to E192), how far off it is in percent, and the whole series across the decade of your value, with the nearest one highlighted.

Source

IEC 60063:2015, *Preferred number series for resistors and capacitors*. It lists every series value by value, and its tolerances: E6 ±20%, E12 ±10%, E24 ±5%, E48 ±2%, E96 ±1%, E192 ±0.5% and better.

What it means

Resistors and capacitors are not made in every value. It would be impossible to stock them all, and pointless: a ±5% 1000 Ω resistor can be anything from 950 to 1050 Ω, so a separate 1010 Ω part would overlap it completely. Instead, manufacturers make a fixed set of **preferred values**, the E-series.

Each series splits a decade — 1 to 10, 10 to 100, and so on — into a fixed number of steps: 6 for E6, 24 for E24, 96 for E96. The steps are not equal in ohms but equal in **ratio**, so each value is the same percentage above the previous one, whatever the decade. And the number of steps is chosen so that the gap between neighbours matches the tolerance: in E24 each value is about 10% above the last, so ±5% parts either side just meet. That is why the tolerance tells you the series — ±5% parts come in E24 values, ±1% in E96.

In practice: pick the tolerance you need, look up the nearest value in its series, and check whether the percent difference matters for your circuit. If it does, a tighter series has a closer value, or two parts in series or parallel can make up the difference.

How the values are worked out

$$E_i = 10^{(i / N)}, \quad i = 0 \ldots N-1, \quad \text{rounded to 2 figures (E6–E24) or 3 (E48–E192)}$$

N is the number of values in the series. E48, E96 and E192 follow this formula, with one exception: the standard lists 920 in E192 where the formula gives 919. E6, E12 and E24 are older than the formula and were chosen by hand; eight E24 values differ from it — 2.7, 3.0, 3.3, 3.6, 3.9, 4.3, 4.7 and 8.2 — which is why 4.7 kΩ, not 4.6 kΩ, is the value everyone knows. The tool uses the standard's lists, not the formula, for those.

The nearest value is the one with the smallest percentage difference from what you typed. The top of a decade counts as the start of the next one, so 97 kΩ in E24 gives 100 kΩ, not 91 kΩ.

Assumptions and limits

- **E3** (±40%: 1.0, 2.2, 4.7) exists in the standard but is rarely used for new parts and is not offered.
- **Exactly between two values** the lower one is given; both are equally far off.
- **One tolerance, one series.** Throughout the app a tolerance picks exactly one series: ±1% means E96, and the sliders and hints step through E96 only. Manufacturers also sell the E24 values at ±1% — Yageo's RC_L datasheet lists "1% (E24/E96)" — so a 1% 4.7 kΩ part is easy to buy even though the app suggests 4.75 kΩ. The simpler rule is kept on purpose.
- **Capacitors** follow the same series, but in practice ceramic capacitors are mostly stocked in E6 or E12 values, and electrolytics in E3 or E6.
- The table shows the decade of the nearest value, so the value found is always highlighted in it: for 97 kΩ in E24 that is the 100 kΩ decade.

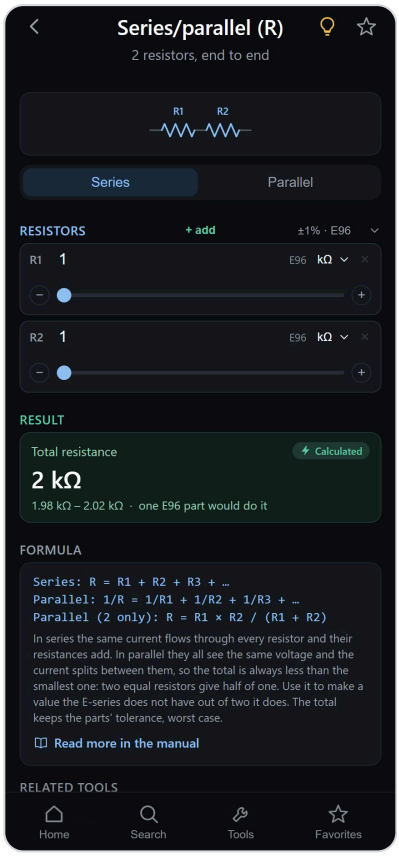
What it deliberately does not do

- **Combining two standard values** to hit an exact one is a series/parallel question, the Series/parallel tool's job.

How it was checked

Every E48 and E96 value against the IEC 60063 list, and E192 against the same list. The review found the E192 exception missing: the tool showed 919 where the standard has 920, so 9.2 kΩ was reported as not standard; it now uses 920. The E6, E12 and E24 lists, including the eight values that differ from the formula, match the standard. The table also showed the decade of the typed value, so a nearest value in the next decade (97 kΩ → 100 kΩ) was not highlighted; it now shows the nearest value's decade.

Series/parallel (R)



What it computes

The total resistance of two to four resistors connected in series or in parallel, the range it can take given the parts' tolerance, and the nearest single standard part to that total. Each resistor has a slider that steps through standard values.

Source

Ohm's law and Kirchhoff's laws; no standard is needed beyond them. The standard values come from IEC 60063, as in the E-series tool.

What it means

In series — resistors connected end to end, so the same current has to pass through each of them in turn. Each one drops a voltage, and the drops add up, so the resistances add:

```
R = R1 + R2 + R3 + …
```

The total is always more than the largest one.

In parallel — resistors connected side by side between the same two points, so each sees the same voltage and the current divides between them. Each gives the current another path, so the total is always less than the smallest one. It is the conductances, 1/R, that add:

```
1/R = 1/R1 + 1/R2 + 1/R3 + …
R = R1 × R2 / (R1 + R2)       for two
```

Two equal resistors in parallel give half of one; ten in parallel, a tenth. A resistor much larger than another barely changes it: 1 kΩ in parallel with 1 MΩ is 999 Ω.

Why combine them. The common reason is to get a value the E-series does not stock from two it does — 1 kΩ in parallel with 4.7 kΩ is 824.6 Ω, within a tenth of a percent of the E96 value 825 Ω. Others: to share the power or the voltage between several parts, each within its own rating.

Tolerance of the total

If every resistor is off by the same fraction in the same direction, the series sum and the parallel combination are both off by exactly that fraction. So in the worst case the total has the same tolerance as the parts, no better and no worse, and that is the range the tool shows. In practice the errors of separate parts are

independent and partly cancel, so the total is usually closer than that — but a worst-case design should not count on it.

Assumptions and limits

- **Up to four resistors**, all in series or all in parallel. Mixed networks (two in series, in parallel with a third) are worked out in steps: combine one group, then use its total as a single resistor.
- **A zero-ohm resistor in parallel** shorts the group, and the total is 0.
- Wire and contact resistance are ignored, which matters only for milliohm values.

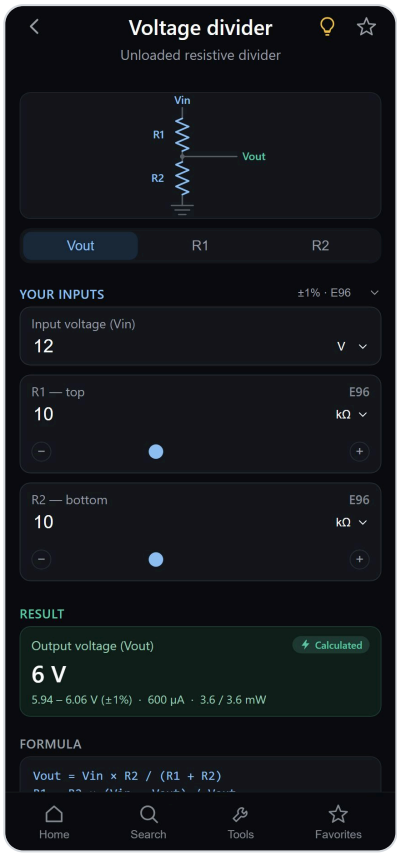
What it deliberately does not do

- **Capacitors and inductors** combine the other way round and have their own tools.
- **Finding the best pair** for a target value automatically is not done; the sliders make trying pairs quick.

How it was checked

By hand: 1 k + 2.2 k = 3.2 kΩ; 1 k ‖ 1 k = 500 Ω; 1 k ‖ 4.7 k = 824.6 Ω, nearest E96 825 Ω; a third 1 kΩ in parallel gives 451.9 Ω; 1 k ‖ 0 = 0 Ω. The tolerance ranges are the total × (1 ± t).

Voltage divider



What it computes

For two resistors in series across a voltage, any one of the output voltage, the top resistor R1 or the bottom resistor R2, from the other three. It also gives the output's worst-case spread from the resistors' tolerance, the current through the pair and the power in each resistor, and when a resistor is solved for, the nearest standard value and the output that value would give.

Source

Ohm's law applied to two resistors in series; standard values from IEC 60063.

What it means

A voltage divider makes a smaller voltage out of a larger one. Two resistors are connected in series from the input voltage to ground. The same current flows through both, so each drops a share of the voltage in proportion to its resistance, and the point between them — the output — sits at a fixed fraction of the input:

```
Vout = Vin × R2 / (R1 + R2)
```

Equal resistors give half. A 10 kΩ over a 4.7 kΩ gives 4.7 / 14.7, about a third. Typical uses: bringing a 12 V signal down to what a 3.3 V ADC can read, setting a reference voltage, or biasing a transistor.

Solved for either resistor:

```
R1 = R2 × (Vin − Vout) / Vout
R2 = R1 × Vout / (Vin − Vout)
```

Choosing the values. Only the ratio sets Vout, so 1 kΩ/1 kΩ and 1 MΩ/1 MΩ both halve the voltage. What the size decides is the current: small values waste power (12 V across 2 kΩ is 6 mA, 72 mW), large ones make the output weak, easily pulled down by whatever it feeds and more sensitive to noise. Tens of kΩ is a common middle ground for signals.

Tolerance

The output is furthest off when the two resistors miss in opposite directions: R2 high and R1 low pushes Vout up, the reverse pulls it down.

```
Vout max = Vin × R2(1+t) / (R1(1−t) + R2(1+t))
Vout min = Vin × R2(1−t) / (R1(1+t) + R2(1−t))
```

So a divider built from ±1% parts is not a ±1% divider: 10 kΩ over 4.7 kΩ at ±1% spreads by ±1.37%. The spread is widest when the output is a small fraction of the input, and approaches ±2t.

Loading

The formula assumes nothing draws current from the output. Anything that does — an ADC input, a transistor base, a meter — is a resistor in parallel with R2, and lowers the output. Seen from the output, the divider behaves like a source with an internal resistance of R1 ‖ R2, so a load RL lowers Vout by roughly

```
(R1 ‖ R2) / RL
```

1% for a load a hundred times R1 ‖ R2, 10% for one ten times. The fix is lower divider resistances or a buffer (the op-amp voltage follower).

Assumptions and limits

- **Unloaded**, as above.
- **Positive voltages.** For a negative supply, enter both voltages as positive values; the ratio and the resistors are the same.
- Vout must be between 0 and Vin when solving for a resistor; a divider can only divide.
- The standard value offered is from the one series the chosen tolerance implies (±1% → E96).

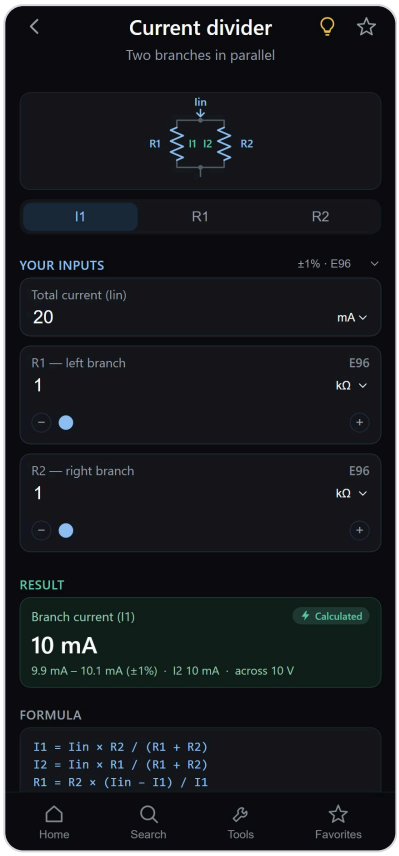
What it deliberately does not do

- **A load resistor** is not an input; the loading rule above covers it.
- **Adjustable dividers** (a potentiometer with fixed end resistors) are not modelled.

How it was checked

By hand: 12 V over 10 k/10 k gives 6 V, 5.94–6.06 V at ±1%, 600 µA, 3.6 mW each. 10 k over 4.7 k gives 3.837 V, 3.78–3.89 V (±1.37%). R1 for 12 V to 3.3 V with R2 = 10 kΩ is 26.36 kΩ, nearest E96 26.1 kΩ giving 3.32 V. R2 for 5 V to 3.3 V with R1 = 10 kΩ is 19.41 kΩ, E96 19.6 kΩ giving 3.31 V. Vout equal to Vin, or below zero, is refused with a reason.

Current divider



What it computes

For a current flowing into two resistors in parallel, any one of the current in the first branch I1, R1 or R2, from the other three. It also gives the current in the second branch, the voltage across both, the spread of I1 from the resistors' tolerance, and when a resistor is solved for, the nearest standard value and the current it would give.

Source

Ohm's law and Kirchhoff's current law; standard values from IEC 60063.

What it means

When a current reaches two resistors connected in parallel, it splits: part goes through each branch and the two parts add back up to the whole (Kirchhoff's current law). Both branches have the same voltage across them, so by Ohm's law each takes a current inversely proportional to its resistance — the easier path takes more:

```
I1 = Iin × R2 / (R1 + R2)
I2 = Iin × R1 / (R1 + R2)
```

This is the voltage divider turned inside out, and the place people trip: the current in branch 1 is set by the *other* resistor, R2. With 20 mA into 1 kΩ in parallel with 2 kΩ, the 1 kΩ branch takes two thirds, 13.3 mA, and the 2 kΩ branch one third, 6.7 mA.

Solved for a resistor:

```
R1 = R2 × (Iin − I1) / I1
R2 = R1 × I1 / (Iin − I1)
```

Where it is used. Shunts that extend a meter's range (a small resistor in parallel takes most of the current, the meter a known fraction), sharing current between parallel parts, and understanding why a low-value resistor in parallel with a circuit steals most of its current.

Tolerance

As with the voltage divider, I1 is furthest off when the two resistors miss in opposite directions, so the spread of I1 can exceed the parts' own tolerance. When I1 is the larger share, it is less sensitive: 1 k ‖ 2 k at ±1% moves I1 by only ±0.66%, because most of the error lands in the smaller branch current.

Assumptions and limits

- **The total current is taken as given**, as if from a current source. In a real circuit fed from a voltage, Iin itself depends on R1 ‖ R2.
- **Two branches.** For more, combine all but one into a single resistance first with the Series/parallel tool.
- I1 must be between 0 and Iin when solving for a resistor.
- The standard value offered is from the one series the chosen tolerance implies.

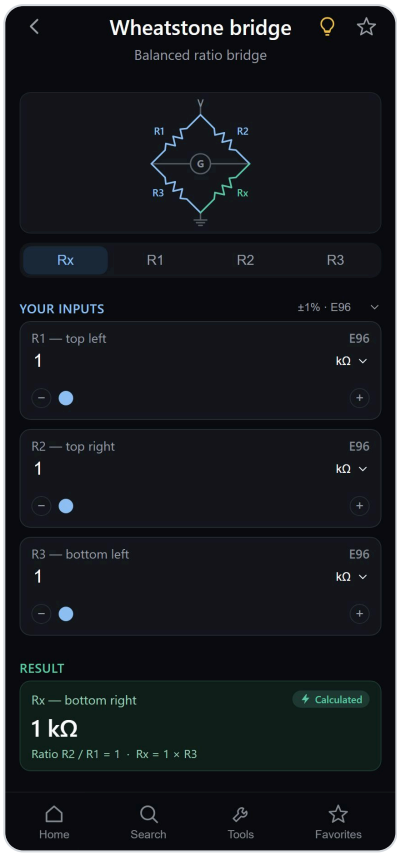
What it deliberately does not do

- **Power in each branch** is not shown; the voltage across the branches is, and $P = V^2 / R$ from there.

How it was checked

By hand: 20 mA into 1 k ‖ 1 k is 10 mA each at 10 V; into 1 k ‖ 2 k, 13.33 mA and 6.67 mA at 13.3 V, I1 between 13.2 and 13.4 mA at ±1%. R1 for 15 mA of 20 mA with R2 = 1 kΩ is 333.3 Ω, E96 332 Ω. R2 for 5 mA of 20 mA with R1 = 1 kΩ is 333.3 Ω. I1 above Iin is refused. The review labelled the voltage in the result line, which had been a bare "10 V" with nothing to say what it was.

Wheatstone bridge



What it computes

For a balanced Wheatstone bridge, any one of its four resistors from the other three. Solving for the unknown Rx, it also gives the ratio R2 / R1 that multiplies R3 into Rx; solving for one of the fixed arms, the nearest standard value.

Source

The balance condition follows from Ohm's law applied to the two dividers that make up the bridge. The circuit is Samuel Hunter Christie's (1833), popularised by Charles Wheatstone.

What it means

A Wheatstone bridge measures an unknown resistance by comparing it with known ones, instead of measuring a current and a voltage.

Four resistors form a diamond. A supply V is connected across the top and bottom corners, so the left side (R1 over R3) and the right side (R2 over Rx) are two voltage dividers on the same supply. A sensitive meter — a galvanometer, G — joins the midpoints of the two sides.

If the two midpoints are at the same voltage, no current flows through the meter and it reads zero. The bridge is then **balanced**, and that happens when both dividers have the same ratio:

```
R1 / R2 = R3 / Rx      so      Rx = R2 × R3 / R1
```

The supply voltage and the meter's own characteristics drop out entirely: the only thing that matters is whether the meter reads zero, which even a crude meter can tell very precisely. That is what made the bridge the standard way to measure resistance accurately.

In use. R1 and R2 are fixed and set the range: with R2 / R1 = 10, Rx is ten times R3. R3 is a calibrated adjustable resistor. Turn R3 until the meter reads zero, and Rx = 10 × R3.

The same circuit, left unbalanced, is how strain gauges, pressure sensors and many thermometers are read: a small change in one arm produces a small voltage across the meter's position, measured by an amplifier.

Assumptions and limits

- **Balance only.** The tool gives the resistance that balances the bridge; it does not compute the voltage or current through the meter when the bridge is off balance.

- **The precision is that of the known arms.** Rx can be no more accurate than R1, R2 and R3 combined; with ±0.1% parts, roughly ±0.3% worst case.
- The arm used as a divisor cannot be zero: R1 when solving Rx, Rx when solving R1, R3 for R2 and R2 for R3.

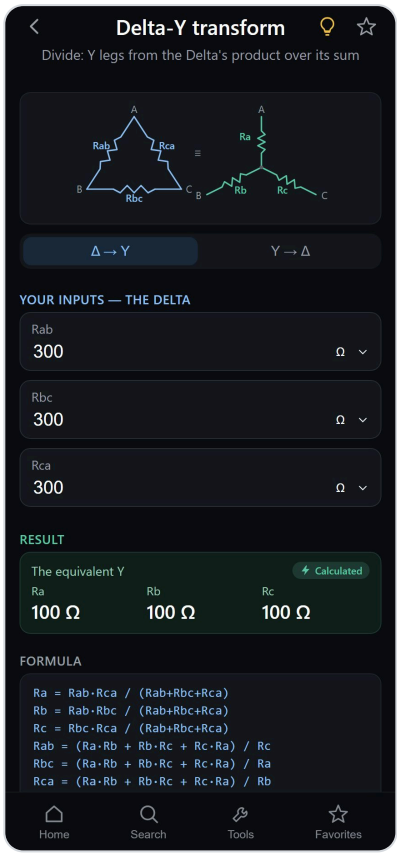
What it deliberately does not do

- **Unbalanced output** for sensor bridges (the voltage across G as a function of a changing arm) is a different calculation, not done here.
- AC bridges for capacitance and inductance (Maxwell, Schering, Wien) are not covered.

How it was checked

By hand: R1 = 1 k, R2 = 2 k, R3 = 3 k balance with Rx = 6 kΩ, ratio 2; each of the other three arms solved from the remaining ones gives back 1 k, 2 k and 3 k. R1 = 0 is refused. The review replaced the "nearest standard value" once shown for Rx, which is the resistance being measured rather than a part to buy, with the bridge ratio; dropped "(unknown)" from Rx's label, which stayed even when Rx was an input; and made the supply's V label neutral, which had taken R1's colour.

Delta-Y transform



What it computes

Converts three resistors connected as a triangle (Delta, Δ) into the three of an equivalent star (Y), or the reverse, so that the two behave identically between their three terminals.

Source

Arthur Edwin Kennelly's transform (1899), derived from requiring the resistance between each pair of terminals to be the same in both networks.

What it means

Three resistors can be connected to three terminals A, B and C in two ways:

- **Delta (Δ)** — a triangle. One resistor between each pair of terminals: Rab, Rbc, Rca.
- **Y (star, also called T)** — three resistors meeting at a common centre point, one leg to each terminal: Ra, Rb, Rc.

Seen only from the three terminals, a Delta and a suitably chosen Y cannot be told apart: the resistance between any two terminals is the same, whatever else is connected. So one can always be replaced by the other.

Why that is useful. Many networks are neither series nor parallel anywhere — the classic case is a bridge, five resistors in a diamond with one across the middle. Replacing one of its triangles by the equivalent star turns it into ordinary series and parallel combinations that can be simplified step by step. The same transform turns up in three-phase power, where loads are wired in delta or in star.

The formulas

Delta to Y — each leg of the star is the product of the two Delta sides that touch its terminal, divided by the sum of all three:

```
Ra = Rab × Rca / (Rab + Rbc + Rca)
Rb = Rab × Rbc / (Rab + Rbc + Rca)
Rc = Rbc × Rca / (Rab + Rbc + Rca)
```

Y to Delta — each side of the triangle is the sum of the products of the legs taken in pairs, divided by the leg *opposite* that side:

```
Rab = (Ra·Rb + Rb·Rc + Rc·Ra) / Rc
Rbc = (Ra·Rb + Rb·Rc + Rc·Ra) / Ra
Rca = (Ra·Rb + Rb·Rc + Rc·Ra) / Rb
```

A way to remember them: Delta to Y divides, so the star's legs are smaller; Y to Delta multiplies, so the triangle's sides are larger. With three equal parts the Y is a third of the Delta: a 300 Ω triangle is a 100 Ω star.

Assumptions and limits

- **All three must be greater than zero.** A zero-ohm side or leg joins two terminals directly, and the network is no longer a true three-terminal one.
- **Resistance only.** The transform holds just as well for impedances (capacitors and inductors, in AC circuits), but this tool works in ohms.

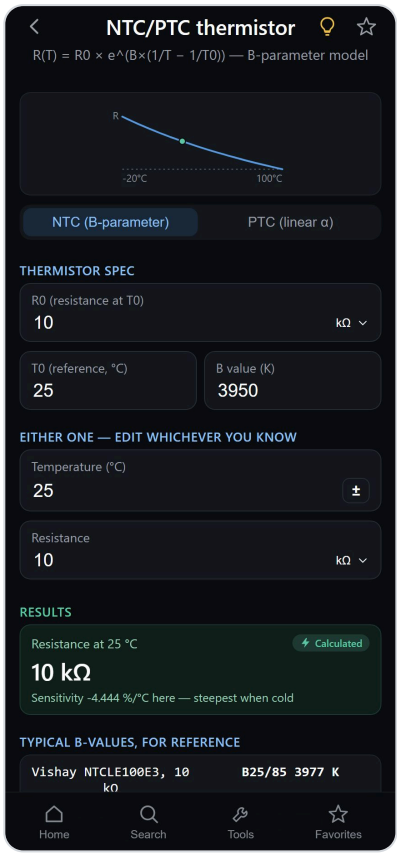
What it deliberately does not do

- **Solving a whole network** (such as the full bridge) is not automated; the tool does the one step that makes the rest ordinary.

How it was checked

By hand: a 300 Ω Delta gives a 100 Ω Y, and back. Rab = 10, Rbc = 20, Rca = 30 Ω give Ra = 5, Rb = 3.333, Rc = 10 Ω, and those three converted back give 10, 20 and 30 Ω.

NTC/PTC thermistor



What it computes

For a temperature-sensing resistor, its resistance at a given temperature or the temperature for a measured resistance, from the datasheet's reference resistance and coefficient; and its sensitivity, in percent per degree, at that point. A curve from −20 to 100 °C shows where the point sits.

- **NTC** — the B-parameter model, used for ordinary NTC thermistors.
- **PTC** — a linear model, for silicon (KTY) sensors and, approximately, platinum RTDs.

Source

- The B-parameter equation, as NTC datasheets define it. The Vishay NTCLE100E3 datasheet defines B as ln(R1/R2) / (1/T1 − 1/T2) and gives B25/85 = 3977 K for its 10 kΩ part, with a full resistance-temperature table, which is what the tool was checked against.
- The linear PTC model is the definition of a temperature coefficient α.

What it means

A **thermistor** is a resistor whose resistance changes strongly with temperature, made to be used as a temperature sensor. Put it in a voltage divider, read the voltage with an ADC, work out the resistance, and from that the temperature.

- **NTC** (negative temperature coefficient): resistance falls as temperature rises, steeply — about 4% per °C near room temperature for a typical part. A 10 kΩ NTC is 10 kΩ at 25 °C, about 33 kΩ at 0 °C and about 1 kΩ at 85 °C.
- **PTC** (positive temperature coefficient): resistance rises with temperature. The sensors modelled here rise steadily; a KTY silicon sensor by about 0.7% per °C, a Pt100 platinum RTD by 0.385% per °C.

The datasheet gives two numbers that define the part:

- **R0 at T0** — its resistance at a reference temperature, almost always 25 °C. "10 kΩ NTC" means 10 kΩ at 25 °C.
- **B** (for an NTC), in kelvin — how steep the curve is. A higher B means a larger change per degree. Or **α** (for a linear PTC) in percent per degree.

The formulas

```
NTC:  R = R0 × e^(B × (1/T − 1/T0))
      T = 1 / (1/T0 + ln(R/R0) / B)
PTC:  R = R0 × (1 + α × (T − T0))
```

T and T0 are in kelvin (°C + 273.15) in the NTC equations. The sensitivity is the slope of the curve as a percentage: $-B / T^2$ for the NTC, which grows steeper the colder it is, and simply α for the linear PTC.

How good the B model is

B is measured between two temperatures, written as a suffix: **B25/85** is worked out from the resistance at 25 °C and at 85 °C, B25/50 from 25 and 50 °C. The model then goes exactly through those two points and bends away from the real curve outside them. Checked against the Vishay 10 kΩ part's own table, with B25/85 = 3977 K:

Temperature	Datasheet	B model	Difference
−40 °C	332.1 kΩ	412 kΩ	+24%, about 3.5 °C
−20 °C	96.4 kΩ	107 kΩ	+11%, about 2 °C
0 °C	32.6 kΩ	33.9 kΩ	+4%, under 1 °C
85 °C	1.07 kΩ	1.07 kΩ	exact
100 °C	677 Ω	685 Ω	+1%

So: use the B value whose temperature pair covers the range you measure, and for accuracy well below 0 °C use the datasheet's table or the Steinhart-Hart equation, which fits three points instead of two.

Assumptions and limits

- **Self-heating is ignored.** The current used to measure a thermistor also warms it. Datasheets give a dissipation factor (a few mW/°C); keep the measuring current small enough that it adds a fraction of a degree.
- **The part's own tolerance** — typically ±1 to ±5% on R25 and ±0.5 to ±3% on B — adds to the model error.
- **Switching PTCs** (resettable polymer fuses, ceramic PTC heaters and motor protectors) jump sharply by orders of magnitude at a trip temperature. No single smooth formula describes them, and they are not modelled.
- A platinum RTD is not exactly linear; the linear model is within a few tenths of a degree over 0–100 °C, and the full Callendar-Van Dusen equation is needed beyond that.

What it deliberately does not do

- **Steinhart-Hart** three-coefficient fitting is not offered.

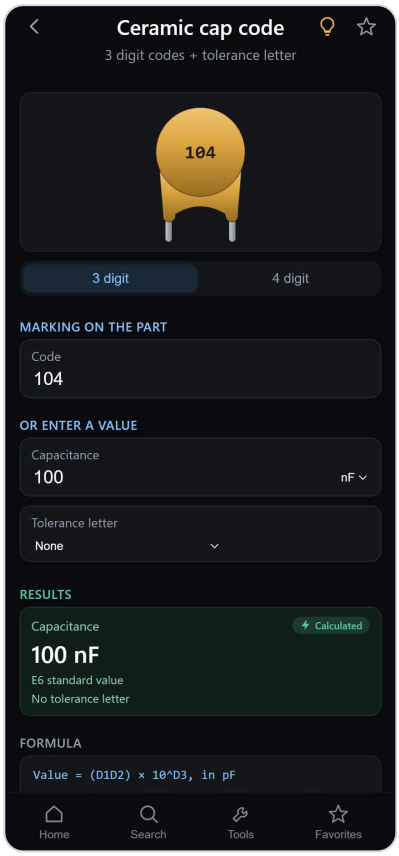
- **The divider around the sensor** (choosing the fixed resistor for best resolution) is the Voltage divider tool's arithmetic.

How it was checked

Against the Vishay NTCLE100E3 datasheet table as above. At 85 °C the model gives 1.07 kΩ, as the datasheet does, and typing 1.07 kΩ back gives 85.0 °C; the sensitivity at 25 °C is −4.47 %/°C. The linear PTC with R0 = 1 kΩ, α = 0.7 %/°C gives 1.525 kΩ at 100 °C.

The review made the temperature field accept negative values on an iPhone, whose number keypad has no minus key; limited the resistance units to Ω, kΩ and MΩ; and replaced the table of "typical" B values, which had no source, with the Vishay figures.

Ceramic cap code



What it computes

The capacitance and tolerance printed on a ceramic capacitor as a three- or four-digit code with an optional letter, and the other way round: the marking for a value. It says whether the value is a standard E-series value.

Source

- **EIA-198** (RS-198), the code for ceramic capacitors: digits in picofarads, with 8 and 9 as the multipliers ×0.01 and ×0.1 for values under 10 pF.
- **IEC 60062:2016**, for R as the decimal point, its three-character code for capacitors (clause 4.3.2) and the tolerance letters (clause 5: relative tolerances, asymmetrical ones such as Z, and absolute ones in pF).

What was read: IEC 60062's capacitor letter code (clause 4.3) and all its tolerance letters (clause 5, tables 6 to 8) in **GOST IEC 60062-2014**, the official Russian edition of IEC 60062:2004, declared identical to it (IDT) and published free by Standartinform; its tables 3 and 4 match the official IEC 2004 preview word for word. The digit code (104 = 100 nF) is given "according to IEC 60062" in Vishay's ceramic capacitor catalogue, which also writes values below 10 pF as 1p0 to 9p1. EIA-198 is not freely available: the 479 form, with 8 and 9 as ×0.01 and ×0.1, is found on parts and in reference guides but was not read in any standard.

What the marking means

Ceramic disc and small film capacitors have no room for "100 nF", so the value is printed as a code, always counted in **picofarads** (pF). 1 nF is 1000 pF, and 1 µF is 1 000 000 pF.

- **Three digits** — the first two are the value's figures, the last is how many zeros follow them. 104 is 10 followed by four zeros: 100 000 pF, which is 100 nF or 0.1 µF — the most common capacitor there is. 472 is 4700 pF, 4.7 nF. 220 is 22 pF, not 220 pF.
- **Small values** — below 10 pF the standard puts p where the decimal point would be: 4p7 is 4.7 pF, 1p0 is 1 pF. Parts are also seen with R in its place (4R7), or with a last digit 9 meaning ×0.1 and 8 meaning ×0.01, the EIA practice: 479 is 47 × 0.1 = 4.7 pF. The tool reads all three and writes the standard's 4p7.
- **Four digits** — three figures and a multiplier, for closer values: 1002 is 10 000 pF, 10 nF.
- **A letter after the digits** is the tolerance:

Letter	Below 10 pF	10 pF and above
B	±0.1 pF	±0.1%
C	±0.25 pF	±0.25%
D	±0.5 pF	±0.5%
F	±1 pF	±1%
G	±2 pF	±2%
J		±5%
K		±10%
M		±20%
Q		−10% / +30%
T		−10% / +50%
S		−20% / +50%
Z		−20% / +80%

Small capacitors use a tolerance in picofarads because a percentage of a few picofarads would be a fraction of the stray capacitance of the leads. Q, T, S and Z are uneven tolerances, for parts whose value is mostly guaranteed not to be too low; Z is typical of cheap high-capacitance ceramics. Some makers use letters of their own outside the standard — Vishay prints Y for −20/+50% and P for −0/+100% — which the tool does not read.

So **473K** is 47 nF ±10%, **104M** 100 nF ±20%, **220J** 22 pF ±5%.

Other markings on the part. A code such as **X7R**, **X5R**, **C0G** or **NP0** is the dielectric, which says how the capacitance changes with temperature and voltage — C0G/NP0 barely at all, X7R by ±15% over −55 to +125 °C, Y5V by much more. A number followed by V, or a code such as 1H or 2A, is the voltage rating. Neither is part of the value code.

Assumptions and limits

- **The value is nominal.** Class 2 ceramics (X7R, X5R, Y5V) lose capacitance with DC voltage applied: a 10 µF X5R part can be well under half of that at its rated voltage. The code says nothing about it; the datasheet's DC-bias curve does.
- **SMD ceramic chips are almost never marked**: they are too small, and the value is only on the reel. The code is mostly seen on through-hole discs and on film capacitors.
- A value typed in is encoded with p below 10 pF (4p7), as the standard writes it; 4R7 and 479 mean the same and are read as well.
- **Standard value.** The tolerance letter picks the one series the value is checked against: J (±5%) E24, K (±10%) E12, M (±20%) E6, as for resistors. With no letter, or an uneven one (Q, T, S, Z), it is E6, the

series most ceramics are stocked in. Below 10 pF, B to G give a tolerance in picofarads, which implies no series, and the tool says so.

What it deliberately does not do

- **Film capacitor markings** such as 4n7 and the voltage codes have the Film code tool; **SMD capacitor** two-character codes have their own tool.

How it was checked

104 → 100 nF, 473K → 47 nF ±10%, 220J → 22 pF ±5%, 106M → 10 µF ±20%, 4R7 → 4.7 pF, 8R2C → 8.2 pF ±0.25 pF.

The review found three faults, now fixed. Codes ending in 8 or 9 were read as powers of ten, so 479 came out as 47 mF instead of 4.7 pF; they now follow EIA-198. B, C and D above 10 pF were read as picofarads, so 101D showed ±0.5 pF instead of ±0.5%. And the units offered ran to millifarads and farads, which no ceramic capacitor reaches; they are now pF, nF and µF. The drawing now shows the marking as typed (479 stays 479 rather than becoming 4R7).

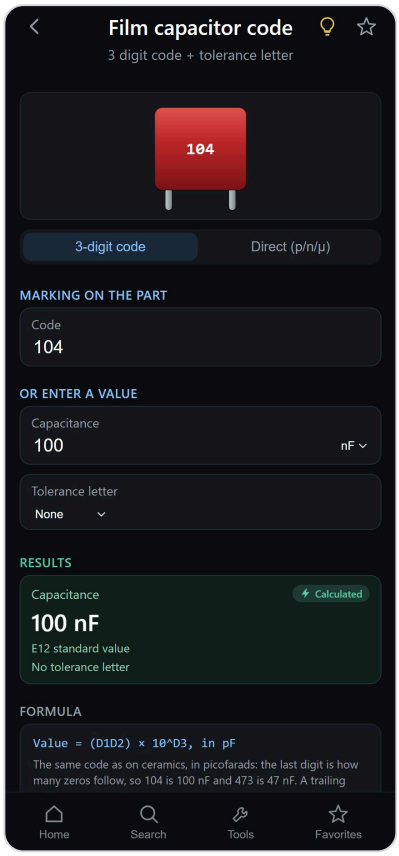
The re-walk checked the value against the tolerance's own series rather than the coarsest one it happened to be in: 182 is "Not in E6 — nearest is 1.5 nF", 182K an E12 value, 479C "no E-series implied".

A reading of the standard's text (GOST IEC 60062-2014, identical to the 2004 edition) and Vishay's catalogue then made four changes: the picofarad letters apply below 10 pF, not at 10 pF itself (100C is ±0.25%, 8p2C ±0.25 pF); the uneven letters Q, T and S were added; values below 10 pF are written 4p7, the standard's form; and 4p7 is read.

Film capacitor code

What it computes

The capacitance and tolerance printed on a film capacitor, in either of the two ways film parts are marked — the 3-digit picofarad code or the letter code (4n7) — and the other way round: the marking for a value.



Source

- **EIA-198** for the 3-digit code, the same as on ceramic capacitors.
- **IEC 60062:2016** for the letter code (the "RKM" code: p, n, µ in place of the decimal point, clause 4.3.1) and the tolerance letters (clause 5).

The letter code (clause 4.3) and the tolerance letters (clause 5, table 6) were read in **GOST IEC 60062-2014**, the official Russian edition of IEC 60062:2004, declared identical to it (IDT) and published free by Standartinform; its tables 3 and 4 match the official IEC 2004 preview word for word. TDK's film capacitor marking guide says its parts use "the coded forms specified in IEC 60062:2004". EIA-198 is not freely available and was not read.

What the marking means

Film capacitors — polyester (PET), polypropylene (PP) and similar, usually a small rectangular box with two leads — carry their value in one of two ways.

The 3-digit code, in picofarads. The first two digits are the figures, the last how many zeros follow: 104 is 100 000 pF = 100 nF, 473 is 47 nF, 102 is 1 nF. As on ceramics, a last digit of 9 or 8 means ×0.1 or ×0.01 for values under 10 pF, though film parts that small are rare.

The letter code. The unit letter stands where the decimal point would be:

- p = picofarad, n = nanofarad, µ (often written u) = microfarad
- 4n7 = 4.7 nF, n33 = 0.33 nF (330 pF), µ1 or u1 = 0.1 µF (100 nF), 2µ2 = 2.2 µF, 100p = 100 pF

It is the same idea as 4k7 for a 4.7 kΩ resistor: no decimal point to rub off or be misread.

The tolerance letter that often follows either form:

Letter	Tolerance
F	±1%
G	±2%
H	±3%
J	±5%
K	±10%
M	±20%

H is where the sources disagree. The standard's text gives H = ±3% (IEC 60062:2004, table 6, in its official identical GOST edition). WIMA, TDK and KEMET use H for ±2.5% in their part numbers and marking guides — TDK while stating it follows IEC 60062:2004 — and WIMA also uses F for ±1.5% and E for ±1%. The tool follows the standard's text, ±3%; on a WIMA, TDK or KEMET part, check the datasheet.

The voltage code. Many film capacitors add a two-character voltage code in front of the value, per the same EIA scheme: 1H = 50 V, 2A = 100 V, 2E = 250 V, 2G = 400 V, 2J = 630 V. So **2A104J** is a 100 V, 100 nF, ±5% part. Others print the voltage plainly (63 V, 400 V=).

Assumptions and limits

- **Plain decimals** such as ".1" or "0.1" on older and larger parts mean microfarads (0.1 µF); the tool reads only the two coded forms.
- **The voltage code is not read** from the marking; type the part of the marking that follows it.
- A value typed in is encoded with the unit that gives the shortest marking; n33 and 330p mean the same.
- **Standard value.** The tolerance letter picks the one series the value is checked against: J (±5%) E24, K (±10%) E12, M (±20%) E6, G and H E48, F E96. With no letter it is E12, since film parts are mostly ±10% and stocked in E12 values.

What it deliberately does not do

- **Ceramic codes with the pF-based tolerance letters** (B, C, D) are the Ceramic code tool; film capacitors are not made small enough to need them.
- **Electrolytic** capacitors print their value and voltage in plain text.

How it was checked

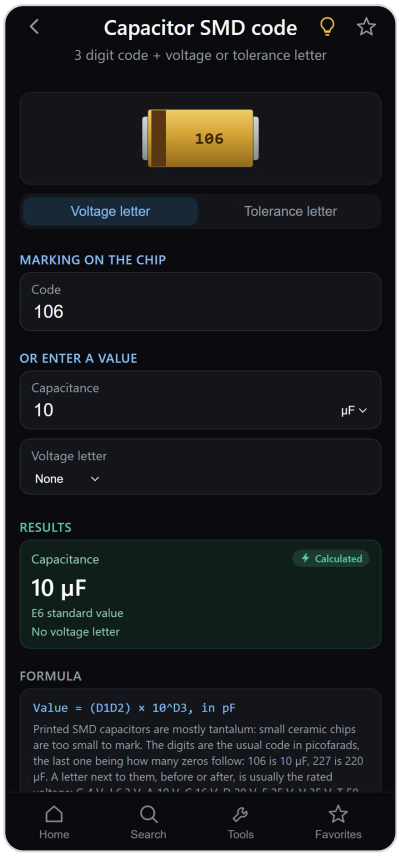
104J → 100 nF ±5%, 473K → 47 nF ±10%, 102H → 1 nF ±3%, 4n7 → 4.7 nF, 4N7J → 4.7 nF ±5%, n33 → 330 pF, u1 and µ1 → 100 nF, 2u2K → 2.2 µF ±10%, 100p → 100 pF.

The review found the tolerance table borrowed from the inductor tool, which had no H, a common film tolerance; added it, first as ±2.5% from WIMA's datasheets, then as ±3% once the standard's own table was read. It made the unit letter case-blind (4N7 was rejected), applied the 8 and 9 multipliers and the R-code

length fix from the ceramic tool, limited the units to pF, nF and µF, and made the drawing show the marking as typed.

The re-walk checked the value against the tolerance's own series: 562 is an E12 value, 912J an E24 one, 562M "Not in E6 — nearest is 4.7 nF". A value read from a marking now shows in its own unit, n33 as 330 pF.

SMD capacitor code



What it computes

The capacitance printed on a surface-mount capacitor, usually a tantalum part, with the letter printed next to it read either as the rated voltage or as the tolerance; and the other way round, the marking for a value.

Source

- The capacitance digits are the EIA-198 picofarad code, as on ceramic parts.
- The marking layout and voltage letters from the **Kyocera-AVX TAJ** tantalum datasheet: "227 A" is 220 µF at 10 V (A = 10 V) on its larger cases; on its small cases the letter sits above the digits, "J" over "106" for 10 µF at 6.3 V. The rest of the voltage letters are the EIA tantalum code as the makers print it; the EIA document itself was not read.
- The **KEMET T491** datasheet lists what its marking holds: polarity band, picofarad code, rated voltage and a date code.

What the marking means

Most small ceramic chip capacitors carry no marking at all — they are too small to print on, and the value is only on the reel. The SMD capacitors that are marked are mostly **tantalum** (and polymer) capacitors, which are larger and come in moulded cases.

The digits are the capacitance in picofarads, as on a ceramic disc: the last digit is how many zeros follow the first two. 106 is 10 000 000 pF = 10 µF; 227 is 220 µF; 475 is 4.7 µF.

The letter is, on most tantalum parts, the **rated voltage** — the most it may see continuously:

Letter	Rated voltage
G	4 V
J	6.3 V
A	10 V
C	16 V
D	20 V
E	25 V
V	35 V
T	50 V

It may be printed after the digits (227A) or before them (J106, often on a line of its own above); the tool reads both. Some parts instead use a **tolerance** letter (K ±10%, M ±20%). The two tables share letters — G and J mean 4 V and 6.3 V in one, ±2% and ±5% in the other — so the marking alone cannot say which is meant; the datasheet can, and the pill chooses.

Polarity. A tantalum capacitor is polarised: the band or bevel at one end marks the **positive** terminal — the opposite of an aluminium electrolytic, whose stripe marks the negative. Fitted backwards, a tantalum can fail short and burn.

Assumptions and limits

- **Derate the voltage.** Tantalum (MnO_2) capacitors are usually run at no more than half their rated voltage, to survive turn-on surges; the letter gives the rating, not the voltage to design for.
- **Makers' own marks** — date codes, logos, ID codes — share the same face and are not read.
- A value typed in is encoded with R below 10 pF; tantalum parts never go that low.
- **Standard value.** A tolerance letter picks the one series the value is checked against (K, ±10%, E12; M, ±20%, E6). A voltage letter says nothing about tolerance, so the value is checked against E6, the series tantalum and polymer capacitors are stocked in.

What it deliberately does not do

- **The two-character SMD capacitor code** (a letter for the figures and a digit for the multiplier, IEC 60062 Annex B) is not read.
- **Ceramic discs and film parts** have their own code tools.

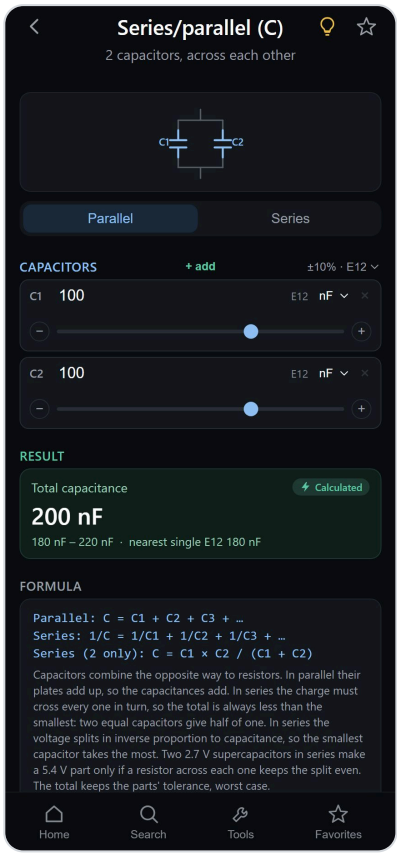
How it was checked

Against the AVX datasheet's own examples: 227A reads 220 µF at 10 V, J106 and J 106 read 10 µF at 6.3 V. Also 475V → 4.7 µF at 35 V, 106K → 10 µF ±10%, 226M → 22 µF ±20%.

The review made the tool read a voltage letter printed before the digits (J106 was rejected), applied the 8 and 9 multipliers and R-code length fix from the ceramic tool, limited the units to pF, nF and µF, and made the drawing show the marking as typed.

The re-walk checked the value against the tolerance's own series (156A is an E6 value, 825K an E12 one), showed a value read from a code in its own unit (471 as 470 pF) and put a space in "10 V".

Series/parallel (C)



What it computes

The total capacitance of two to four capacitors in parallel or in series, the range it can take given their tolerance, and the nearest single standard value. Each capacitor has a slider that steps through standard values.

Source

The charge relation Q = C × V applied to parts sharing a voltage (parallel) or a charge (series); standard values from IEC 60063.

What it means

Capacitors combine the **opposite way to resistors**.

In parallel — side by side between the same two points. Each has the same voltage across it and stores its own charge, so the charges add, and so do the capacitances — it is as if the plates were one larger plate:

```
C = C1 + C2 + C3 + …
```

A 100 nF and a 10 µF in parallel make 10.1 µF. This is how a board gets both a large capacitance for slow supply dips and a small one for fast noise: in parallel they add, and each does the part of the job it is good at.

In series — end to end. The same charge has to appear on every capacitor in turn, and their voltages add up to the total, so it is the reciprocals that add:

```
1/C = 1/C1 + 1/C2 + 1/C3 + …
C = C1 × C2 / (C1 + C2)       for two
```

The total is always less than the smallest one. Two equal capacitors give half of one; 1 µF in series with 10 µF gives 909 nF, a little less than the 1 µF.

Voltage in series

The reason to put capacitors in series is usually voltage: two 2.7 V supercapacitors in series make a part that can be charged to 5.4 V. But the voltage does not split evenly on its own. With the same charge on each,

```
V1 / V2 = C2 / C1
```

so the **smaller** capacitor takes the **larger** share of the voltage. Real capacitors differ by their tolerance and by their leakage, so one of them can be pushed past its rating. Series capacitors that are meant to share a voltage — supercapacitors, electrolytics in a high-voltage supply — get a resistor across each one, or an active balancing circuit, to hold the split even.

Tolerance of the total

As for resistors, if every part is off by the same fraction in the same direction the total is off by exactly that fraction, so the worst-case range is the total × (1 ± t). Many capacitors have wide tolerances — ±10% or ±20% for ceramics and electrolytics, and some ceramics +80/−20% — so the range is often the more important number.

Assumptions and limits

- **Up to four capacitors**, all in series or all in parallel.
- **Nominal values.** Class 2 ceramics lose capacitance with applied DC voltage and with temperature; electrolytics lose it as they age. The total is only as good as the values typed in.
- Leakage, ESR and inductance are ignored.

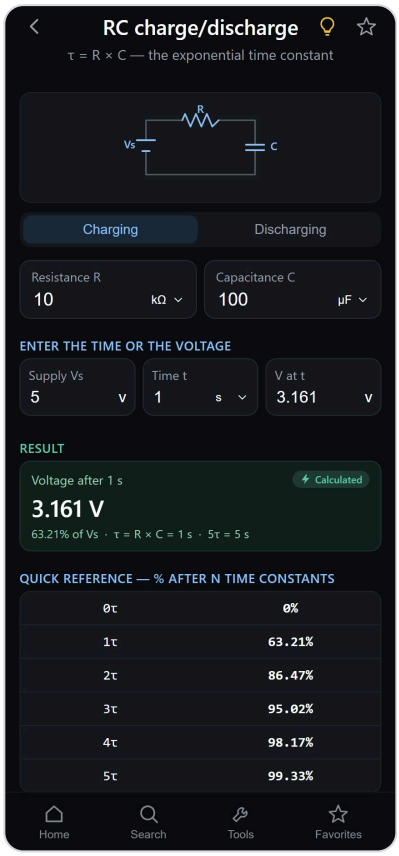
What it deliberately does not do

- **Balancing resistor values** for series capacitors are not calculated.
- **Resistors** have their own series/parallel tool.

How it was checked

By hand: 100 nF + 100 nF in parallel is 200 nF, in series 50 nF; 1 µF and 10 µF in series give 909.1 nF; two 2.7 F supercapacitors in series give 1.35 F. The ranges are the total × (1 ± t).

RC charge and discharge



What it computes

For a capacitor charging from a supply through a resistor, or discharging through one: the time constant τ, the voltage after a given time, or the time it takes to reach a given voltage. A table gives the percentage reached after one to five time constants.

Source

The solution of the circuit's differential equation, from Ohm's law and the capacitor law I = C × dV/dt. No standard is involved.

What it means

A capacitor stores charge, and its voltage rises as the charge builds up. When it charges through a resistor, the current is largest at the start, when the capacitor is empty and the whole supply voltage is across the resistor. As the capacitor fills, less voltage is left across the resistor, the current falls, and the charging slows. So the voltage does not climb in a straight line but bends over, approaching the supply ever more slowly.

The pace is set by one number, the **time constant**:

$$\tau = R \times C$$

10 kΩ and 100 µF make τ = 1 second. In each time constant the capacitor covers 63% of the distance that is still left:

After	Charging, % of Vs	Discharging, % of V0 left
1 τ	63.2%	36.8%
2 τ	86.5%	13.5%
3 τ	95.0%	5.0%
4 τ	98.2%	1.8%
5 τ	99.3%	0.7%

It never quite arrives: mathematically the curve only approaches Vs. In practice **5τ** counts as fully charged or discharged.

Discharging is the same curve upside down: a charged capacitor connected across a resistor, with no supply, loses 63% of what is left in each τ.

The formulas

```
Charging:     V(t) = Vs × (1 − e^(−t/τ))       t = −τ × ln(1 − V/Vs)
Discharging:  V(t) = V0 × e^(−t/τ)             t = −τ × ln(V/V0)
```

The second form of each answers the practical question — how long until the capacitor reaches a threshold. For example, how long a power-on reset holds, or when a slowly rising input crosses a logic level: charging to 2/3 of Vs takes τ × ln 3 = 1.1 τ, which is where the 555 timer's 1.1 R C comes from.

Assumptions and limits

- **An ideal source and capacitor.** The supply is taken as having no internal resistance, and the capacitor as having no leakage. A leaky capacitor, such as an electrolytic, levels off below Vs.
- **Nominal R and C.** With a ±20% capacitor the times are ±20% too.
- The target voltage must lie between 0 and Vs: a charging capacitor never reaches Vs, so asking for exactly Vs has no finite answer.

What it deliberately does not do

- **Switch debouncing and RC timing for logic inputs** have the Debounce / RC timing tool, which adds input thresholds.
- **Energy** stored in the capacitor is the Stored energy tool.

How it was checked

By hand: 10 kΩ and 100 µF give τ = 1 s; at 1 s a 5 V charge reaches 3.161 V (63.21%); 4.5 V is reached at 2.303 s (ln 10 × τ); discharging from 5 V, 1.839 V is left at 1 s, and 0.5 V is reached at 2.303 s.

The review fixed the layout, which put the voltage result 24 px under the tab bar on the target phone; R and C now share a line, the voltage and time fields share the next, and the results sit in one row. The schematic no longer shows a supply when discharging, and the resistance units are Ω, kΩ and MΩ.

Capacitor energy



What it computes

The energy stored in a charged capacitor from its capacitance and voltage, or either of those from the other two; and the charge it holds.

Source

The energy of a capacitor charged from zero: the work done moving charge onto it against its own rising voltage. A standard result; no standard document is involved.

What it means

A capacitor stores energy in the electric field between its plates. Charging it takes work, because each bit of charge has to be pushed on against the voltage already there; that work is held, and comes back out when the capacitor discharges.

```
E = ½ × C × V²      in joules, with C in farads and V i
Q = C × V            the charge, in coulombs
```

The square matters. Twice the voltage stores four times the energy. A 1000 µF capacitor at 12 V holds 72 mJ — a small spark; the same capacitor at 400 V would hold 80 J. This is why the capacitors in mains power supplies, camera flashes and microwave ovens are dangerous long after the power is removed, and why they are discharged through a resistor before being touched.

How much is a joule. One joule is one watt for one second. A 1 F supercapacitor at 2.7 V holds 3.6 J: enough to keep a 10 mW real-time clock circuit running for six minutes.

Rearranged:

```
V = √(2E / C)       the voltage needed to store E in C
C = 2E / V²         the capacitance needed to store E at V
```

Usable energy

A circuit seldom uses all of it. A device powered from a capacitor stops working when the voltage falls below its minimum, so the energy it can draw is the difference between two voltages:

```
E usable = ½ × C × (V start² − V min²)
```

A supercapacitor charged to 5 V feeding a circuit that needs at least 3 V delivers only $1 - (3/5)^2 = 64\%$ of what it holds. Use the tool twice, at the two voltages, and subtract.

Assumptions and limits

- **An ideal capacitor.** Leakage slowly drains the stored energy; ESR turns some of it to heat when it is drawn fast.
- **Nominal capacitance.** Class 2 ceramics hold less than their marked value at high voltage, and electrolytics lose capacitance with age.

What it deliberately does not do

- **Discharge time** through a load is the RC charge and discharge tool's job.
- **Hold-up time** from the usable energy is not calculated directly; it is the usable energy divided by the power drawn.

How it was checked

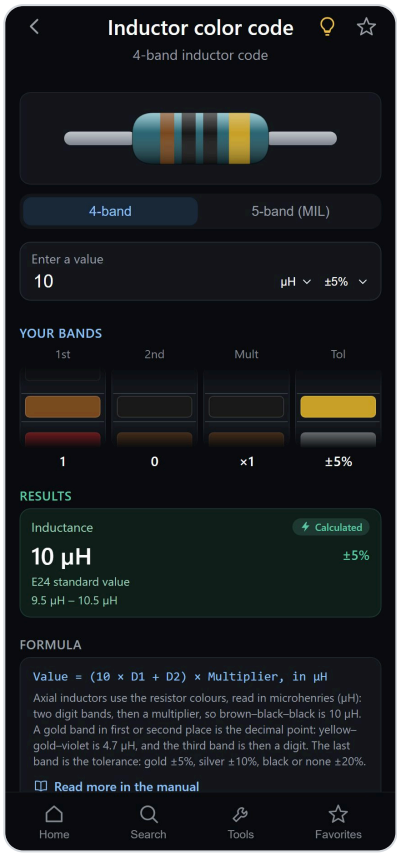
By hand: 1000 µF at 12 V holds 72 mJ and 12 mC; 1 F at 2.7 V holds 3.645 J; storing 1 J in 100 µF needs 141.4 V; storing 10 J at 400 V needs 125 µF.

PASSIVE COMPONENTS › INDUCTORS

Inductor color code

What it computes

The inductance and tolerance that the colour bands on an axial (leaded) inductor stand for, and the other way round: the bands for a value. A second pill handles the 5-band form with the wide silver identifier band.



Source

- **Inductors Inc., colour band guide**: the colours carry the same digits and multipliers as the resistor code, read in microhenries; gold among the digit bands is the decimal point; the tolerance colours are gold ±5%, silver ±10% and black ±20%; the wide silver band is a "military identifier" that does not imply military qualification.
- **Vishay IM series** (MIL-PRF-15305 molded inductors): inductance tolerances of ±1, ±3, ±5, ±10 and ±20%, of which only the last three have a colour.
- IEC 60062, the standard for resistor and capacitor codes, does not cover inductors; the inductor code borrows its colours.

What the bands mean

Small axial inductors look like fat resistors, and use the same colours — but the value is in **microhenries (µH)**, not ohms.

- **Band 1 and 2** — the two digits (black 0, brown 1, red 2 … white 9).
- **Band 3** — the multiplier, how many zeros follow: black ×1, brown ×10, red ×100; gold ×0.1 and silver ×0.01 for small values.
- **Band 4** — the tolerance: gold ±5%, silver ±10%, black or no band ±20%.

So brown–black–black–gold is 10 × 1 = 10 µH ±5%, and red–red–brown–silver is 22 × 10 = 220 µH ±10%.

Gold as the decimal point. A gold band in first or second place is not a digit but the decimal point, and the band after it is then a digit too: yellow–gold–violet is 4.7 µH, gold–yellow–violet is 0.47 µH. The same 4.7 µH can also be written yellow–violet–gold, with gold as a ×0.1 multiplier; both forms are read.

The 5-band form. Some parts start with a wide silver band. It is an identifier in the military style and carries no digit; the four bands after it read as above. It does not by itself mean the part is military-qualified.

Assumptions and limits

- **Tolerance colours are only three.** Precision inductors (±1%, ±2%, ±3%) are not marked with a tolerance colour; their tolerance is on the datasheet or in the part number.

- **Makers vary.** Colour-coded inductors are old enough, and made by enough small makers, that some use their own scheme. When the result looks implausible for the part's size, measure it.
- Inductance alone does not say what current the part can carry; that is on its datasheet.
- **Standard value.** The tolerance band picks the one series the value is checked against: gold (±5%) E24, silver (±10%) E12, black or none (±20%) E6.
- **Units.** The value is shown in nH, µH or mH, whichever reads best: 1000 µH is shown as 1 mH. No colour-coded inductor reaches a henry, so H is not offered.

What it deliberately does not do

- **SMD inductor markings** are the SMD code tool.
- **Resistor bands** are the resistor Color code tool, which uses the full IEC 60062 tolerance colours.

How it was checked

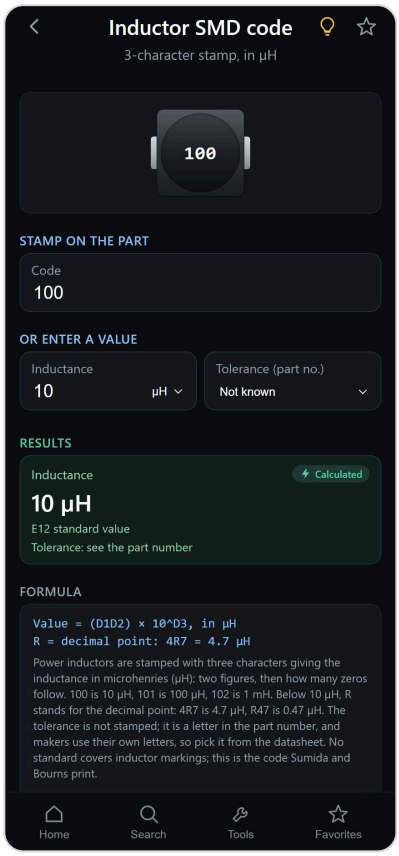
Brown–black–black–gold reads 10 µH ±5%; red–red–brown–silver reads 220 µH ±10%; yellow–gold–violet 4.7 µH; gold–yellow–violet 0.47 µH; typing 4.7 µH gives yellow–violet–gold.

The review found the tolerance list offering the resistor's precision colours, brown ±1% down to grey ±0.01%, which no colour-coded inductor uses; it is now gold, silver, black and none. It added gold as a decimal point in the digit bands, which the tool could not read, and corrected the note's claim that the silver band marks a part as military-qualified.

The re-walk checked the value against the tolerance's own series (4.7 µH with silver is an E12 value), showed a value from the bands in its natural unit (brown–black–red reads 1 mH, not 1000 µH) and dropped H from the units.

PASSIVE COMPONENTS › INDUCTORS

Inductor SMD code



What it computes

The inductance that the three-character stamp on a surface-mount power inductor stands for — and the other way round, the stamp for a value. With the tolerance taken from the part number, it also says whether the value is a standard one.

Source

There is no standard for it. IEC 60062, the marking standard, covers resistors and capacitors only; inductor makers borrow its principle and apply it in microhenries. What the tool follows is what their datasheets show:

- **Sumida CDRH74** lists a "Stamp" for every part: 100 for 10 µH, 101 for 100 µH, 102 for 1.0 mH. The tolerance (M, ±20%) is in the part name, CDRH74NP-100MC, not in the stamp.
- **Bourns SRR1260** shows "100" on the body in its drawing; its tolerances, in the part number, are Y ±30%, M ±20% and K ±10%. It is "available in E12 values".
- **Bourns SRN4018**, the part drawn, shows "4R7" on its core and is 4.0 × 4.0 mm, the core at most 3.6 mm across, 4.55 mm across the terminals.

What it means

An inductor stores energy in a magnetic field; its inductance, in henries (H), says how strongly it opposes a change of current. Power inductors in DC/DC converters are a few hundred nanohenries to a few millihenries, so the stamp counts in **microhenries** (µH): 1 µH is a millionth of a henry, 1000 nH, and 1 mH is 1000 µH.

- **Three digits** — the first two are the figures, the last is how many zeros follow. 100 is 10 and no zeros: 10 µH. 101 is 100 µH, 102 is 1000 µH, 1 mH. 470 is 47 µH, not 470.
- **R** — below 10 µH, R stands where the decimal point is: 4R7 is 4.7 µH, 1R0 1 µH, R47 0.47 µH (470 nH).
- **The tolerance is not stamped.** It is a letter in the part number, and makers do not agree on the letters — Bourns uses Y for ±30%, a letter the resistor and capacitor standard does not have — so the tool asks for the percentage.

How the value is worked out

```
Value = D1D2 × 10^D3, in µH        (100 → 10 × 10⁰ = 10 µH)
xRy   = x.y µH                      (4R7 → 4.7 µH)
```

A stamp holds two figures, so a typed value with more is rounded to what the part could carry, and the tool says so. The shortest stamp is R10, 0.1 µH.

Assumptions and limits

- **Not every maker uses this stamp.** Many small chip inductors carry no marking at all, and some makers print a code of their own or a colour dot. When the reading makes no sense for the part, trust its datasheet.
- **Standard value.** The tolerance picks the one series the value is checked against, as elsewhere in the app: ±5% E24, ±10% E12, ±20% and ±30% E6. With the tolerance not known it is E12, the series Bourns quotes. Power inductors are often sold in E12 values at ±20%, so an E12 value at ±20% can read "Not in E6" and still be a stock part.
- Inductance alone does not say what current the part can carry before its core saturates; that is in the datasheet, as the saturation current.

What it deliberately does not do

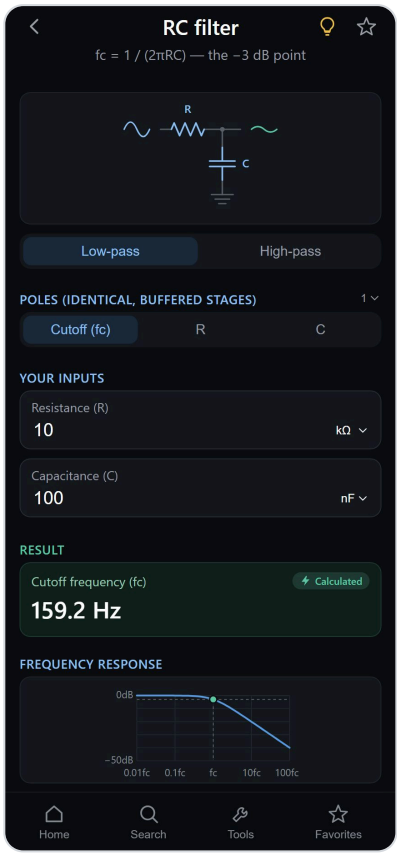
- **Colour-banded inductors** are the Inductor color code tool.
- **A 4-digit code** is not offered: no inductor datasheet read shows one.

How it was checked

Against the stamps in the Sumida CDRH74 datasheet (100 → 10 µH, 101 → 100 µH, 102 → 1 mH) and the Bourns SRR1260 and SRN4018 drawings (100, 4R7). R47 reads 470 nH, 4.7 µH is stamped 4R7, and a typed "10M" is refused with a note that the tolerance letter belongs to the part number.

The review corrected three things. The tool offered a 4-digit mode, which no standard or datasheet shows. It read a tolerance letter as part of the stamp, where datasheets put it only in the part number, and it had no ±30%. And its note said that "100" means 10 nH on some parts, a claim no source supported; it was removed.

RC filter



What it computes

The cutoff frequency of a filter made of one resistor and one capacitor — or the resistor or capacitor for a cutoff you want — as a low-pass or a high-pass, with up to six identical stages. It draws the frequency response, and gives the attenuation and phase shift at any frequency you pick.

Source

No standard is needed: the results follow from Ohm's law and the impedance of a capacitor. The derivation is below.

What it means

A **filter** passes some frequencies and weakens others. A **low-pass** (resistor in series, capacitor to ground) lets slow signals through and weakens fast ones: it smooths PWM into a steady voltage, or removes noise above the band you care about. A **high-pass** (capacitor in series, resistor to ground) does the reverse: it blocks DC and passes the signal riding on it, as a coupling capacitor does in audio.

- **Cutoff frequency, fc** — where the output has fallen to 70.7% of the input (1/√2). Below it a low-pass passes nearly everything; above it, less and less.
- **Decibels (dB)** — a way of comparing two levels on a scale that suits signals: 20 × $\log_{10}$(output ÷ input). −3 dB is 70.7%, −20 dB a tenth, −40 dB a hundredth.
- **Pole** — one RC stage. Past fc a single pole weakens the signal by a further 20 dB for every tenfold increase in frequency (20 dB per decade); two poles 40 dB, and so on.
- **Phase shift** — how far the output's waveform is behind (lag) or ahead of (lead) the input's, in degrees of a cycle. At fc it is 45°.

Why the formulas are these

A capacitor's impedance falls with frequency: Z = 1 / (2πfC). The low-pass is a voltage divider of R on top and C below, so its output is the part of the input across C:

```
Vout / Vin = 1 / (1 + j·f/fc),     fc = 1 / (2πRC)
```

At f = fc the real and imaginary parts are equal, so the magnitude is 1/√2 (−3.01 dB) and the phase −45°. Well above fc the magnitude is fc/f: ten times the frequency, a tenth of the output, −20 dB. The high-pass swaps R and C, and its response is the same with f/fc turned upside down.

Solving fc = 1 / (2πRC) for R or C gives the other two forms the tool uses.

With N identical stages each **buffered** (an op-amp between them, so none loads the next), the responses multiply: the dB figures and the phase are N times one stage's. At each stage's fc the total is already −3N dB, so the system's own −3 dB point moves in:

```
f(−3 dB) = fc × √(2^(1/N) − 1)          (low-pass; high-pass divides instead)
```

For two stages that is 0.644 × fc.

Assumptions and limits

- **Nothing loads the output.** A load in parallel with the output changes the divider: keep it well above R (ten times R or more costs under 10%), or buffer it.
- **The source drives it with little resistance of its own.** A source resistance adds to R and lowers fc.
- **Several poles here means buffered stages.** Unbuffered RC stages in a chain load each other and give a softer knee than this.
- Parts have tolerances: a ±5% R and a ±10% C move fc by up to about 15%.

What it deliberately does not do

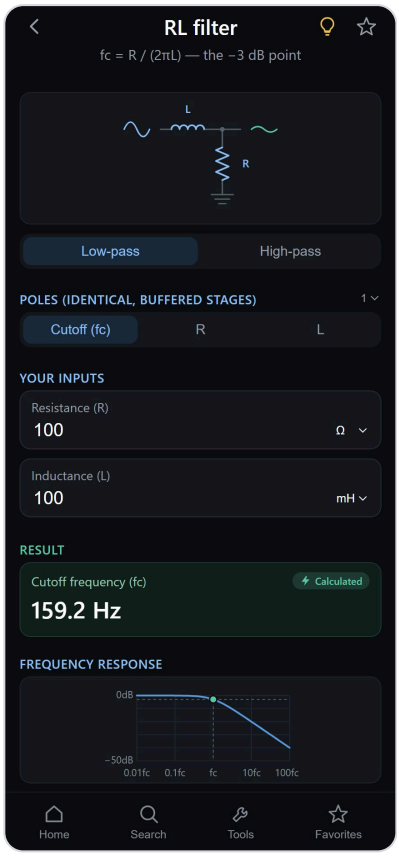
- **RL filters** have their own tool; **op-amp active filters** with gain and sharper shapes (Butterworth, Sallen-Key) are not covered.
- **The time response** — how the output settles after a step — is the RC charge/discharge tool.

How it was checked

10 kΩ with 100 nF gives 159.2 Hz; at fc the attenuation is −3.01 dB and the phase −45°. Two buffered poles put the system's −3 dB point at 102.4 Hz, 0.644 × fc. 10 kΩ with 1 µF gives 15.92 Hz, and −35.97 dB at 1 kHz. A high-pass of the same parts is 2.13 dB down at 20 Hz. fc = 20 kHz with 1 kΩ needs 7.958 nF.

The review found the frequency to explore opening at 159.2 Hz, a rounded fc, which showed −45.01°; it now starts at fc exactly. The units offered were cut to those RC filters use: Ω to MΩ, pF to µF, Hz to MHz.

RL filter



What it computes

The cutoff frequency of a filter made of one resistor and one inductor — or the resistor or inductor for a cutoff you want — as a low-pass or a high-pass, with up to six identical stages; the frequency response, and the attenuation and phase shift at any frequency you pick.

Source

No standard is needed: the results follow from Ohm's law and the impedance of an inductor. The derivation is below.

What it means

It is the RC filter's counterpart with an inductor in place of the capacitor (see that section for what a filter, fc, a decibel, a pole and a phase shift are). An inductor's impedance **rises** with frequency, where a capacitor's falls, so the two parts swap places:

- **Low-pass** — inductor in series, resistor to ground. The inductor blocks the fast signals. Often the "resistor" is the load itself: a loudspeaker in a crossover, a circuit fed through a choke.
- **High-pass** — resistor in series, inductor to ground. The inductor shorts the slow signals to ground and lets the fast ones through, as in RF circuits where the series resistance is the 50 Ω of the system.

Why the formulas are these

An inductor's impedance is Z = 2πfL. The low-pass is a divider of L on top and R below, so its output is the part across R:

```
Vout / Vin = R / (R + j·2πfL) = 1 / (1 + j·f/fc),     fc = R / (2πL)
```

This is the same shape as the RC low-pass, so everything that follows from it is the same: −3.01 dB and −45° at fc, 20 dB per decade beyond, and for N buffered stages a system −3 dB point at fc × √(2^(1/N) − 1). The high-pass swaps R and L and turns f/fc upside down. Solving fc = R / (2πL) for R or L gives the other two forms.

Assumptions and limits

- **A real inductor has resistance.** Its winding's DC resistance (DCR, in the datasheet) adds to R in a low-pass and flattens the high-pass at low frequencies. For small R — an 8 Ω speaker — it matters.

- **Cores saturate.** Past its rated current an inductor with a ferrite or iron core loses much of its inductance, and fc moves up.
- **Every inductor resonates** with its own winding capacitance at its self-resonant frequency (SRF, in the datasheet); well below it the model holds, near it the part stops behaving as an inductor.
- **Nothing loads the output**, and several poles mean buffered stages, as for the RC filter.

What it deliberately does not do

- **RC filters** have their own tool; **LC filters** (both an inductor and a capacitor, 40 dB per decade from one pair) are not covered.
- **Loudspeaker crossover design** beyond one coil — the speaker's impedance is not a pure 8 Ω across the band — is not covered.

How it was checked

100 Ω with 100 mH gives 159.2 Hz; at fc −3.01 dB and −45°. An 8 Ω woofer crossed over at 2.5 kHz needs 509.3 µH. A 10 µH choke into 10 Ω is a 159.2 kHz low-pass, −16.07 dB at 1 MHz. 100 nH to ground in a 50 Ω system is a high-pass at 79.58 MHz.

The review made the same corrections as on the RC filter: the frequency to explore opens at fc exactly, Low-/High-pass take the mode pills, and the resistor units are ohms to megohms. Inductors keep nH to H and frequencies up to GHz, which RF inductors and supply chokes really reach.

PASSIVE COMPONENTS › REFERENCE

SMD package sizes



What it computes

A reference table of the common surface-mount chip sizes: each one's imperial and metric name, its length and width in inches and millimetres, and what it is typically used for. It can be searched by either name or by a dimension.

Source

The body sizes are those manufacturers' datasheets give:

- **Panasonic ERJ** thick-film chip resistors — 01005 to 2512, e.g. 01005 0.40 × 0.20 mm, 0805 2.00 × 1.25 mm, 1210 3.20 × 2.50 mm, 2512 6.40 × 3.20 mm.
- **Vishay D/CRCW e3** (April 2026) — the same sizes, and 1218, 3.2 × 4.6 mm; 2512 is 6.3 × 3.15 mm there.
- **KEMET C1002 X7R** (September 2026) ceramic capacitors — every size in millimetres and in inches (1210: 3.20 mm (0.126″) × 2.50 mm (0.098″)), and 1808, 4.70 × 2.00 mm.
- **Murata LQM2HP** inductors — 1008 (metric 2520), 2.5 × 2.0 mm.

Where makers differ by a few hundredths of a millimetre, the metric name's nominal size is given.

What it means

A surface-mount chip — resistor, capacitor, inductor — is a small block with metal ends, soldered flat onto pads on the board. Its **size code** names its length and width:

- **Imperial** (the usual one in the US and in most catalogues): four digits, two for the length and two for the width, in hundredths of an inch. 0805 is about 0.08″ × 0.05″.
- **Metric**: the same idea in tenths of a millimetre. The 0805 body is metric 2012, about 2.0 × 1.2 mm.

The two collide: **imperial 0402** is 1.0 × 0.5 mm, but **metric 0402** is 0.4 × 0.2 mm, the part imperial calls 01005 — a body six times smaller in area. A bare "0402" in a drawing or a bill of materials is ambiguous until you know which system it is in, which is why every row shows both.

The names are rounded; the table gives the real size. 0805's metric name says 1.2 mm wide, but the parts are 1.25 mm. 1210's name says 0.12″ × 0.10″; the parts are 3.2 × 2.5 mm, 0.126″ × 0.098″. The inches here are the millimetres converted, as KEMET prints them.

Assumptions and limits

- **Sizes are nominal.** Each has a tolerance of ±0.05 mm on the smallest to about ±0.2 mm on the larger ones; the height depends on the part and its value, and is not listed.
- **Some parts use a size but not its exact body**: power inductors and electrolytic capacitors have their own case codes, not these.
- The notes on hand-soldering are practical guidance, not a specification.

What it deliberately does not do

- **Pad (footprint) dimensions** depend on the soldering process and the maker; take them from the part's datasheet or IPC-7351.
- **Power ratings** by size are in the Resistor power rating tool.

How it was checked

Every size against the datasheets above. The review found three faults. 1210 was listed as 0.12″ × 0.125″, where the parts are 0.126″ × 0.098″; the code claimed 1210, 1812 and 2512 were 0.125″ wide by exception, which the datasheets do not support for 1210. 0805 was listed as 1.2 mm wide, the metric name, where the parts are 1.25 mm. And 1806 (4516), found only in online size charts and in no maker's datasheet read, was replaced by 1808, from KEMET's; 1218, a common power-resistor size, was added from Vishay's. The inches in the Resistor power rating table were aligned to the same figures.

ACTIVE & SEMICONDUCTOR DEVICES › DIODES

Diode forward voltage



What it computes

Nothing is calculated: it is a reference. It shows which way round a diode conducts, and lists the forward voltage that common real diodes drop, each at the current its datasheet quotes it for.

Source

- **Vishay 1N4148** (Rev. 1.6, 2024): VF ≤ 1 V at 10 mA; its curves show VF at 25, 75 and 150 °C.
- **Vishay 1N4001–1N4007**: VF ≤ 1.1 V at 1.0 A.
- **Vishay 1N5817–1N5819**: VF ≤ 0.450 / 0.550 / 0.600 V at 1.0 A, and 0.750 / 0.875 / 0.900 V at 3.1 A.
- **Diodes Inc. BAT54**: VF ≤ 240 mV at 0.1 mA, 320 mV at 1 mA, 400 mV at 10 mA, 500 mV at 30 mA, 800 mV at 100 mA.
- **Taitron 1N34A** (germanium): VF ≤ 1.0 V at 5 mA.

These are guaranteed maximums; a typical part drops somewhat less.

What it means

A **diode** lets current through one way only, like a valve. Its two leads are the **anode (A)** and the **cathode (K)**, marked by the band on the body; in the symbol the triangle points from A to K, the way current flows.

- **Forward bias** — the anode more positive than the cathode. Once the voltage across the diode reaches about its **forward voltage, Vf**, it conducts. From then on its voltage hardly changes: a little more voltage means a lot more current. So a diode must never be put straight across a supply; a resistor in series sets the current, taking the rest of the supply's voltage.
- **Reverse bias** — the cathode more positive. The diode blocks, apart from a tiny leakage current, up to its reverse rating (VRRM). Past it the diode breaks down and conducts, which destroys an ordinary diode. A Zener diode is made to work in that region, as a voltage reference.

Vf is not one number. It rises with the current — the BAT54 goes from 0.24 V at 0.1 mA to 0.8 V at 100 mA — and falls as the diode warms, by about 2 mV per °C for silicon. The familiar "0.6–0.7 V" is a rule of thumb for a silicon diode at a few milliamps; a 1N4007 carrying its rated 1 A may drop 1.1 V. That is why each figure here comes with its current.

Which kind to choose. A Schottky drops less (useful where every tenth of a volt counts, as in a supply's reverse-polarity protection) but leaks more when blocking. A silicon rectifier such as the 1N4007 blocks up to 1000 V. A small-signal diode such as the 1N4148 switches fast, at small currents.

Assumptions and limits

- The figures are the datasheets' maximums at 25 °C, at the stated current. Between the listed currents, read the datasheet's VF–IF curve.
- Other makers' versions of the same part numbers have slightly different limits.

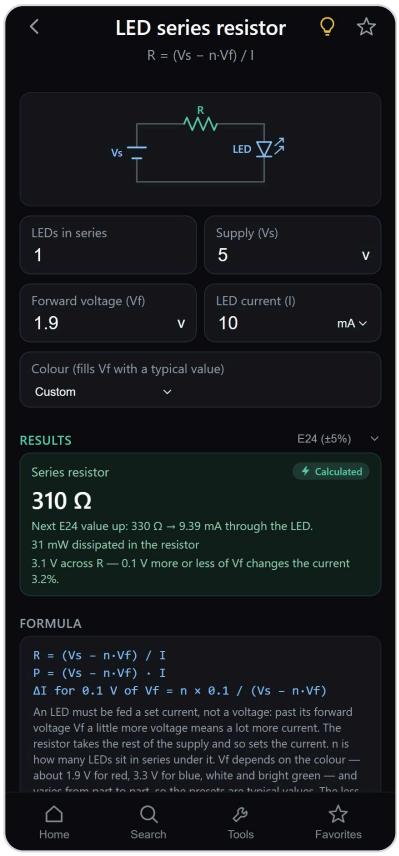
What it deliberately does not do

- **Working out the series resistor** is the LED series resistor tool, which handles any diode's Vf.
- **Zener regulation** will have its own tool.

How it was checked

Every figure against the datasheet named. The review found the table giving bare ranges with no current ("silicon ≈0.6–0.7 V"), which a 1N4007 exceeds at its rated current, and "germanium ≈0.2–0.3 V", which its datasheet does not state; it now gives each part's figure with its current. It also found the drawing putting a diode straight across a battery with no resistor — the circuit that burns a diode out — and the note saying Vf stays "pinned regardless of current"; both were corrected.

LED series resistor



What it computes

The resistor that sets an LED's current from a given supply — for one LED or several in series — the power it dissipates, the next standard value up and the current that value gives, and how much the current moves if the LED's forward voltage is 0.1 V off.

Source

- The formula is Ohm's law, applied to the voltage the LEDs leave.
- The colour presets are typical forward voltages from **Kingbright's WP7113** 5 mm LED datasheets (typical / maximum): red 1.9 / 2.3 V and yellow 1.95 / 2.4 V at 10 mA, standard (GaP) green 2.0 / 2.4 V at 10 mA; bright (InGaN) green 3.3 / 4.1 V, blue 3.3 / 4.0 V and white 3.3 / 4.0 V at 20 mA.
- Infrared from **Vishay's TSAL6200**: 1.35 V typical, 1.6 V maximum, at 100 mA; its forward voltage falls 1.8 mV per kelvin.

What it means

An LED is a diode that gives light (see Diode forward voltage). Like any diode, once past its **forward voltage, Vf**, its current rises steeply with the voltage, so it cannot be fed from a voltage directly: it would take as much current as the supply can give and burn out. A **series resistor** takes up the rest of the supply's voltage and so sets the current:

$$R = (Vs - n \cdot Vf) / I$$

with **n** LEDs in series, each dropping Vf, and **I** the current you want — typically 10–20 mA for an indicator, and the datasheet's maximum is the limit. The resistor turns P = (Vs − n·Vf) × I into heat; choose its power rating from that (see Resistor power rating).

The colour sets Vf. Red and yellow LEDs are about 1.9–2 V; blue, white and the bright green ones about 3.3 V. The old, dimmer green is about 2 V — the tool offers both greens because they differ by more than a volt.

Headroom. The voltage the resistor is left with, Vs − n·Vf, is what fixes the current. Vf varies from one LED to the next (the datasheets give a typical and a maximum a few tenths of a volt apart) and with temperature. With volts to spare, that hardly matters; with a few tenths left, it moves the current a lot. The tool shows how much a 0.1 V change would move it, and flags it in red above 20%. A white LED on 3.3 V has no headroom at all.

Rounding up. The standard value offered is the next one up in the chosen series, never the nearest one down, so the LED gets at most the current asked for.

Assumptions and limits

- Vf is taken as fixed at the current asked. It really rises slightly with current, so the true current differs a little from the figure.
- **Several LEDs in parallel on one resistor** do not share the current evenly: the one with the lowest Vf takes most of it and runs hottest. Give each its own resistor.
- High-power LEDs (hundreds of milliamps) are usually driven by a constant-current regulator rather than a resistor, which would waste too much power.

What it deliberately does not do

- **Brightness** (luminous intensity) is in the LED's datasheet, not calculated.
- **Constant-current drivers** are not covered.

How it was checked

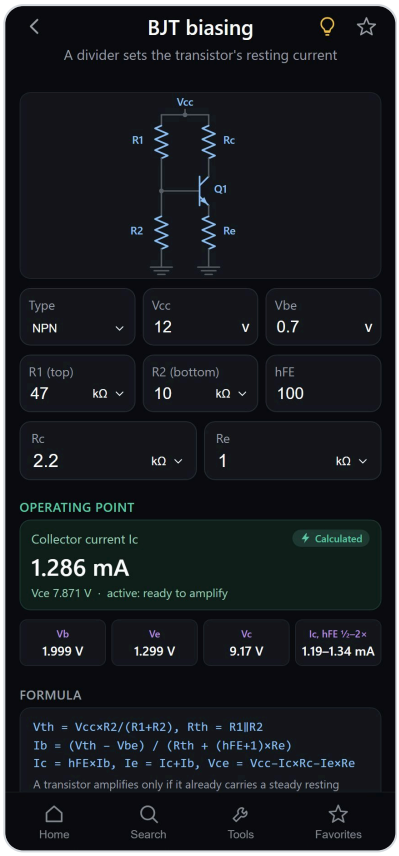
A red LED (1.9 V) on 5 V at 10 mA needs 310 Ω; the next E24 value up, 330 Ω, gives 9.39 mA. 2.0 V at 20 mA from 5 V is exactly 150 Ω, an E24 value. 345 Ω goes up to 360 Ω, not down to 330. Three red LEDs on 12 V at 20 mA need 315 Ω, 330 Ω in E24.

The review found the drawing's LED not joined to the wires either side, the standard value rounded to the nearest (so the current could exceed the one asked), and a green preset of 2.2 V that fits only the old GaP LEDs — a bright green LED is 3.3 V. The presets now follow the datasheets, the value rounds up, and the drawing is laid out like the diode screen's.

ACTIVE & SEMICONDUCTOR DEVICES › TRANSISTORS (BJT)

BJT biasing

(formerly BJT biasing (voltage divider))



What it computes

For an NPN or PNP bipolar transistor biased by a voltage divider with an emitter resistor, it gives: - the collector current Ic and the voltage across the transistor (Vce, or Vec for a PNP), which together are the operating point; - whether the transistor is active, saturated or off; - the voltages at the base, emitter and collector; - Ic if the current gain hFE were half or twice the value entered; - a warning when the divider is too weak.

Source

The analysis is the standard Thevenin treatment of the divider, found in every electronics textbook. Transistor data read from the datasheets themselves: - **Diodes Inc. MMBT3904** (DS30036 rev. 27, 2025), the SOT-23 version of the 2N3904: - hFE at least 40 at 0.1 mA and 70 at 1 mA, 100–300 at 10 mA, at least 60 at 50 mA; - VCE(sat) 0.2 V maximum at 10 mA (with IB = 1 mA); - VBE(sat) 0.65–0.85 V at 10 mA. - **ST 2N3906 PNP** (2003): hFE at least 80 at 1 mA, 100–300 at 10 mA; VCE(sat) 0.25 V maximum at 10 mA.

The tool uses 0.2 V for Vce in saturation.

What it means

A transistor amplifies only if it already carries a steady resting current. That current, and the voltage across the transistor with no signal, are its **operating point** (or Q-point).

- **The divider** — R1 and R2 divide the supply and hold the base near a fixed voltage, Vth.
- **Vbe** — the emitter sits one base-emitter voltage away from the base: about 0.6–0.7 V for silicon. So the emitter voltage is fixed, Re fixes the emitter current, and Ic is almost the same.
- **hFE** (or β) — the current gain, Ic / Ib. It varies a lot between transistors of one type: 100 to 300 for a 2N3904 at 10 mA. Here it only enters through the small base current. That is why this circuit is used so widely: the "hFE ½–2×" cell shows how little Ic moves when hFE halves or doubles.
- **Regions:**
- **active** — the transistor can amplify;
- **saturated** — fully on, with Vce about 0.2 V;
- **off** (cut off) — no current flows.

Saturated and off are both useless for an amplifier. - **Vce near half the supply** — leaves the most room for the signal to swing both ways. - **Stiff divider** — the current in the divider should be at least 10 times the base current, or the base voltage sags with hFE. This is a rule of thumb, and the tool warns below it. - **PNP** —

the same circuit upside down: the emitter and Re near the supply, the collector and Rc near ground. Switching the type swaps R1 and R2, so the base lands near the supply as a PNP needs.

Why the formulas are these

The divider seen from the base is a source Vth = Vcc × R2 / (R1 + R2) with a resistance Rth = R1 ∥ R2. Around the loop through the base and emitter,

```
Vth = Ib × Rth + Vbe + (hFE + 1) × Ib × Re
```

so

```
Ib = (Vth − Vbe) / (Rth + (hFE + 1) × Re),   Ic = hFE × Ib
```

For a PNP, Vth is replaced by Vcc − Vth. Then Vce = Vcc − Ic × Rc − Ie × Re.

If that would leave less than the saturation voltage, the transistor is saturated instead. Ic is then (Vcc − 0.2 V) / (Rc + Re). If Vth does not exceed Vbe, it is off.

When (hFE + 1) × Re is much larger than Rth, Ib × hFE ≈ (Vth − Vbe) / Re: hFE drops out. That is the design aim.

Assumptions and limits

- **Fixed Vbe.** Vbe also falls as the transistor warms, about 2 mV per °C (a textbook figure, not from these datasheets). Re keeps the effect small, but it is not calculated.
- **Constant hFE.** hFE is taken as constant; really it changes with Ic and temperature.
- **DC only.** Signal gain, coupling capacitors and the emitter bypass capacitor are not covered.
- **Simple saturation.** The saturation current neglects the base current's share of the emitter current.

What it deliberately does not do

- It does not design the divider for a target current; enter values and adjust.
- Small-signal gain and input impedance are not calculated.

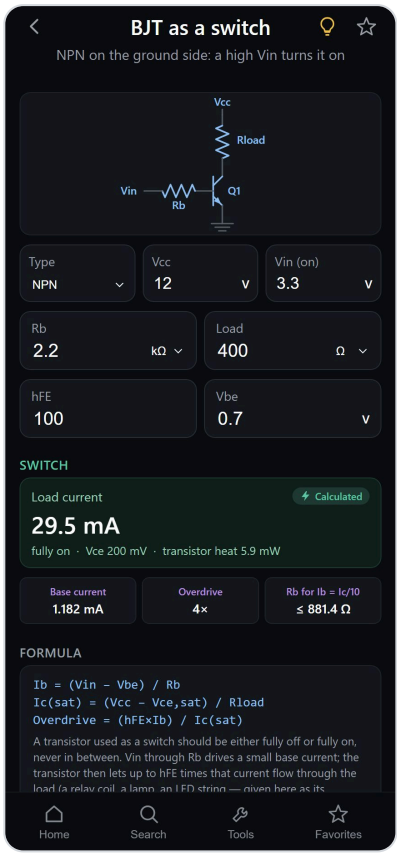
How it was checked

All on 12 V, with Vbe 0.7 V and hFE 100: - **NPN, 47 k / 10 k divider, Rc 2.2 k, Re 1 k:** - Ic 1.286 mA, Vce 7.871 V, Vb 1.999 V, Ve 1.299 V, Vc 9.17 V; - 1.19–1.34 mA for hFE 50–200. - **Rc 3.3 kΩ:** Vce 6.456 V. - **PNP with the divider swapped (10 k / 47 k):** the same 1.286 mA. - **Rc 10 kΩ:** saturated, Ic (12 − 0.2) / 11 kΩ = 1.073 mA. - **470 k / 100 k divider:** only about 2× the base current, so the tool warns.

The review made these changes: - It put Ic first and large, with Vce and the region in words. - It added the hFE ½–2× robustness cell and the stiff-divider check. - It limited the units to real ones and remembers them. - It replaced the NPN/PNP pills with a Type choice in the fields, so the result stays above the tab bar on a

smaller phone. - It rewrote the note in plain words and added examples. - The tool was renamed BJT biasing, on Pierre's choice, to fit beside the bulb.

BJT as a switch



What it computes

For a bipolar transistor switching a load, it gives: - the load current and whether the transistor is fully on, partly on or off; - the voltage left across the transistor and the heat it produces; - the base current; - the overdrive, which is how much spare base drive there is; - the largest base resistor that drives the transistor the way datasheets test it.

The transistor is an NPN on the ground side of the load, or a PNP on the supply side.

Source

Saturation data read from the datasheets themselves: - **Diodes Inc. MMBT3904** (DS30036 rev. 27, 2025): - VCE(sat) at most 0.2 V at IC = 10 mA with IB = 1 mA, and 0.3 V at 50 mA with 5 mA; - hFE 100–300 at 10 mA; - collector current at most 200 mA. - **ST 2N3906 PNP** (2003): VCE(sat) at most 0.25 V at 10 mA with 1 mA.

The saturation figures are given with the base current at a **tenth** of the collector current. The tool's "Rb for Ib = Ic/10" applies that same condition, and it uses 0.2 V for Vce when the transistor is fully on.

What it means

A transistor used as a switch should be either fully off or fully on, never in between.

- **Base current** — Vin, through Rb, drives a small current into the base: (Vin − Vbe) / Rb, with Vbe about 0.6–0.7 V.
- **Gain (hFE)** — the transistor then allows up to hFE times that current through the load. The load can be a relay coil, a lamp or an LED string, entered as its resistance.
- **Fully on (saturated)** — the load would accept less current than that, so the transistor drops only about 0.2 V and stays cool. The load current is then set by the load alone: (Vcc − 0.2 V) / R.
- **Partly on** — there is not enough base current. The transistor drops a large voltage and turns the difference into heat. This is the fault to avoid.
- **Overdrive** — hFE × Ib divided by the current the load needs.
- hFE varies about 3:1 between transistors of one type, and falls when cold, so the design needs spare base drive.
- The tool flags overdrive below 2×. That threshold is a rule of thumb.
- The surest design follows the datasheet test condition: base current at a tenth of the collector current, whatever the gain. The "Rb for Ib = Ic/10" cell gives the largest Rb that achieves it.
- **Coils** — a relay or motor coil needs a diode across it. Otherwise the voltage spike when the current stops can destroy the transistor.
- **NPN or PNP:**

- **NPN** — switches the ground side of the load, and turns on with a high Vin.
- **PNP** — switches the supply side, and turns on when its base is pulled low. To turn it off, the base must come back up to Vcc. A 3.3 V or 5 V logic pin cannot do that on a higher supply; drive the PNP's base with an NPN instead.

Why the formulas are these

```
Ib = (Vin − Vbe) / Rb                (NPN)
Ib = (Vcc − Vin − Vbe) / Rb          (PNP)
Ic(sat) = (Vcc − 0.2 V) / Rload
```

- **Fully on** — if hFE × Ib ≥ Ic(sat), the transistor is saturated and Ic = Ic(sat).
- **Partly on** — otherwise Ic = hFE × Ib, and Vce = Vcc − Ic × Rload.
- **Heat** — the transistor dissipates Vce × Ic.
- **Largest Rb** — Rb = (Vin − Vbe) / (Ic(sat) / 10), and the same with Vcc − Vin − Vbe for a PNP.

Assumptions and limits

- **A resistive load.** For a coil, use its DC resistance, and fit the diode.
- **Steady state.** Switching speed is not calculated, and neither is the storage time a saturated transistor takes to turn off.
- **Ratings.** The transistor's own limits are not checked: a 2N3904 or MMBT3904 is rated 200 mA. Heavier loads need a bigger transistor or a MOSFET.

What it deliberately does not do

- MOSFET switches have their own tool.
- Darlington pairs are not covered.

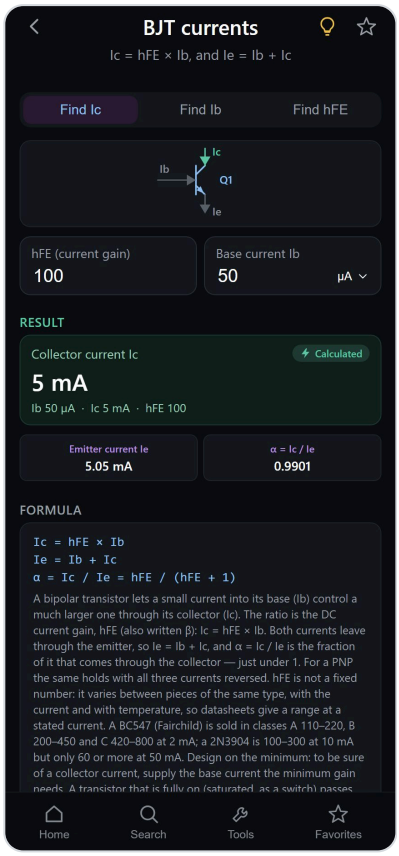
How it was checked

All with Vbe 0.7 V and hFE 100: - **12 V, 400 Ω load, 3.3 V through 2.2 kΩ:** - 29.5 mA, fully on, 5.9 mW of heat; - Ib 1.182 mA, 4× overdrive, Rb ≤ 881.4 Ω for Ic/10. - **120 Ω on 12 V from 5 V through 1 kΩ:** 98.33 mA, 4.4× overdrive. - **47 kΩ base resistor:** partly on, 5.532 mA, with 9.787 V across the transistor. - **PNP on 5 V, 100 Ω, base pulled to 0 V through 1 kΩ:** 48 mA, with the turn-off note. - **5.6 kΩ base resistor:** on, but with under 2× overdrive, so the tool flags it.

The review made these changes: - It put the load current first and large, with the state, Vce and the transistor's heat in words. - It added the "Rb for Ib = Ic/10" design value, from the datasheets' own test condition. - It added the partly-on, low-overdrive and PNP turn-off notes, and the coil diode in the note. - It replaced the NPN/PNP pills with a Type choice in the fields. - It limited the units to real ones and remembers them. - It rewrote the note in plain words and added examples.

BJT currents

(formerly BJT currents (hFE, IC))



What it computes

For a bipolar transistor, enter two of base current Ib, collector current Ic and current gain hFE, and it gives the third. It also gives the emitter current Ie and α, the share of the emitter current that comes through the collector.

Source

The relations are the definitions of the DC current gain and Kirchhoff's current law. The gain figures are read from the datasheets themselves: - **Fairchild BC546–BC550** (rev. A2, 2002): - hFE 110–800 at VCE 5 V, IC 2 mA, sold in classes A 110–220, B 200–450 and C 420–800; - collector current at most 100 mA. - **Diodes Inc. MMBT3904** (the SOT-23 2N3904, DS30036 rev. 27, 2025): hFE at least 40 at 0.1 mA, 70 at 1 mA, 100–300 at 10 mA, at least 60 at 50 mA and 30 at 100 mA.

What it means

A bipolar transistor lets a small current into its base control a much larger one through its collector.

- **hFE** (also written β) — the DC current gain: Ic = hFE × Ib.
- **Ie** — both currents leave through the emitter of an NPN, so Ie = Ib + Ic. For a PNP, all three currents flow the other way.
- **α** — Ic / Ie, the fraction of the emitter current that comes through the collector. It is just under 1: 0.990 for hFE 100.
- **hFE is not a fixed number.** It differs between pieces of the same type, changes with the current and with temperature. That is why datasheets give a range at a stated current, and why the BC547 is sorted and sold in gain classes.
- **Design on the minimum.** To be sure of a collector current, supply the base current the *minimum* gain needs, at the current you will use. A 2N3904 is 100–300 at 10 mA but only 60 or more at 50 mA.
- **Saturated transistors.** A transistor used as a fully-on switch passes less than hFE × Ib, because the load sets the current; see BJT as a switch. Circuits that must not depend on hFE use the divider bias of BJT biasing.

The current being solved for is drawn green in the diagram.

Why the formulas are these

```
hFE = Ic / Ib      (the definition)
Ie = Ib + Ic       (the current in equals the current out)
α = Ic / Ie = hFE / (hFE + 1)
```

Assumptions and limits

- **Active region.** The transistor is assumed to work in the active region, where Ic = hFE × Ib holds.
- **DC gain only.** The DC gain hFE is used. The small-signal gain hfe is similar but not identical.
- **Leakage ignored.** Leakage currents (nanoamps for these parts) are neglected.

What it deliberately does not do

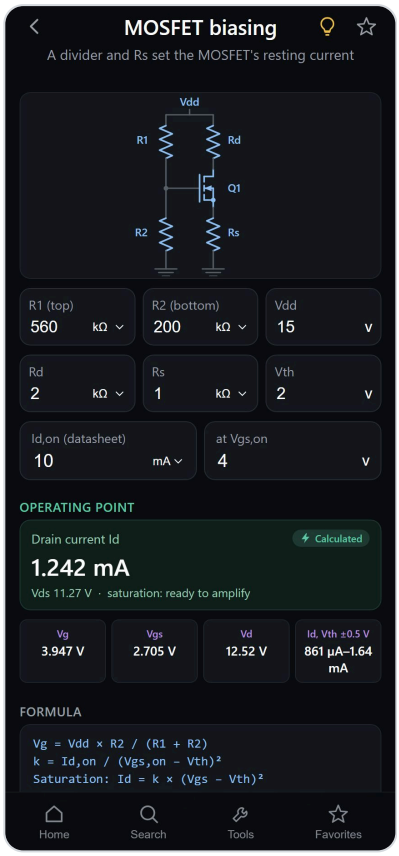
- Bias networks and switching are covered by the other two BJT tools.

How it was checked

- **hFE 100, Ib 50 µA:** Ic 5 mA, Ie 5.05 mA, α 0.9901.
- **hFE 200 (BC547B minimum), Ib 10 µA:** Ic 2 mA, α 0.995.
- **50 mA at hFE 60 (2N3904 minimum at 50 mA):** Ib 833.3 µA.
- **20 µA in, 5.2 mA out:** hFE 260, α 0.9962.

The review made these changes: - It replaced an unchecked 2N2222 figure with gain data read from the BC547 and 2N3904 datasheets. - It added α and colours the solved current green in the diagram. - It limited the units to µA and mA for Ib, and µA, mA and A for Ic, and remembers them. - It labels the choice as "Find Ic / Ib / hFE". - It rewrote the note in plain words and added examples. - The tool was renamed BJT currents, on Pierre's choice, to fit beside the bulb.

MOSFET biasing



What it computes

For an N-channel enhancement MOSFET biased by a voltage divider on its gate, with a drain resistor Rd and a source resistor Rs, it gives: - the drain current Id and the voltage across the MOSFET, Vds, which together are the operating point; - the region: saturation (amplifying), triode (fully on) or off; - the gate, gate-source and drain voltages; - Id if the threshold voltage were 0.5 V lower or higher.

Source

The device model is the textbook square law, solved for the circuit as drawn. Device data read from the datasheet itself: - **Zetex 2N7000** (issue 2, November 1994): - gate threshold Vgs(th) 0.8 V minimum, 3.0 V maximum, at Id = 1 mA; - on-state drain current Id(on) at least 75 mA at Vgs 4.5 V, Vds 10 V; - Rds(on) at most 5 Ω at Vgs 10 V, 500 mA.

The 3.7:1 threshold range is why the source resistor matters.

What it means

A MOSFET is controlled by the voltage on its gate, which draws no current.

- **Threshold Vth** — below it, no drain current flows. Above it, the current grows with the square of how far the gate is above it.
- **The divider** — R1 and R2 divide the supply to set the gate voltage Vg. No gate current loads them, unlike a BJT's base.
- **The source resistor Rs** — pulls back: more current raises the source, which lowers Vgs, so the current settles at the operating point.
- This feedback reduces how much a different threshold moves the current. The "Id, Vth ±0.5 V" cell shows it.
- It works better with a higher gate voltage and a larger Rs.
- **k** — the device's square-law constant. It comes from one datasheet point: a drain current Id,on at a gate voltage Vgs,on.
- **The names are a trap:**
- **saturation** is where a MOSFET *amplifies*: its drain current is set by the gate, not by Vds. That is the opposite of a BJT's saturation;
- **triode** (or ohmic, or linear) is where it is fully on and acts as a resistor. A switch wants this; an amplifier does not;
- **off** — the gate is below the threshold.

Why the formulas are these

```
Vg = Vdd × R2 / (R1 + R2)
k = Id,on / (Vgs,on − Vth)²
Saturation (Vds ≥ Vgs − Vth):   Id = k × (Vgs − Vth)²
Triode     (Vds < Vgs − Vth):   Id = k × (2 (Vgs − Vth) Vds − Vds²)
Vgs = Vg − Id × Rs,   Vds = Vdd − Id × (Rd + Rs)
```

The two device equations meet at the boundary Vds = Vgs − Vth. As Id rises, Vgs and Vds both fall, and the device current falls with them, so there is exactly one crossing. The tool finds it by bisection, in whichever region it lies.

Assumptions and limits

- **The square law is an approximation.** Real MOSFETs follow it only roughly, and power MOSFETs poorly. Use it for understanding and first values, then measure.
- **k from one datasheet point.** Datasheet Id(on) figures are often minimums, measured in pulses.
- **Ignored effects.** Channel-length modulation (the slight rise of Id with Vds) and temperature are not modelled.
- **N-channel only.** A P-channel circuit is the mirror image.

What it deliberately does not do

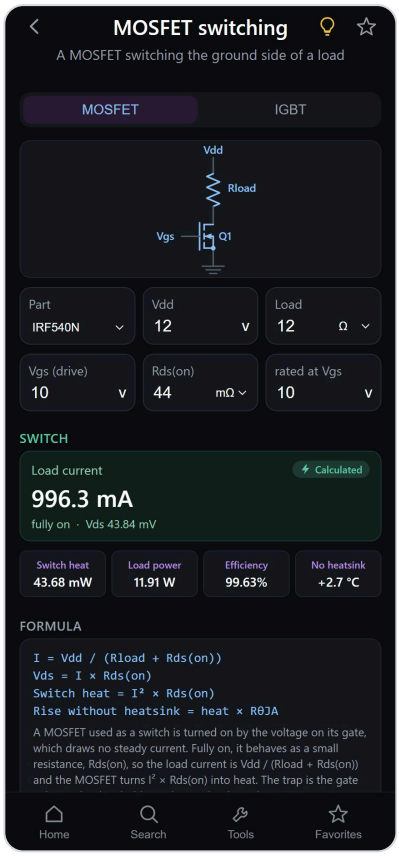
- Small-signal gain is not calculated.
- MOSFET switches have their own tool.

How it was checked

All on 15 V with a 560 k / 200 k divider (Vg 3.947 V), Vth 2 V and 10 mA at Vgs,on 4 V (k = 2.5 mA/V²). The figures were also computed in a separate script. - **Rd 2 kΩ, Rs 1 kΩ:** - Id 1.242 mA, Vds 11.27 V, saturation; - 861 µA–1.64 mA for Vth ±0.5 V. - **Rd 1 kΩ, no Rs:** - Id 9.481 mA; - 5.24–13.4 mA for Vth ±0.5 V (the top end limited by the triode region). - **Rd 12 kΩ, Rs 1 kΩ:** triode, Id 1.127 mA, Vds 349 mV. The old version applied the saturation formula there. - **R2 50 kΩ (Vg 1.23 V):** off.

The review made these changes: - It fixed the triode region. The current and voltages were computed with the saturation equation even there; now each region uses its own. - It explains the region names in plain words. - It put Id first and large, and added the Vth ±0.5 V cell. - It limited the units to real ones and remembers them. - It rewrote the note and added examples. - A global layout fix made three-to-a-row fields slightly tighter, so a three-digit value beside a unit is no longer cut on a 375 px phone.

MOSFET switching



What it computes

For a MOSFET or an IGBT switching the ground side of a resistive load, fully on, it gives: - the load current and the voltage left across the switch; - the heat in the switch, the power in the load and the efficiency; - for parts whose datasheet gives it, how much the switch warms without a heatsink; - whether the gate drive is enough for the on-state figures to hold.

Source

Device data read from the datasheets themselves:

Part	Rds(on) max	at Vgs	Vgs(th)	RθJA (no heatsink)
2N7000 (Zetex, issue 2, 1994)	5 Ω	10 V	0.8–3.0 V at 1 mA	not used
IRLZ44 (Vishay 91328 rev. D, 2021)	28 mΩ	5 V	1.0–2.0 V at 250 µA	62 °C/W
IRF540N (Infineon rev. 2.1)	44 mΩ	10 V	2.0–4.0 V at 250 µA	62 °C/W

The IRLZ44 is also specified at 39 mΩ at Vgs 4 V. The IGBT model, a knee voltage V0 plus a slope resistance Rce, is the usual straight-line fit of an IGBT datasheet's Vce–Ic curve. Its default values are illustrative, not from a particular part.

What it means

A MOSFET used as a switch is turned on by the voltage on its gate, which draws no steady current.

- **Fully on** — it behaves as a small resistance, Rds(on). The load current is set almost entirely by the load, and the MOSFET turns I^2 × Rds(on) into heat.
- **The gate-voltage trap:**
- the **threshold** Vgs(th) is only where the MOSFET *starts* to conduct: 250 µA for the IRF540N, 1 mA for the 2N7000;
- **Rds(on) is guaranteed only at a stated, higher gate voltage** — 10 V for a standard part, 4.5 or 5 V for a logic-level one;
- a 3.3 V microcontroller pin cannot properly drive an IRF540N. Its threshold alone may be up to 4 V, so some pieces would not turn on at all. Use a logic-level part rated at your drive voltage, or a gate

driver.
- **The three drive verdicts:**
- **fully on** — the drive reaches the voltage the on-resistance is rated at;
- **on, but not fully** — the drive is above the highest threshold but below that voltage, so the real resistance and heat are higher than shown;
- **may not turn on** — the drive does not exceed the highest threshold the part may have.
- **Heat without a heatsink** — the heat × RθJA: about 62 °C per watt for these TO-220 parts. The tool warns above 100 °C of rise, a rule of thumb that leaves room for warm surroundings. The IRF540N's datasheet allows 175 °C at the junction; check each part's own limit.
- **IGBT** — like a MOSFET, driven by its gate voltage, and suited to high voltages. Fully on, it drops a knee voltage plus a small resistance. That figure is specified at a stated gate voltage, usually 15 V.
- **Coils** — a relay or motor coil needs a diode across it, as with any switch.

Choosing a part fills in its datasheet figures. Typing your own switches it to Custom.

Why the formulas are these

```
MOSFET:  I = Vdd / (Rload + Rds(on)),   Vds = I × Rds(on),
         heat = I² × Rds(on)
IGBT:    I = (Vdd − V0) / (Rload + Rce), Vce = V0 + I × Rce,
         heat = Vce × I
Rise without heatsink = heat × RθJA
```

Assumptions and limits

- **Fully-on state only.** Switching losses, the energy lost during each turn-on and turn-off, are not calculated. They matter at high switching frequencies (PWM).
- **Rds(on) at 25 °C.** It rises as the MOSFET warms (each datasheet plots it against temperature), so a hot switch dissipates more than shown.
- **A resistive load** on the ground side (low-side switch).

What it deliberately does not do

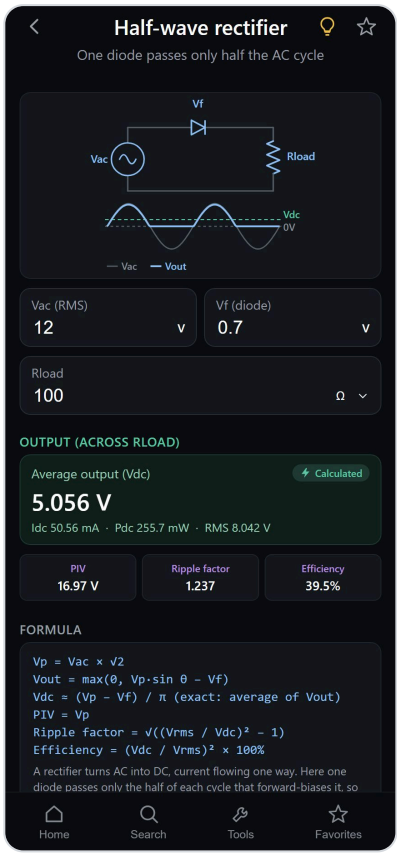
- Gate-driver design, gate charge and switching speed are not covered.
- High-side switching (P-channel, or N-channel with a driver) is not covered.

How it was checked

These figures were also computed in a separate calculation. - **IRF540N, 12 V, 12 Ω load, 10 V drive:** 996.3 mA, 43.68 mW in the MOSFET, +2.7 °C, 99.63 % efficient. - **The same from a 3.3 V pin:** may not turn on. - **6 V of drive:** on, but not fully. - **IRLZ44, 12 V, 2 Ω, 5 V drive:** 5.917 A, 980.4 mW, +61 °C. - **IRF540N with a 1 Ω load:** about 5.8 W, so the tool says a heatsink is needed. - **IGBT, 12 V, 12 Ω, V0 0.8 V, Rce 150 mΩ, 15 V drive:** 921.8 mA, Vce 938.3 mV.

The review made these changes: - The on-state check called the switch fine once the drive was 2 V above the threshold. It now compares the drive with the gate voltage the datasheet guarantees Rds(on) or Vce(sat) at, and with the part's highest threshold. - It added datasheet parts, and the temperature rise without a heatsink. - It put the load current first and large. - It gives the IGBT its own 15 V drive. - It limited the units to real ones and remembers them. - It rewrote the note in plain words and added examples.

Half-wave rectifier



What it computes

For one diode feeding a resistive load from an AC source: the average (DC) output voltage and current, the DC power, the RMS output, the peak reverse voltage the diode must withstand, the ripple factor and the efficiency. It draws the input and output waveforms.

Source

No standard is needed: the results are averages of the output waveform over one cycle, derived below. With a zero diode drop they reduce to the textbook figures for a half-wave rectifier — Vdc = Vp/π, ripple factor 1.21, efficiency $4/\pi^2$ = 40.5%.

What it means

A **rectifier** turns alternating current into current that flows one way only. The **half-wave** rectifier is the simplest: a single diode in series with the load. During the half of each cycle that makes the diode's anode positive, it conducts and the load sees the input, less the diode's forward voltage **Vf**; during the other half it blocks and the load sees nothing. The output is one hump per cycle.

- **Vac** is the source's RMS voltage, the figure on a transformer's label; its **peak**, Vp = Vac × √2, is 1.41 times higher.
- **Vdc** is the average of the output, what a DC voltmeter reads.
- **PIV**, the peak inverse voltage, is the reverse voltage the diode must block while it is off: the whole input peak, since no current flows and the load drops nothing. The diode's rated reverse voltage (VRRM) must exceed it.
- **Ripple factor** — how much AC is left riding on the DC, as the ratio of the AC part's RMS to the DC value. 1.21 means the output is mostly ripple.
- **Efficiency** — the share of the power in the load that is DC, at most 40.5% here.

A bare half-wave rectifier is used where smoothness does not matter, or with a capacitor across the load to fill the gaps (the Rectifier ripple tool).

Why the formulas are these

With a constant drop Vf, the output is Vout = max(0, Vp·sin θ − Vf): the diode conducts from θ1 = asin(Vf / Vp) to π − θ1, a little less than half a cycle. Averaging over the cycle:

```
Vdc   = [Vp·cos θ1 − Vf·(π/2 − θ1)] / π
Vrms² = [Vp²·((π − 2θ1)/2 + sin 2θ1 / 2) − 4·Vp·Vf·cos θ1 + Vf²·(π − 2θ1)] / 2π
```

With Vf = 0 these are Vp/π and Vp/2. The common shortcut Vdc ≈ (Vp − Vf)/π treats the output as a whole half-sine of peak Vp − Vf, and reads 2.4% high at 12 V AC with a 0.7 V diode.

Then Idc = Vdc / R, Pdc = Vdc × Idc, ripple factor = $\sqrt{((Vrms/Vdc)^2 - 1)}$ and efficiency = $(Vdc/Vrms)^2$, the DC power over the total power in the load.

Assumptions and limits

- The diode is an ideal switch with a constant drop; a real one's drop rises with current (see Diode forward voltage).
- The source has no resistance of its own: a transformer's winding resistance lowers the output under load.
- The load is a resistor. With a capacitor across it the output, the PIV (up to twice the peak) and the diode's current all change; that is the Rectifier ripple tool.

What it deliberately does not do

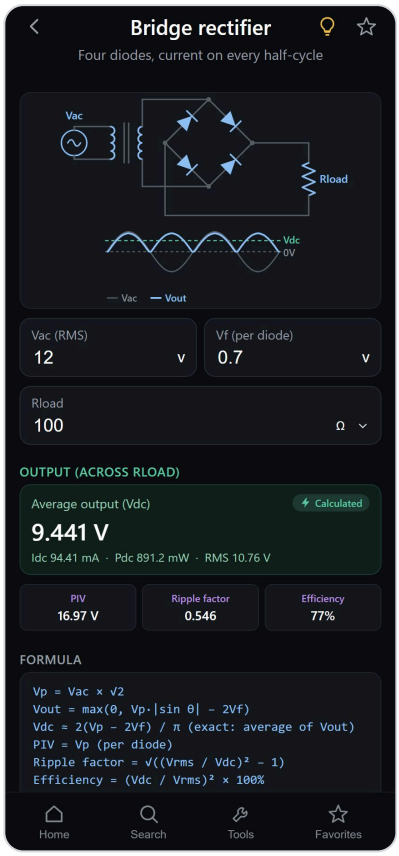
- **Full-wave rectifiers** (bridge, centre-tapped) have their own tools.
- **Smoothing and ripple voltage** with a capacitor is the Rectifier ripple tool.

How it was checked

12 V AC with a 0.7 V diode into 100 Ω: Vp 16.97 V, Vdc 5.056 V, Idc 50.56 mA, ripple factor 1.237, efficiency 39.5%; with Vf = 0, 5.402 V (Vp/π), 1.211 and 40.5%. Both closed forms were checked against a numerical average over the cycle. 3 V AC gives 1.156 V with a 0.4 V Schottky and 1.019 V with 0.7 V silicon; 24 V AC has a 33.94 V PIV.

The review replaced the (Vp − Vf)/π shortcut with the exact averages, which the drawn waveform already followed; put Vdc first and large; limited the units to volts and ohms to megohms; and added plain definitions.

Bridge rectifier



What it computes

For a bridge of four diodes feeding a resistive load from an AC source: the average (DC) output voltage and current, the DC power, the RMS output, the peak reverse voltage each diode must withstand, the ripple factor and the efficiency, with the input and output waveforms.

Source

No standard is needed: the results are averages of the output waveform, derived below. With zero diode drop they reduce to the textbook figures for a full-wave rectifier — Vdc = 2Vp/π, ripple factor 0.483, efficiency $8/\pi^2$ = 81.1%.

What it means

A **bridge rectifier** is four diodes in a diamond. The AC source (usually a transformer's secondary) connects to two opposite corners, the load to the other two. On one half-cycle two of the diodes conduct, on the other half the other two, and in both cases the current passes through the load **the same way**. So where a half-wave rectifier (see that section) gives one pulse per cycle, the bridge gives two, and uses the whole of the input.

The price is two diodes in the current's path at every moment: each pulse is **2 × Vf** lower than the input's peak. From a low voltage that loss is a large part of the output; Schottky diodes, with a smaller Vf, reduce it.

The other figures — **Vdc**, **PIV**, **ripple factor**, **efficiency** — mean the same as for the half-wave rectifier. Each diode, while off, blocks the input's peak Vp, as in the half-wave case; a bridge does not double it.

Why the formulas are these

The output is Vout = max(0, Vp·|sin θ| − 2Vf). Each half-cycle conducts from θ1 = asin(2Vf / Vp) to π − θ1, so the averages are twice the half-wave ones with 2Vf as the drop:

$$V_{dc} = 2\cdot[V_p\cdot\cos\theta_1 - 2V_f\cdot(\pi/2 - \theta_1)] / \pi$$

$$V_{rms}^2 = 2\cdot[V_p^2\cdot((\pi - 2\theta_1)/2 + \sin 2\theta_1 / 2) - 8\cdot V_p\cdot V_f\cdot\cos\theta_1 + 4V_f^2\cdot(\pi - 2\theta_1)] / 2\pi$$

With Vf = 0 these are 2Vp/π and Vp/√2. The shortcut 2(Vp − 2Vf)/π treats each pulse as a whole half-sine and reads 5% high at 12 V AC with 0.7 V diodes. Ripple factor and efficiency follow as for the half-wave rectifier.

Assumptions and limits

- Each diode is an ideal switch with a constant drop.
- The source has no resistance; a transformer's winding resistance lowers the output under load.
- The load is a resistor. A smoothing capacitor changes the diodes' current into short peaks and the output into DC with a small ripple — the Rectifier ripple tool.
- Strictly, a blocking diode sees Vp − Vf, the conducting diode's drop taken off; Vp is the figure to rate it by.

What it deliberately does not do

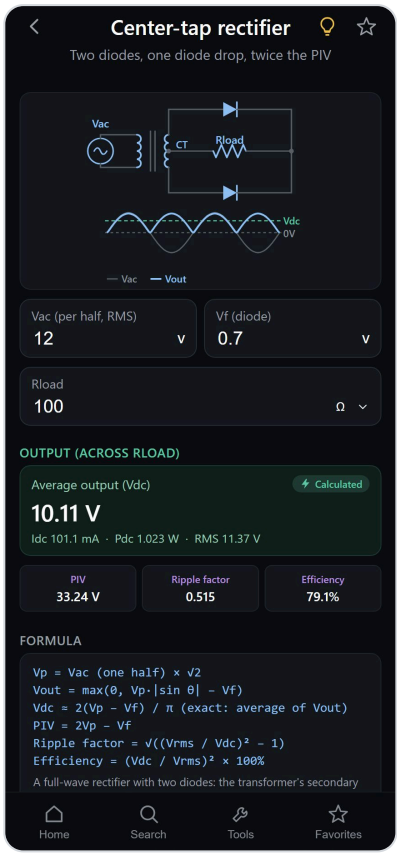
- **The centre-tapped full-wave rectifier**, two diodes and a transformer with a centre tap, has its own tool.
- **Smoothing** with a capacitor is the Rectifier ripple tool.

How it was checked

12 V AC with 0.7 V diodes into 100 Ω: Vp 16.97 V, Vdc 9.441 V, Idc 94.41 mA, ripple factor 0.546, efficiency 77.0%; with ideal diodes 10.80 V (2Vp/π), 0.483 and 81.1%. Both closed forms were checked against a numerical average. 5 V AC gives 3.190 V with 0.7 V diodes and 3.638 V with 0.45 V Schottky ones.

The review replaced the 2(Vp − 2Vf)/π shortcut with the exact averages, which the drawn waveform already followed; put Vdc first and large; limited the units; and added plain definitions and examples.

Center-tap rectifier



What it computes

For a full-wave rectifier made of two diodes and a centre-tapped transformer, feeding a resistive load: the average (DC) output voltage and current, the DC power, the RMS output, the peak reverse voltage each diode must withstand, the ripple factor and the efficiency, with the waveforms.

Source

No standard is needed: the results are averages of the output waveform, as for the bridge rectifier (see that section), with one diode drop instead of two. The peak inverse voltage follows from the circuit, below.

What it means

The transformer's secondary winding has a connection at its middle, the **centre tap**, which becomes the output's negative side. Each end of the winding feeds the load through its own diode. On one half-cycle the top half drives the load through the top diode, on the other half the bottom half through the bottom diode — current flows through the load the same way both times, so the output is full-wave, two pulses per cycle.

- **Vac** is the RMS voltage of **one half** of the secondary. A transformer sold as "24 V CT" (centre-tapped) is two 12 V halves: enter 12.
- **One diode drop.** Only one diode conducts at a time, so each pulse loses Vf, where a bridge loses 2 × Vf — better at low voltage.
- **The catch: PIV.** While one diode conducts, the other has one end at the peak of its own half and the other end at the peak of the conducting half, of opposite sign: it blocks nearly **twice the peak**, 2Vp − Vf. Choose diodes rated for that.
- **Vdc**, **ripple factor** and **efficiency** mean the same as for the half-wave rectifier.

Bridge or centre tap? The centre tap saves two diodes and one drop but needs a winding twice as long, each half working only half the time, and diodes rated for twice the voltage. The bridge uses the whole winding all the time and is the usual choice today.

Why the formulas are these

The output is Vout = max(0, Vp·|sin θ| − Vf), Vp being one half's peak. Each half-cycle conducts from θ1 = asin(Vf / Vp) to π − θ1, so:

```
Vdc   = 2·[Vp·cos θ1 − Vf·(π/2 − θ1)] / π
Vrms² = 2·[Vp²·((π − 2θ1)/2 + sin 2θ1 / 2) − 4·Vp·Vf·cos θ1 + Vf²·(π − 2θ1)] / 2π
PIV   = 2Vp − Vf
```

With Vf = 0, Vdc = 2Vp/π, ripple factor 0.483 and efficiency 81.1%, as for the bridge. The shortcut 2(Vp − Vf)/π reads 2.4% high at 12 V a half.

Assumptions and limits

- Each diode is an ideal switch with a constant drop.
- The transformer has no resistance or leakage, and both halves are equal.
- The load is a resistor; a smoothing capacitor is the Rectifier ripple tool.

What it deliberately does not do

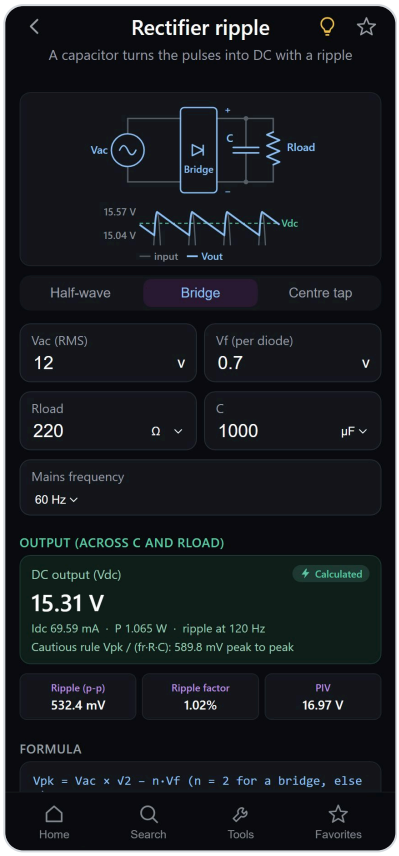
- **The bridge rectifier** has its own tool.
- **Transformer selection** (VA rating, regulation) is not covered.

How it was checked

12 V a half, 0.7 V diodes, 100 Ω: Vp 16.97 V, Vdc 10.11 V, Idc 101.1 mA, PIV 33.24 V, ripple factor 0.515, efficiency 79.1%; with ideal diodes 10.80 V and 0.483. 5 V a half gives 3.824 V; 24 V a half needs 67.18 V PIV. Both closed forms were checked against a numerical average.

The review replaced the 2(Vp − Vf)/π shortcut with the exact averages, put Vdc first and large, limited the units, and added plain definitions and examples. The PIV, 2Vp − Vf, was already right.

Rectifier ripple



What it computes

For a half-wave, bridge or centre-tap rectifier with a smoothing capacitor across a resistive load: the DC output voltage, current and power, the ripple's peak-to-peak voltage and frequency, the ripple factor, the diodes' peak inverse voltage, and the classic design estimate of the ripple for comparison, with the output waveform.

Source

No standard is needed. The figures come from simulating the circuit drawn over a steady cycle; the design estimate is the usual textbook rule. Both are explained below.

What it means

A rectifier alone gives pulses (see the Half-wave and Bridge rectifier sections). A **smoothing capacitor** across the load charges to the peak at each pulse and, between pulses, feeds the load on its own, its voltage sagging as it discharges until the next peak tops it up. The output is then DC with a small sawtooth on top: the **ripple**.

- **Ripple frequency, fr** — how often the capacitor is topped up: the mains frequency for a half-wave, twice it for a bridge or centre tap. That is why full-wave rectification halves the ripple for the same capacitor.
- **Ripple (p-p)** — the height of the sawtooth, from peak to trough.
- **Vdc** — the average output, a little below the peak.
- **Ripple factor** — the ripple's RMS value as a percentage of Vdc.
- **PIV** — with the capacitor holding the output near the peak, a half-wave or centre-tap diode sees nearly twice the peak when off (2Vp − Vf); a bridge diode sees Vp.

More capacitance, or less load current (a larger Rload), gives less ripple.

Why the formulas are these

The simulation. Over each short time step the capacitor either follows the rectified input, while that is higher (the diodes conduct), or decays through the load, v → v·e^(−dt/RC). After a few cycles to settle, one cycle gives the maximum, the minimum, the average (Vdc) and the RMS of the ripple. It models a perfect transformer — an ideal source, one that keeps its voltage whatever current is drawn — and diodes with a constant drop, so it is exact for the circuit drawn.

The design rule. Assuming the capacitor discharges at the full load current for the whole period 1/fr gives

```
Vr(p-p) ≈ Vpk / (fr × R × C),   Vdc ≈ Vpk − Vr / 2
```

with Vpk = Vac × √2 less one diode drop (two for a bridge). The capacitor is really recharged before the period ends, so the rule overstates the ripple — by little at light load, by a lot at heavy load: at 1 A from 2200 µF it gives 4.9 V where the circuit gives 3.4 V. That makes it a safe bound for choosing a capacitor.

Assumptions and limits

- **An ideal source.** A real transformer's winding resistance and leakage inductance stop the capacitor charging fully to the peak and widen the diodes' conduction: Vdc comes out lower and the ripple somewhat larger than simulated. Keep margin, or use the design rule.
- **Diode current.** The diodes conduct in short pulses much larger than the average current; their peak and surge ratings matter, and are not calculated.
- The load is a resistor; a regulator after the capacitor draws a roughly constant current instead, for which the design rule is Vr ≈ I / (fr·C).

What it deliberately does not do

- **Choosing C** for a target ripple is done by trying values; the rule above solves for it directly: C ≈ I / (fr × Vr).
- **Transformer and diode surge ratings** are not covered.

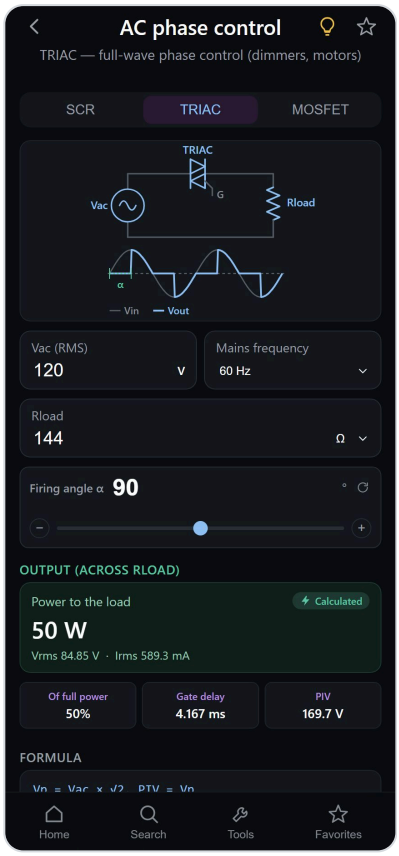
How it was checked

Bridge, 12 V AC, 220 Ω, 1000 µF, 60 Hz: Vdc 15.31 V, ripple 532 mV p-p, design rule 590 mV. 12 Ω and 2200 µF: 13.95 V, 3.40 V p-p against the rule's 4.91 V; on a half-wave, 6.73 V and a PIV of 33.24 V. At 50 Hz with 4700 µF, 2.13 V. The simulation was checked in a separate script with finer steps.

The review found the tool half-wave only, though the name did not say so and the bridge is the common case; its numbers from the design rule while the drawn waveform came from a simulation, so the two disagreed; and a badge warning that the rule was strained, which the simulation makes unneeded. It now offers the three rectifiers, takes its figures from the simulation with the rule beside them, and is renamed Rectifier ripple.

AC phase control

(formerly Thyristor firing angle)



What it computes

For phase control of a resistive load from the mains, with an SCR, a TRIAC or a pair of MOSFETs, at a chosen angle: - the power (TRIAC, MOSFET) or the DC output (SCR); - the RMS voltage and current; - the share of full power; - the delay from each zero crossing at which the device is switched; - the peak voltage the device must block; - for the MOSFETs, the power they lose; - the waveforms.

Source

No standard is needed. The results are averages of the chopped sine, derived below, and match the formulas of power-electronics textbooks for phase-controlled circuits with a resistive load.

What it means

A **thyristor** is a semiconductor switch for AC. It stays off until a short pulse on its **gate** (G) turns it on. It then stays on by itself until the current through it falls to zero, which on AC happens at the end of every half-cycle. So it can be switched on at any point of each half-cycle, but it switches itself off at the next zero crossing.

- **SCR** (silicon-controlled rectifier) — conducts one way only, like a diode with a gate. It passes one half of each cycle, from the moment it is fired, and gives DC.
- **TRIAC** — conducts both ways and is fired in each half-cycle. It passes both halves, so the load gets AC. Lamp dimmers, heater controls and simple motor speed controls use it.
- **Firing angle, α** — how far into each half-cycle the gate is fired, from 0° (at once, full power) to 180° (never, no power). Firing later cuts off the start of each half-wave: this is **leading-edge** control.
- **Gate delay** — the same angle as a time after the zero crossing: α / 360° of a mains period, so 4.17 ms for 90° at 60 Hz. A microcontroller dimmer waits that long after it detects the zero crossing, then pulses the gate.
- **MOSFET** — a transistor switch that turns on and off whenever its gate says, not only at the zero crossing. Each MOSFET has a built-in diode (the body diode) that conducts backwards. One MOSFET alone would therefore still pass one half-wave, so an AC switch uses two back to back with their sources joined: one blocks each direction, and one gate signal drives both.
- **Cut-off angle, β** — with MOSFETs the switch is turned on at each zero crossing and off at β. This cuts the end of each half-wave instead of its start: **trailing-edge** control. β runs from 0° (off at once, no power) to 180° (never off, full power). It suits LED lamps and electronic transformers, and it is quieter, because the current starts gently from zero instead of jumping up.
- **Turns off at** — β as a time after the zero crossing.

- **Rds(on)** — the small resistance of a MOSFET that is switched on, from its datasheet (tens to hundreds of milliohms for mains parts). The load current flows through both MOSFETs, so they dissipate $I_{rms}^2 \times 2 \times R_{ds(on)}$. That power becomes heat in the MOSFETs, which is what the heatsink must carry away.
- **Of full power** — the power at this angle as a share of the power with the switch always on. Power changes slowly near 0° and 180° and fastest around 90°, where it is exactly half.
- **PIV / Vds rating** — the peak voltage the device blocks while it is off: the mains peak. Choose a part rated above it, with margin.

Why the formulas are these

With a TRIAC the load sees $V_p \cdot \sin\theta$ from α to π in each half-cycle. Its power is V_{rms}^2 / R, and the RMS of that chopped sine over a half-cycle is

$$V_{rms} = V_{ac} \times \sqrt{\left(\pi - \alpha + \frac{\sin 2\alpha}{2}\right) / \pi} \qquad \text{(TRIAC)}$$

This is Vac at $\alpha = 0$ and 0 at $\alpha = \pi$. The share of full power is the square of Vrms / Vac, which is the bracket under the root.

An SCR conducts only one half of each cycle. So its RMS is that of a half-wave, $(V_p / 2) \cdot \sqrt{(\ldots)}$, and its average, the DC, is

$$V_{dc} = \frac{V_p}{2\pi} \times (1 + \cos\alpha)$$

This is V_p/π at $\alpha = 0$, the plain half-wave rectifier.

With MOSFETs the load sees $V_p \cdot \sin\theta$ from 0 to β instead. Because the sine is symmetric about 90°, the part from 0 to β carries the same energy as the part from $180° - \beta$ to 180°. So

$$k = \left(\beta - \frac{\sin 2\beta}{2}\right) / \pi, \qquad I_{rms} = V_{ac} \times \sqrt{k} / (R_{load} + 2 \times R_{ds(on)})$$

For a resistive load, trailing-edge control at β gives exactly the power of leading-edge control at $\alpha = 180° - \beta$. The two differ in the shape of the current, not in the power. The MOSFETs' on-resistance is in series with the load, so it is included in the current.

The gate delay (thyristors) and the turn-off time (MOSFETs) are the angle / (360° × f).

Assumptions and limits

- **A resistive load** — lamp, heater. With a motor or a transformer the current lags the voltage, a thyristor does not turn off at the zero crossing of the voltage, and a MOSFET switching off an inductive current must absorb its energy. These formulas do not hold for such loads.
- **Switches.**
- A real thyristor drops 1–2 V while on, small against mains voltages, and needs a minimum holding current to stay on.
- The MOSFET loss is the conduction loss only. Switching losses at each turn-off, and the rise of Rds(on) as the part heats, add to it.

- Phase control chops the current sharply, which causes electrical noise and harmonics, and dimmers need filtering to meet emission limits. This is not covered.

What it deliberately does not do

- **Gate drive** is not covered: pulse current, opto-isolation, snubbers, and the floating supply a MOSFET pair's gate needs.
- **Inductive loads** need a different analysis, as above.

How it was checked

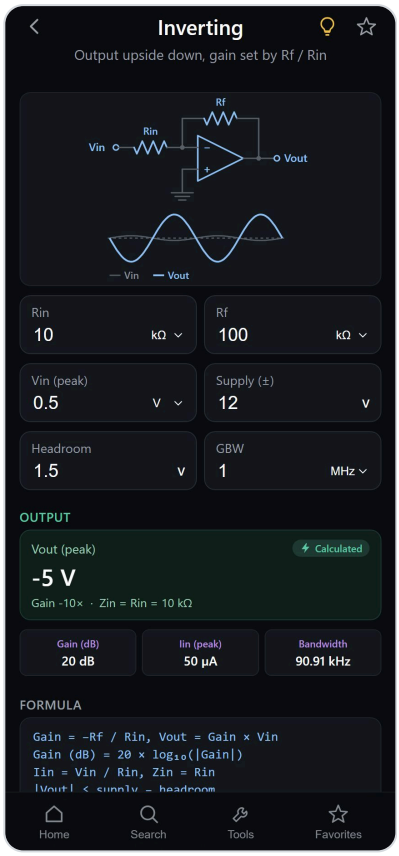
- TRIAC, 120 V, 144 Ω (a 100 W lamp), α = 90°: 84.85 V RMS and 50 W, exactly half of the 100 W at α = 0. Gate delay 4.167 ms at 60 Hz, PIV 169.7 V.
- SCR at 90°: Vdc 27.01 V (Vp / 2π), 60 V RMS, 25 W.
- A 1 kW, 230 V heater at 60°: 80.4% of full power, 3.333 ms at 50 Hz.
- SCR, 24 V AC at 45°: 9.222 V DC.
- MOSFETs, same lamp, β = 90°:
- with Rds(on) = 0: 50 W;
- with 2 × 100 mΩ: 49.86 W, and 69.25 mW lost in the MOSFETs.
- β = 120° gives 80.4%, the TRIAC's figure at 60°.

The review put the power (TRIAC) or the DC (SCR) first and large, showed the gate delay the tool already worked out, and limited the units to real ones (volts, 50 or 60 Hz, ohms to megohms). It also moved the waveform's legend off the waves and added plain definitions and examples.

On Pierre's request it then added the MOSFET pair as a third switch, with trailing-edge control, its own waveform and its conduction loss. The tool was renamed AC phase control, and the old name stays findable in search.

Inverting

(formerly Inverting amplifier)



What it computes

For the inverting op-amp amplifier: - the output voltage for a given input, turned upside down; - the gain, as a ratio and in decibels; - the input current and the input impedance; - where the output clips, from the supply and the op-amp's headroom; - the bandwidth, from the op-amp's gain-bandwidth product.

It draws the input and output waves.

Source

No standard applies. The gain comes from the ideal op-amp model found in every analog textbook. The two real limits, headroom and bandwidth, were checked against manufacturers' datasheets, read from the documents themselves: - **TI TL072** (SLOS080W, July 2025): - maximum peak output ±13.5 V typical, ±12 V minimum, on ±15 V into 10 kΩ, so about 1.5 V short of each rail; - gain-bandwidth product 5.25 MHz (3 MHz for some versions). - **TI LM358** (SLOS068AB, October 2024): - the output stops 2 V typical, 3 V maximum, below the positive rail at 30 V, and 5–20 mV above the negative one; - gain-bandwidth product 0.7 MHz (1.2 MHz for the LM358B). - **Microchip MCP6002** (DS20001733L), a rail-to-rail part: - the output gets within 25 mV of each rail; - gain-bandwidth product 1 MHz.

What it means

An **op-amp** (operational amplifier) is a chip with two inputs, "−" and "+", and one output. It amplifies the difference between its inputs by a huge amount, 100 000 times or more. It is almost always used with **feedback**: part of the output is returned to the "−" input, and the resistors then set the gain.

- **Virtual ground** — the "+" input is at 0 V here. With feedback, the op-amp drives its output until the "−" input sits at the same 0 V. So the "−" input is at ground potential without being connected to ground.
- **Rin** — the input resistor. With one end at Vin and the other at 0 V, it carries Iin = Vin / Rin.
- **Rf** — the feedback resistor, from the output back to the "−" input. The op-amp's input takes no current, so all of Iin continues through Rf.
- **Gain** — Vout / Vin = −Rf / Rin. The minus sign means the output is turned upside down: a positive input gives a negative output. For a sine wave this is a 180° phase shift.
- **Gain in dB** — 20 × $\log_{10}$ of the size of the gain. A gain of 10 is 20 dB, and a gain of 100 is 40 dB.
- **Zin** — the input impedance, the load the signal source sees. It is simply Rin, because the other end of Rin is held at 0 V.

- **Headroom** — how close the output can get to the supply. Enter the value from the op-amp's datasheet:
- a classic op-amp (TL072, LM358) stops about 1.5–2 V short of each rail;
- a rail-to-rail op-amp gets within a few tens of millivolts, so enter 0 for it.
- **Clipping** — when the gain asks for more than the output can give, the output stops at its limit and the tops of the wave are cut flat. That is distortion.
- **GBW** (gain-bandwidth product) — a figure from the datasheet: the gain the op-amp can give, multiplied by the frequency, is at most this. A 1 MHz op-amp can give a gain of 10 up to about 100 kHz.
- **Bandwidth** — the frequency where the stage's gain has fallen by 3 dB, to 71% of its low-frequency value.

Why the formulas are these

The "−" input is at 0 V, so the current through Rin is Vin / Rin. None of it enters the op-amp, so the same current flows through Rf, from the "−" node towards the output. The output must therefore sit Iin × Rf below 0 V:

```
Vout = −Vin × Rf / Rin
```

The output can only swing to the supply less the headroom, so

```
|Vout| ≤ supply − headroom
```

beyond which it clips.

The bandwidth depends on the **noise gain**, 1 + Rf / Rin, rather than on the signal gain Rf / Rin. The noise gain is how much the op-amp's own feedback loop divides down its output, and GBW is shared out by it:

```
Bandwidth = GBW / (1 + Rf / Rin)
```

That is why a unity-gain inverter (Rf = Rin) has only half the GBW as bandwidth, whereas a unity-gain buffer has all of it.

Assumptions and limits

- **An ideal op-amp for the gain.** Its open-loop gain is taken as infinite. A real one's is large but finite, which lowers the gain a little: well under 0.1% for ordinary gains.
- **Symmetric supply.** The supply is ± the value entered, and the headroom is taken as the same on both sides. A single-supply circuit needs its input biased to mid-supply, which is not covered.
- **Slew rate.** A large, fast output can also be limited by how fast the output can move (V/µs on the datasheet). This is not calculated.
- **Offset and bias current** add a small DC error at the output. They are not included.
- The output must also drive its load. A load below about 2 kΩ lowers the swing of most classic op-amps; see the datasheet.

What it deliberately does not do

- It does not choose the resistors. Pick Rin for the input impedance you need (often 10 kΩ), then Rf = gain × Rin from the E-series.
- Frequency response curves and phase are not drawn.

How it was checked

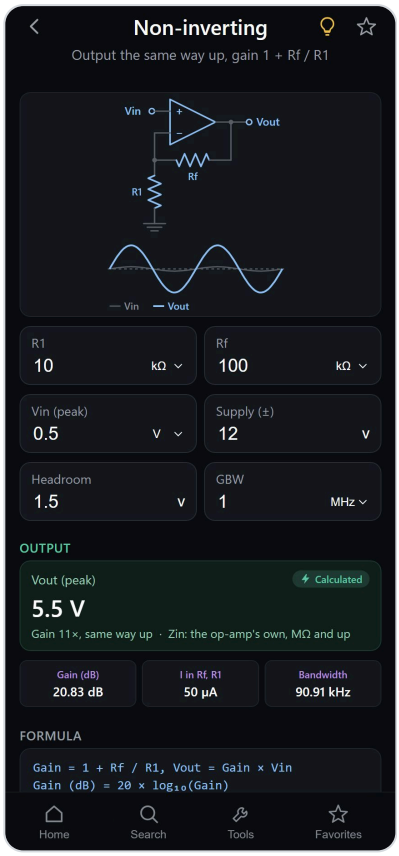
- 10 kΩ / 100 kΩ, 0.5 V peak, ±12 V, 1.5 V headroom, 1 MHz GBW:
- gain −10 (20 dB), Vout −5 V, Iin 50 µA;
- bandwidth 1 MHz / 11 = 90.91 kHz.
- Unity inverter: −1 V out, 500 kHz bandwidth.
- Mic preamp, 1 kΩ / 47 kΩ, 20 mV: −940 mV out, 20.83 kHz bandwidth.
- Gain −100, 0.2 V: the ideal −20 V clips at −10.5 V, or at −12 V with no headroom.

The review found three problems: - the output clipped at the full supply, which only rail-to-rail op-amps reach; - the gain was given without the bandwidth that limits it; - the units included mΩ, GΩ and kV.

It now has the headroom and GBW fields, shows Vout first and large, and remembers the units, and the op-amp's figures for all the op-amp tools. It also adds plain definitions, examples and an 11 px legend under the waves.

Non-inverting

(formerly Non-inverting amplifier)



What it computes

For the non-inverting op-amp amplifier, it gives: - the output voltage for a given input, the same way up; - the gain, as a ratio and in decibels; - the current through the feedback divider; - where the output clips, from the supply and the op-amp's headroom; - the bandwidth, from the op-amp's gain-bandwidth product.

It also draws the input and output waves.

Source

No standard applies: the gain comes from the ideal op-amp model. The headroom and gain-bandwidth figures were read from the TL072, LM358 and MCP6002 datasheets, as listed in the Inverting section.

What it means

The signal goes straight into the op-amp's **+ input**. Rf and R1 form a **voltage divider** from the output to ground, and its middle point feeds the − input. The op-amp drives its output until the divided-down voltage equals Vin.

- **Gain** — Vout / Vin = 1 + Rf / R1. The output is the same way up as the input (in phase), and the gain is never below 1.
- With Rf = 0, or with no R1, the gain is exactly 1: a buffer.
- **Gain in dB** — 20 × $\log_{10}$ of the gain.
- **Zin** — the signal source sees only the op-amp's own input, megohms to teraohms, and no resistor. This is the main reason to choose this circuit over the inverting amplifier, whose input impedance is only Rin.
- **Current in Rf, R1** — the divider current, supplied by the op-amp's output. Keep it to a few milliamps: resistors in the kΩ range.
- **Headroom** — how close the output can get to the supply. It is about 1.5–2 V for a TL072 or LM358; enter 0 for a rail-to-rail op-amp. **Clipping** is when the gain asks for more than that and the tops of the wave are cut flat.
- **GBW** — the op-amp's gain-bandwidth product, from its datasheet. **Bandwidth** — the frequency where the gain has fallen by 3 dB.

The headroom and GBW entered here are remembered for all the op-amp tools, since they describe the op-amp rather than the circuit.

Why the formulas are these

The op-amp holds its − input at the same voltage as its + input, which is Vin. The divider puts Vout × R1 / (R1 + Rf) on the − input, so

```
Vout × R1 / (R1 + Rf) = Vin   →   Vout = Vin × (1 + Rf / R1)
```

The divider current is Vout / (Rf + R1). The output can only reach the supply less the headroom, so |Vout| ≤ supply − headroom.

Here the **noise gain**, which sets the bandwidth, equals the signal gain, so

```
Bandwidth = GBW / Gain
```

A gain of 101 on a 1 MHz op-amp leaves only 9.9 kHz. For audio, use a faster op-amp or split the gain over two stages.

Assumptions and limits

The limits are the same as for the inverting amplifier: - an ideal op-amp for the gain; - a symmetric supply, with the same headroom on both sides; - slew rate, offset and bias current are not calculated; - the load and the divider together must not demand more current than the output can give.

The input voltage must also stay within the op-amp's **common-mode range** (on the datasheet), because here the inputs follow the signal. Many classic op-amps do not accept inputs close to their positive supply.

What it deliberately does not do

- It does not choose the resistors. Pick R1 in the kΩ range, then Rf = (gain − 1) × R1 from the E-series.
- Frequency response curves and phase are not drawn.

How it was checked

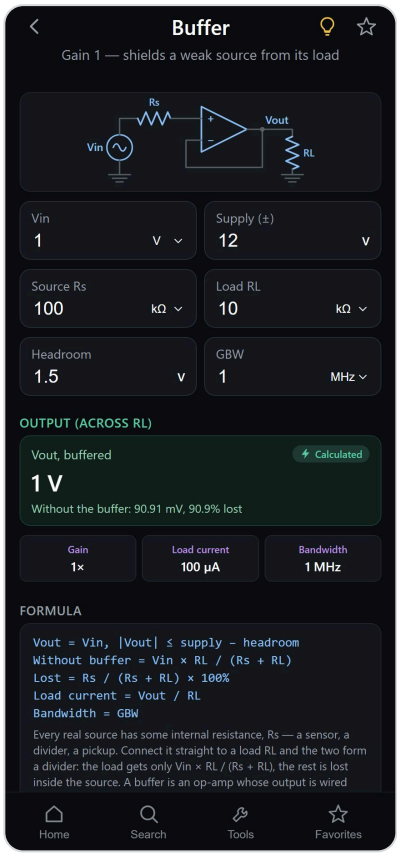
- 10 kΩ / 100 kΩ, 0.5 V peak, ±12 V, 1.5 V headroom, 1 MHz GBW:
- gain 11 (20.83 dB), 5.5 V out, 50 µA in the divider;
- bandwidth 90.91 kHz.
- Rf = R1: gain 2 (6.02 dB), 500 kHz.
- Gain 101, 20 mV: 2.02 V, 9.901 kHz.
- Gain 11, 1.5 V: the ideal 16.5 V clips at 10.5 V.
- Rf = 0: gain 1 and the whole 1 MHz.

The review found the same problems as on the inverting amplifier: - clipping at the full supply; - no bandwidth; - units such as mΩ, GΩ and kV; - a 9 px legend and textbook jargon ("series feedback", "virtual short").

It now shares the headroom and GBW fields and the waveform with the inverting amplifier. It shows Vout first and large, and adds plain definitions and examples.

Buffer

(formerly Buffer (voltage follower))



What it computes

For an op-amp buffer between a source and a load, it gives: - the voltage the load receives through the buffer; - what the load would get without the buffer, and how much is lost; - the load current; - where the output clips, from the supply and the op-amp's headroom; - the bandwidth.

Source

No standard applies. The figures come from the ideal op-amp model and from Ohm's law for the source and load. - **Headroom and gain-bandwidth figures** — read from the TL072, LM358 and MCP6002 datasheets, as listed in the Inverting section. - **The 10 mA warning** — follows the TI LM358 datasheet (SLOS068AB, October 2024). At a 15 V supply it gives an output current of 20 mA minimum (30 typical) when sourcing and 10 mA minimum (20 typical) when sinking.

What it means

Every real signal source has some resistance of its own, **Rs**: a sensor, a voltage divider, a guitar pickup, a reference. When it drives a **load**, RL, current flows, and part of the voltage is lost across Rs before it reaches the load. The two resistances form a divider. This is called **loading**.

- **Without the buffer** — the load gets Vin × RL / (Rs + RL). A 100 kΩ sensor into a 10 kΩ load delivers only 91 mV of its 1 V.
- **Buffer** — an op-amp whose output is wired straight back to its − input. It copies its + input to its output: a gain of 1, the same way up.
- Its input draws almost no current, so no voltage is lost across Rs.
- Its output drives the load from the op-amp's own supply, with almost no resistance of its own.
- **Load current** — Vout / RL. The op-amp's output has to supply it. Above about 10 mA many op-amps lose swing or limit; check the output current on the datasheet.
- **Headroom** — how close the output can get to the supply. It is about 1.5–2 V for a TL072 or LM358, and 0 for a rail-to-rail op-amp. A larger input clips: a buffer has no gain to reduce, so the input itself is too large for the supply.
- **Bandwidth** — with a gain of 1, the bandwidth is the op-amp's whole gain-bandwidth product (GBW).

The headroom and GBW entered here are remembered for all the op-amp tools.

Why the formulas are these

Without the buffer, the same current flows through Rs and RL, so RL gets the share RL / (Rs + RL) of Vin, and the share Rs / (Rs + RL) is lost.

With the buffer, the op-amp holds its − input, which is its output, at the voltage of its + input. Almost no current flows into the + input, so nothing is dropped across Rs, and Vout = Vin, up to supply − headroom. The noise gain is 1, so the bandwidth equals GBW.

Assumptions and limits

- An ideal op-amp, apart from the headroom.
- **Input range.** In a buffer the inputs follow the signal, so Vin must stay within the op-amp's input common-mode range on the datasheet. Many classic op-amps do not accept inputs near their positive supply.
- **Output current.** The 10 mA figure is a rule of thumb from the LM358; other parts deliver more or less.
- **Capacitive loads** (long cables) can make a buffer oscillate. This is not covered.

What it deliberately does not do

- It does not model the source's own frequency response or noise.
- Gain other than 1 belongs to the Non-inverting tool.

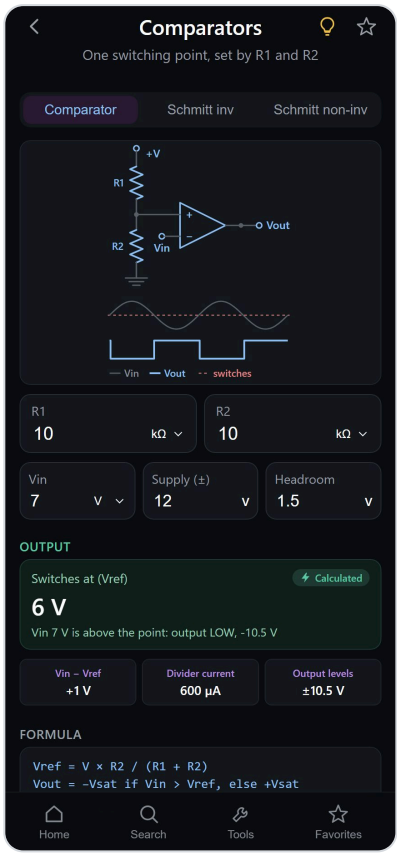
How it was checked

- 1 V from 100 kΩ into 10 kΩ:
- buffered: 1 V, 100 µA, a 1 MHz bandwidth on a 1 MHz op-amp;
- without the buffer: 90.91 mV, 90.9% lost.
- A 10 k / 10 k divider (2.5 V from 5 kΩ) into 1 kΩ: 416.7 mV without the buffer, 2.5 V with it, 2.5 mA.
- 50 Ω into 10 kΩ: 0.498% lost.
- 11 V on ±12 V with 1.5 V headroom: clips at 10.5 V.
- 5 V into 100 Ω: 50 mA, with the warning.

The review kept the drawing from Pierre's sheet and added the source with its Rs and the load RL around it. It added the headroom, the bandwidth and the output-current warning, and put the buffered voltage first and large, with the unbuffered value beside it. It also limited the units to real ones, remembered the choices, and wrote the note in plain words with examples.

Comparators

(formerly Comparator (± hysteresis / Schmitt trigger))



What it computes

For an op-amp used as a comparator, in three forms, it gives: - the switching point or points: one for a plain comparator, two for a Schmitt trigger; - the hysteresis between the two points, and their centre; - whether the output is high or low for a given input, and the two output levels; - for the plain comparator, the reference divider's current.

The waveform shows a sine wave swept across the points and the output it produces.

Source

No standard applies. The points come from the resistor network at the + input; the derivation is below. The output levels use the op-amp's headroom, read from the TL072, LM358 and MCP6002 datasheets as listed in the Inverting section.

What it means

A **comparator** answers one question: which input is higher? It has no feedback to hold its output in between, so the output sits at one of two levels, **high** or **low**. Those levels are the supply less the op-amp's headroom, called **Vsat**. They are about 1.5–2 V short of the rails for a TL072 or LM358, and at the rails for a rail-to-rail op-amp.

- **Comparator** — R1 and R2 divide the supply into a reference, Vref, on the + input. Vin goes to the − input. Vin above Vref sends the output low, and below it high: it is inverting. The divider draws its current all the time.
- **The problem with a plain comparator** — when the input is slow or noisy near Vref, the noise carries it back and forth across the point, and the output flips many times. This is called **chatter**.
- **Schmitt trigger** — the cure is **hysteresis**: two switching points instead of one. A rising input switches at the upper point, **VT+**, and a falling one at the lower point, **VT−**. Noise smaller than the gap between them cannot flip the output back.
- **Inverting** — Rf feeds part of the output back to the divider's middle point, so the reference moves with the output. While the output is high, the point is VT+; while it is low, VT−. When Vin is between the points, the output keeps its last state.
- **Non-inverting** — Vin comes to the + input through Rin, with Rf from the output to the same input, and the − input at 0 V. The points sit symmetrically around 0 V, and the output follows the input.
- **Centre** — the middle of the two points.

The headroom entered here is remembered for all the op-amp tools.

Why the formulas are these

Comparator. Vref = V × R2 / (R1 + R2), and the divider current is V / (R1 + R2).

Inverting Schmitt. Three resistors meet at the + input: R1 from the supply V, R2 to ground, and Rf from the output (±Vsat). With no current into the op-amp, the currents into that node sum to zero, so its voltage is

```
V+ = (V / R1 + Vout / Rf) / G,   G = 1/R1 + 1/R2 + 1/Rf
```

Putting Vout = +Vsat gives VT+, and −Vsat gives VT−. The gap is 2 × Vsat / (Rf × G). A smaller Rf makes it wider.

Non-inverting Schmitt. The output switches when the + input crosses 0 V. There Vin / Rin = −Vout / Rf, so

```
VT± = ± Vsat × Rin / Rf
```

and the gap is 2 × Vsat × Rin / Rf.

The output level Vsat appears in both Schmitt formulas. That is why the headroom changes the switching points, not only the output.

Assumptions and limits

- **An op-amp used as a comparator.**
- An op-amp is not made for this. It switches slowly (its slew rate), and some op-amps misbehave with their inputs far apart.
- A dedicated comparator chip (LM393, LM339) is faster. Its output is usually open-collector: it only pulls low, a pull-up resistor makes the high level, and the high level is the pull-up's voltage. That case is not modelled.
- **Symmetric supply** of ± the value entered, with the same headroom on both sides.
- **Input range** — both inputs must stay within the op-amp's input common-mode range from the datasheet.
- The source driving Vin is taken as having no resistance of its own. In the non-inverting form, any source resistance adds to Rin.

What it deliberately does not do

- It does not choose the resistors for given switching points. Try values instead: a smaller Rf widens the gap, and R1 and R2 move the centre.
- Switching speed and propagation delay are not calculated.

How it was checked

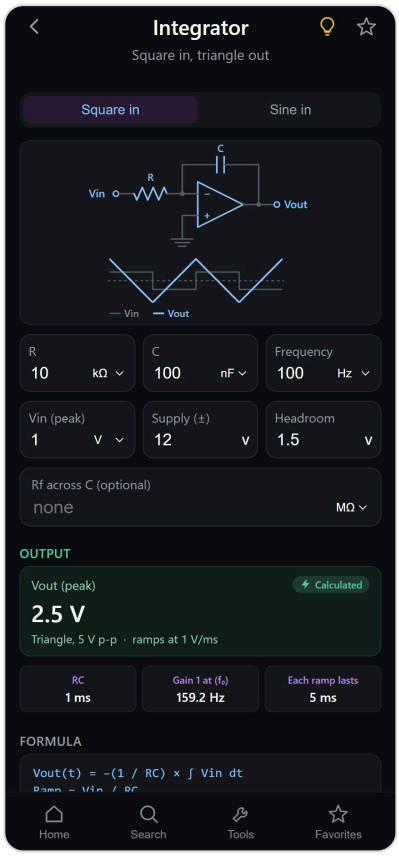
All on ±12 V with 1.5 V headroom, so the output levels are ±10.5 V: - **Comparator**, 10 k / 10 k: switches at 6 V. Vin 7 V gives a low output, −10.5 V, and the divider draws 600 µA. - **Inverting Schmitt**, 10 k / 10 k: - Rf 100 k: VT+ 6.214 V, VT− 5.214 V, a 1 V gap; - Rf 22 k: 6.833 V and 2.944 V, 3.889 V apart; - Rf 100 k on a rail-

to-rail op-amp (headroom 0): 6.286 V and 5.143 V. - **Non-inverting Schmitt**, 10 k / 100 k: ±1.05 V. A 2 V input gives a high output.

The review made several changes: - The output levels were the full supply, so the switching points were wrong for most op-amps. The shared headroom field fixes both. - The switching points come first and large. - The Schmitt mode's Rf no longer squeezes three fields into one row. - The waveform legend is now 11 px, below the plot. - Units are limited to real ones and remembered, the note is in plain words, and there are examples. - The tool was renamed Comparators to fit on one line on the phone.

Integrator

(formerly Op-amp integrator)



What it computes

For the op-amp integrator, with a square or a sine wave at its input: - the output's peak; - for a square, the triangle's peak-to-peak, its ramp rate and how long each ramp lasts; - for a sine, the gain, as a ratio and in decibels; - the time constant RC and the frequency f_0 where the gain is 1; - with the optional drift resistor Rf across C, its corner frequency fL, and the output it then gives; - where the output clips, from the supply and the op-amp's headroom.

It draws the input and output waves.

Source

No standard applies. The output is the integral of the input, from the ideal op-amp model. The headroom figures come from the TL072, LM358 and MCP6002 datasheets, as listed in the Inverting section.

What it means

To **integrate** is to keep adding up: the output is the running total of the input over time.

- **How the circuit does it** — the op-amp holds its − input at 0 V, so the current through R is Vin / R, and none of it enters the op-amp. All of it charges C. A capacitor charged by a steady current changes its voltage at a steady rate, so the output moves at Vin / RC volts per second. It moves downwards for a positive input: the circuit inverts.
- **RC** — the time constant, in seconds. A larger RC makes a slower ramp and a smaller output.
- **Square in, triangle out** — a square wave is a steady +Vin, then a steady −Vin. The output ramps down, then up, which makes a triangle. Its height is the ramp rate times half a period.
- **Sine in** — a sine comes out as a sine moved a quarter cycle (90°) earlier, its size multiplied by the **gain**, 1 / (2πfRC). The gain halves each time the frequency doubles (−6 dB per octave).
- **f_0** — the frequency where the gain is exactly 1. Below it the output is larger than the input, and above it smaller.
- **Headroom** — how close the output can get to the supply. It is about 1.5–2 V for a TL072 or LM358, and 0 for a rail-to-rail op-amp. At low frequencies the integrator asks for large outputs and clips.
- **Rf across C** (optional) — a real integrator also adds up the op-amp's own small offset voltage, so its output slowly drifts into a rail. A large resistor across C, around 100 × R, stops that. Leave the field empty for the ideal circuit.
- **fL, the Rf corner** — 1 / (2π × Rf × C). Above fL the circuit integrates; well below it, it is just an inverting amplifier of gain Rf / R.
- Under 10 × fL the tool warns that the integration is poor.

The headroom is remembered for all the op-amp tools.

Why the formulas are these

The current into C is Vin / R. A capacitor's voltage changes at I / C per second, and the output is the far side of C from the 0 V input:

```
Vout(t) = −(1 / RC) × ∫ Vin dt
```

- **Square wave** — ±Vin holds for half a period, 1 / (2f), at the rate Vin / RC. The swing is

  Vout p-p = Vin / (2 × f × RC)

and the peak is half of it, with the triangle centred on 0 V. - **Sine** — integrating Vp × sin(2πft) gives −Vp × cos(2πft) / (2πf). With the minus sign of the circuit, that becomes +Vp × cos / (2πfRC): the same shape, a quarter cycle ahead, scaled by 1 / (2πfRC) = f_0 / f. - **With Rf** — Rf in parallel with C makes a first-order low-pass of DC gain Rf / R and corner fL. - A sine sees a gain of (Rf / R) / √(1 + (f / fL)^2) and leads by 180° − atan(f / fL). Well above fL this is the ideal f_0 / f and 90°. - A square settles to exponential edges. Each half period starts at +A and heads towards −(Rf / R) × Vin with time constant Rf × C. Requiring it to end at −A gives

```
    A = (Rf / R) × Vin × tanh(1 / (4 × f × Rf × C))

As Rf grows this becomes the ideal Vin / (4 × f × RC). The slope
through 0 V is always Vin / RC.
```

Assumptions and limits

- **Drift.** Without Rf, the tool shows the ideal integrator. A real one drifts into a rail; fit Rf to model the practical circuit. The offset itself is not calculated.
- **Steady state.** The output is shown as it settles, centred on 0 V. Without Rf, its level in a real circuit depends on where it started.
- **Op-amp speed.** At high frequencies the op-amp's gain-bandwidth and slew rate limit the output. This is not calculated.
- The input is assumed to have no DC part. A DC input makes a ramp that never stops.

What it deliberately does not do

- The output's DC error from the offset voltage, multiplied by Rf / R, is not calculated.
- Other waveshapes (pulses, ramps) are not offered.

How it was checked

All on ±12 V with 1.5 V headroom: - Square, 10 kΩ, 100 nF, 1 V, 100 Hz: RC 1 ms, a 5 V p-p triangle (2.5 V peak), ramping 1 V/ms, each ramp 5 ms. f_0 159.2 Hz. - 10 nF at 1 kHz: RC 100 µs, 5 V p-p, 10 V/ms. - Sine at 100 Hz with 10 kΩ and 100 nF: gain 1.592 (4.04 dB), 1.592 V peak. - Square at 10 Hz: the ideal 25 V peak clips

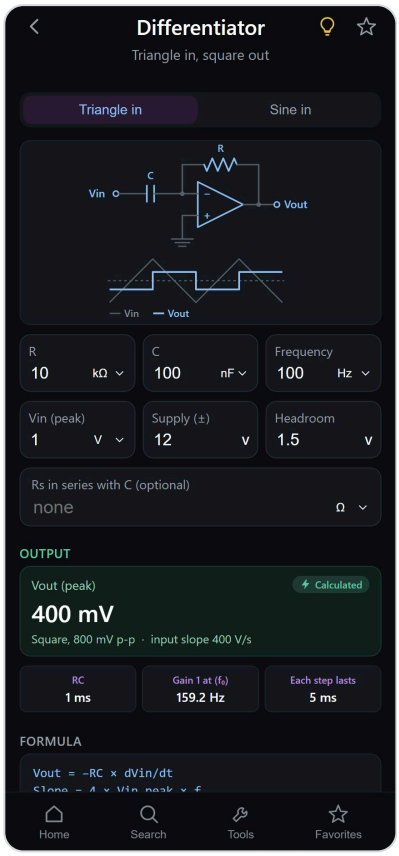
at 10.5 V. - With Rf = 1 MΩ: - fL is 1.592 Hz; - the 100 Hz square gives 2.499 V peak, against 2.5 V ideal; - a 100 Hz sine has a gain of 1.591 and leads by 90.9°; - at 10 Hz, below 10 × fL (15.92 Hz), the tool warns.

These were checked in a separate script.

The review made several changes: - It moved clipping from the full supply to the supply less the headroom. - It put the output peak first and large, with the triangle's size and ramp beside it. - It shows the ramp in V/ms or V/µs instead of kV/s. - It replaced a row with three fields squeezed into two places by two rows of three. - It moved the 11 px legend below the waves. - It limited the units to real ones and remembers them. - It rewrote the note in plain words and added examples. - On Pierre's request, it added the optional Rf across C, drawn as a second rung above C.

Differentiator

(formerly Op-amp differentiator)



What it computes

For the op-amp differentiator, with a triangle or a sine wave at its input, it gives: - the output's peak; - for a triangle, the square's peak-to-peak, the input's slope and how long each step lasts; - for a sine, the gain, as a ratio and in decibels, and the phase; - the time constant RC and the frequency f_0 where the gain is 1; - with the optional resistor Rs in series with C, its corner frequency fH and the output it then gives; - where the output clips, from the supply and the op-amp's headroom.

It draws the input and output waves.

Source

No standard applies. The output is the derivative of the input, from the ideal op-amp model. The headroom figures come from the TL072, LM358 and MCP6002 datasheets, as listed in the Inverting section.

What it means

To **differentiate** is to measure how fast something changes: the output follows the input's slope, not its level. It is the mirror image of the integrator.

- **How the circuit does it** — a capacitor passes current only while its voltage changes, so the current through C is C × (rate of change of Vin). The op-amp holds its − input at 0 V and sends that current through R, so Vout = −RC × slope. A rising input gives a negative output: the circuit inverts.
- **Triangle in, square out** — a triangle rises at one steady rate, then falls at the same rate. The output is therefore a steady negative level, then a steady positive one: a square. A triangle of peak Vp at frequency f rises at 4 × Vp × f.
- **Sine in** — a sine comes out as a sine a quarter cycle (90°) **behind** the input, multiplied by the gain 2πfRC. The derivative of a sine is a cosine, which is ahead, but the circuit's inversion turns it round. The gain doubles each time the frequency doubles (+6 dB per octave).
- **f_0** — the frequency where the gain is exactly 1.
- **Headroom** — how close the output can get to the supply: about 1.5–2 V for a TL072 or LM358, 0 for a rail-to-rail op-amp. At high frequencies the differentiator asks for large outputs and clips.
- **Rs in series with C** (optional) — because the gain keeps rising with frequency, a bare differentiator amplifies high-frequency noise enormously and can oscillate. A small resistor in series with C stops the rise. Leave the field empty for the ideal circuit.
- **fH, the Rs corner** — 1 / (2π × Rs × C). Above it the gain stops rising, at R / Rs.
- The circuit differentiates well only below about fH / 10, and the tool warns above that.

The headroom is remembered for all the op-amp tools.

Why the formulas are these

The current into the − node is C × dVin/dt, and it all flows through R:

```
Vout = −RC × dVin/dt
```

- **Triangle** — the slope is ±4 × Vp × f, so the output is a square of ±RC × 4 × Vp × f. Each level lasts half a period.
- **Sine** — the derivative of Vp × sin(2πft) is 2πf × Vp × cos(2πft). With the minus sign it becomes −2πfRC × Vp × cos: the same shape, a quarter cycle behind, scaled by 2πfRC = f / f_0.
- **With Rs** — Rs in series with C makes a first-order high-pass of top gain R / Rs and corner fH.
- A sine sees a gain of 2πfRC / √(1 + (f / fH)^2) and lags by 90° + atan(f / fH). Well below fH this is the ideal f / f_0 and 90°.

- A triangle's slope drives the input current towards C × slope with time constant Rs × C, so the square gets exponential edges. Its peak is

  ```
  RC × slope × tanh(1 / (4 × f × Rs × C))
  ```

  which becomes the ideal RC × slope as Rs shrinks.

Assumptions and limits

- **Ideal circuit without Rs.** It is shown as drawn on the reference sheet, but a real one needs Rs; fit it to model the practical circuit.
- **Input source.** The source driving Vin is taken as having no resistance of its own. A real source's resistance acts like part of Rs.
- **Op-amp speed.** The op-amp's gain-bandwidth and slew rate also limit the output at high frequencies. This is not calculated.
- **Sharp triangle corners** are taken as perfect, so the square switches instantly.

What it deliberately does not do

- The small capacitor sometimes added across R, for more noise filtering, is not modelled.
- Other waveshapes are not offered.

How it was checked

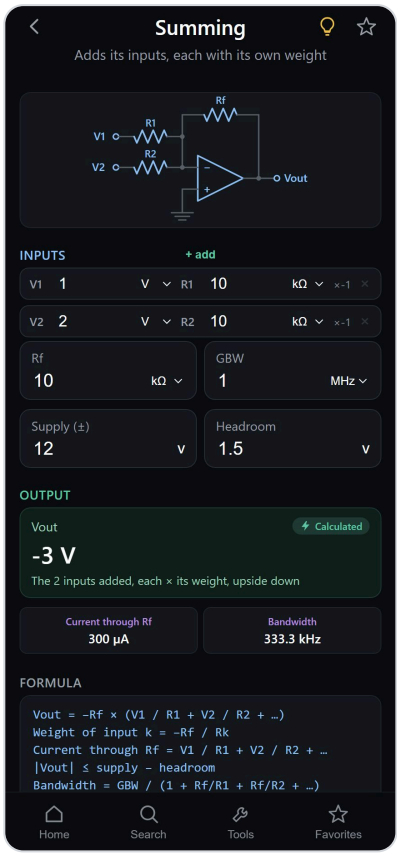
All with 10 kΩ, on ±12 V with 1.5 V headroom: - **Triangle**, 100 nF, 1 V, 100 Hz: - RC 1 ms, slope 400 V/s; - a ±400 mV square (800 mV p-p), each step 5 ms. - **Triangle**, 10 nF at 1 kHz: 4 V/ms, ±400 mV. - **Sine**, 100 nF at 1 kHz: gain 6.283 (15.96 dB), lags 90°. - **Sine** at 10 kHz: the ideal 62.83 V clips at 10.5 V. - **With Rs = 100 Ω**: - fH 15.92 kHz; - at 1 kHz the gain is 6.271 and the lag 93.6°. - **With Rs = 1 kΩ** at 1 kHz: above fH / 10 (159.2 Hz), so the tool warns that the gain tends to R / Rs = 10.

The review made these changes: - It fixed the subtitle, which said a sine comes out a quarter cycle early; the output lags by 90°. - It moved clipping from the full supply to the supply less the headroom, and put the

output peak first and large. - It shows slopes in V/s, V/ms or V/µs. - It replaced a row with three fields squeezed into two places by two rows of three. - It put an 11 px legend below the waves. - It limited the units to real ones and remembers them, rewrote the note in plain words and added examples. - On Pierre's request, it added the optional Rs, drawn in series ahead of C.

Summing

(formerly Summing amplifier)



What it computes

For the inverting summing amplifier, with two to four inputs (+ add and × on each input's line), it gives: - the output voltage; - each input's weight, shown on its line; - the current through Rf; - where the output clips, from the supply and the op-amp's headroom; - the bandwidth, from the op-amp's gain-bandwidth product.

Source

No standard applies. The output comes from the ideal op-amp model. The headroom and gain-bandwidth figures come from the TL072, LM358 and MCP6002 datasheets, as listed in the Inverting section.

What it means

- **Virtual ground** — the op-amp holds its − input at 0 V. So each input drives a current V / R through its own resistor, and is not affected by the others.
- **Adding** — those currents meet at the − input, add up, and all flow on through Rf. Rf turns the total into the output voltage, upside down (inverted).
- **Weight** — each input's weight is −Rf divided by its own resistor:
- equal resistors give a plain sum;
- unequal ones mix channels at different levels, as in an audio mixer;
- Rf equal to half the input resistors gives the average;
- binary-weighted resistors (R, 2R, 4R…) make a simple digital-to-analog converter.
- **Headroom** — how close the output can get to the supply. It is about 1.5–2 V for a TL072 or LM358, and 0 for a rail-to-rail op-amp. Past that, the output clips.
- **Bandwidth** — the op-amp's gain-bandwidth product (GBW) is shared out by the noise gain, 1 + Rf/R1 + Rf/R2 + …. Every input resistor counts, so the bandwidth is lower than any one channel's gain would suggest.

The headroom and GBW entered here are remembered for all the op-amp tools.

Why the formulas are these

The − input is at 0 V, so input k drives Vk / Rk into it. No current enters the op-amp, so the sum flows through Rf, from the − node towards the output:

```
Vout = −Rf × (V1 / R1 + V2 / R2 + …)
```

The feedback network divides the output by 1 + Rf / (R1 ‖ R2 ‖ …) on its way back to the − input, which equals 1 + Rf/R1 + Rf/R2 + …. That noise gain sets the bandwidth: GBW / (1 + Rf/R1 + Rf/R2 + …). Each input added lowers it.

Assumptions and limits

- An ideal op-amp for the sum.
- A symmetric supply, with the same headroom on both sides.
- Each source is taken as having no resistance of its own. A source's resistance adds to its input resistor and lowers its weight.
- Up to four inputs are offered, which is what still fits above the tab bar. More work the same way, each adding its own V / R term.

What it deliberately does not do

- Non-inverting summing, with the inputs on the + side, is not covered.
- Subtracting belongs to the Differential tool.

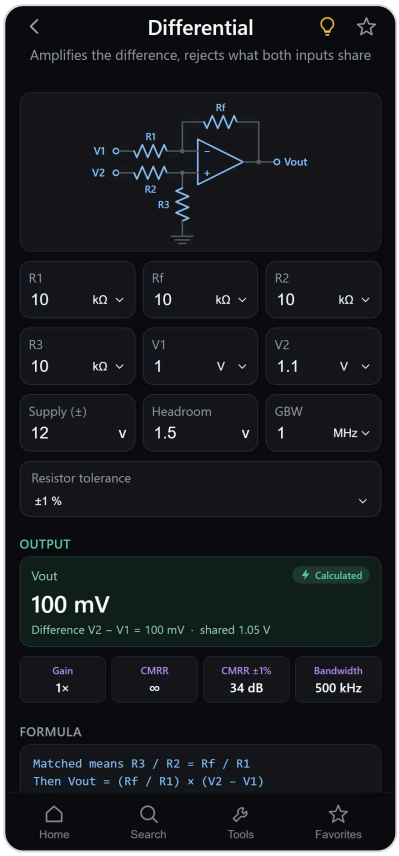
How it was checked

All on ±12 V with 1.5 V headroom and a 1 MHz GBW: - 10 kΩ all round, 1 V + 2 V: - −3 V, weights −1 and −1, 300 µA through Rf; - bandwidth 333.3 kHz (1 MHz / 3). - **+ add**, with a third input of 1 V through 10 kΩ: −4 V, 250 kHz. - Three-channel mixer, R3 = 20 kΩ, 0.5 V on each input: −1.25 V, weight −0.5 on V3, 285.7 kHz. - Averager, Rf = 5 kΩ: −1.5 V. - 3-bit DAC, 10 / 20 / 40 kΩ, code 101 (5 V, 0 V, 5 V): −6.25 V. - 6 V + 6 V: the ideal −12 V clips at −10.5 V.

The review made these changes: - It moved clipping from the full supply to the supply less the headroom, and added the bandwidth. - It put Vout first and large, replacing a row of six small cells. - It limited the units to real ones and remembers them. - It rewrote the subtitle and note in plain words, and added examples. - The tool was renamed Summing, on Pierre's request. - Also on Pierre's request, inputs can be added, up to four. Each is one compact line with its weight, and the drawing grows one arm per input.

Differential

(formerly Differential amplifier)



What it computes

For the one-op-amp difference amplifier, it gives: - the output for two input voltages; - the gain; - the common-mode rejection (CMRR), both as drawn and at worst with real resistors of a chosen tolerance; - the bandwidth; - where the output clips.

When the two arms are not balanced, it also gives how much of the output is leaked common-mode voltage.

Source

No standard applies. The output follows from the ideal op-amp model by superposition. The worst-case CMRR with resistor tolerance is derived below from the same equation. The headroom and gain-bandwidth figures come from the TL072, LM358 and MCP6002 datasheets, as listed in the Inverting section.

What it means

A **difference amplifier** outputs the difference between its inputs, V2 − V1, times a gain, and ignores the voltage both inputs share.

- **Common-mode voltage** — the shared part, (V1 + V2) / 2. It is often much larger than the difference:
- a sensor bridge sitting at 2.5 V with millivolts between its sides;
- a current shunt at 12 V;
- hum picked up equally on both wires.
- **Balanced arms** — the circuit rejects the shared part only when R3 / R2 equals Rf / R1. The gain is then Rf / R1.
- **CMRR** (common-mode rejection ratio) — how much more the difference is amplified than the shared voltage, in dB. With exactly matched resistors it is infinite.
- **CMRR with tolerance** — real resistors are off by their tolerance. At worst, four ±1 % resistors at gain 1 leave only about 34 dB: a shared 12 V would still put about 240 mV on the output. That is why precision resistors, or a ready-made difference-amplifier chip with laser-trimmed resistors, are used.
- **Input loading** — V1 sees R1 to the virtual input, and V2 sees R2 + R3 to ground. Both draw current from their sources, unlike an instrumentation amplifier.
- **Headroom** and **bandwidth** — as on the other op-amp tools. The bandwidth is GBW / (1 + Rf / R1).

The headroom and GBW are remembered for all the op-amp tools.

Why the formulas are these

With k = Rf / R1 and β = R3 / (R2 + R3), the + input sits at β × V2, and the op-amp holds the − input there too. So

```
Vout = β(1 + k) × V2 − k × V1
```

Writing V1 and V2 as the common part Vcm plus or minus half the difference Vd gives

```
Vout = [β(1 + k) + k] / 2 × Vd + [β(1 + k) − k] × Vcm
     =        Ad        × Vd +       Acm        × Vcm
```

Acm is zero exactly when β(1 + k) = k, which means R3 / R2 = Rf / R1. Then Ad = k.

Worst case with tolerance. Let the ratios Rf / R1 and R3 / R2 be off by fractions a and b. To first order, Acm = k(b − a) / (1 + k). Each ratio of two ±t resistors can be off by 2t, in opposite directions, so at worst |Acm| = 4tk / (1 + k). With Ad ≈ k:

```
CMRR (worst) = (1 + Rf / R1) / 4t
```

A higher gain helps: the same ±1 % resistors give about 49 dB at gain 10.

Assumptions and limits

- **An ideal op-amp.** The op-amp's own CMRR, usually 70–100 dB, is assumed far better than the resistors'. With 0.1 % resistors at high gain, the two become comparable.
- **Ideal sources.** Each source is taken as having no resistance of its own. A source's resistance adds to R1 or R2 and unbalances the arms.
- **Inputs within range.** The op-amp's inputs sit at β × V2, which must stay within its input range even when V1 and V2 are higher. In the 12 V shunt example they sit at about 6 V.
- **Symmetric supply**, with the same headroom on both sides.

What it deliberately does not do

- The three-op-amp instrumentation amplifier, with high input impedance, is not covered.
- Frequency-dependent CMRR is not calculated.

How it was checked

- **Four 10 kΩ, 1 V and 1.1 V:** 100 mV out, CMRR ∞ as drawn, 34 dB at ±1 %, 500 kHz on a 1 MHz op-amp.
- **Bridge at 2.5 V, 10 k / 100 k:** 2.49 V and 2.51 V give 200 mV, gain 10, 48.8 dB at ±1 %, 90.91 kHz.
- **R3 = 110 kΩ instead of 100 kΩ:**
- 409.2 mV out;
- CMRR 41.6 dB;

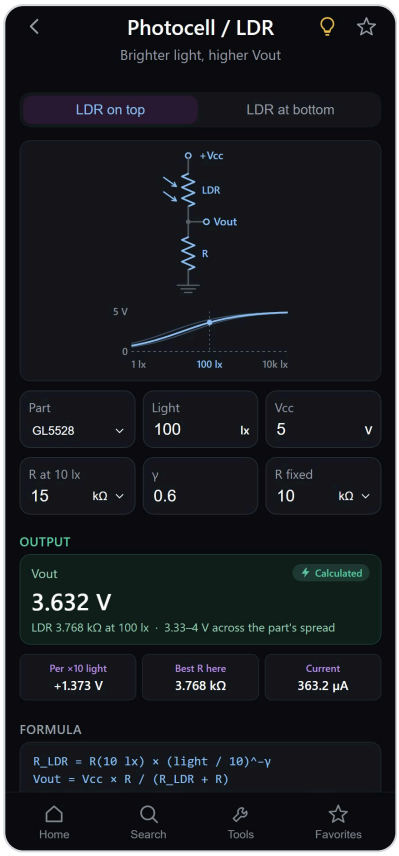
- 50.9 % of the output is leaked common-mode voltage.
- **Shunt at 12 V (11.9 V and 12 V):** 100 mV out, common-mode voltage 11.95 V.
- **±0.1 % at gain 1:** 54 dB.

These figures were checked in a separate calculation.

The review made these changes: - It moved clipping from the full supply to the supply less the headroom, and added the bandwidth. - It added the worst-case CMRR for a chosen resistor tolerance. - It put Vout first and large, replacing a row of six small cells. - It removed the waveform. That waveform showed an invented slow interference unrelated to the entered inputs. - It limited the units to real ones and remembers them. - It rewrote the note in plain words and added examples. - The tool was renamed Differential, on Pierre's request.

ACTIVE & SEMICONDUCTOR DEVICES › OPTOELECTRONICS

Photocell / LDR



What it computes

For a light-dependent resistor in a voltage divider with a fixed resistor, it gives: - the output voltage at a given light level; - the LDR's resistance at that light; - the range of output the part's 2:1 manufacturing spread can give; - how much the output changes for each ×10 of light; - the fixed resistor that would make the divider most sensitive; - the current drawn.

It plots the output across four decades of light.

Source

GL55-series data from the series' datasheet, "GL55 Series CdS Photoresistor Manual" (5 pages; the copy read names no manufacturer). - **Test conditions** — light resistance is measured at 10 lx under standard light source A (2856 K), after two hours at 400–600 lx. - **γ** — defined as lg(R10 / R100), the logarithm of the ratio of the resistances at 10 and 100 lx.

Part	R at 10 lx	Dark resistance (min)	γ	Rise / fall
GL5516	5–10 kΩ	0.5 MΩ	0.5	30 / 30 ms
GL5528	10–20 kΩ	1 MΩ	0.6	20 / 30 ms
GL5537-1	20–30 kΩ	2 MΩ	0.6	20 / 30 ms
GL5537-2	30–50 kΩ	3 MΩ	0.7	20 / 30 ms
GL5539	50–100 kΩ	5 MΩ	0.8	20 / 30 ms
GL5549	100–200 kΩ	10 MΩ	0.9	20 / 30 ms

Other vendors' sheets for the same names differ a little. One gives 8–20 kΩ for the GL5528, for example, so the spread is real.

What it means

An **LDR** (light-dependent resistor, photoresistor or photocell) is a resistor whose value falls as the light rises. It is thousands of ohms in a dim room, a few hundred in daylight, and megohms in the dark. The common ones are made of cadmium sulphide (CdS).

- **Lux (lx)** — the unit of illuminance, how much light falls on a surface. Rough values:
- about 1 lx at deep twilight;
- 50–100 lx in a dim room;
- 300–500 lx in an office;
- 10 000 lx and more in daylight.
- **R at 10 lx** — the datasheet's reference resistance.
- **γ** — how steeply the resistance falls: each ×10 of light divides it by 10^γ. With γ = 0.6, 100 lx gives about a quarter of the 10 lx value.
- **The divider** — the LDR and a fixed resistor R between the supply and ground. The midpoint, Vout, follows the light:
- with the LDR **on top**, brighter light gives a higher Vout;
- with it **at the bottom**, darker gives a higher Vout, as for a night light that switches on at dusk.
- **Sensitivity** — the change of Vout for each ×10 of light. It is largest when R equals the LDR's resistance at the light level you care about, which puts Vout at half the supply. "Best R here" gives that value.
- **Spread** — pieces of the same part differ by up to 2:1 at 10 lx. With a part chosen, the tool shows the range of Vout this gives, and draws it as two faint curves.

Choosing a part fills in its middle R at 10 lx and its γ. Typing your own values switches the choice to Custom.

Why the formulas are these

The datasheet's γ = lg(R10 / R100) describes a straight line of slope −γ on log-log axes, so

```
R_LDR = R(10 lx) × (light / 10)^−γ
```

The divider gives Vout = Vcc × R / (R_LDR + R) with the LDR on top, or Vcc × R_LDR / (R_LDR + R) with it at the bottom.

Differentiating with respect to $\log_{10}$ of the light gives

```
± ln 10 × γ × Vcc × R × R_LDR / (R_LDR + R)²
```

per ×10 of light. Setting its derivative with respect to R to zero gives R = R_LDR.

Assumptions and limits

- **The power law** is a straight-line fit through the datasheet's two defining points, 10 lx and 100 lx. It is least reliable far from them. Below about 1 lx the cell heads towards its dark resistance instead, and the tool warns. The dark resistance is measured 10 s after the light is switched off, so the cell also recovers slowly.

- **Slow.** CdS cells take tens of milliseconds to respond (20–30 ms here), so they suit day/night switching, not fast signals.
- **Temperature and history.** The resistance also drifts with temperature and with recent exposure. An LDR is not a precision light meter.
- **Light source.** The values are for an incandescent-like source (2856 K). LEDs and daylight have different spectra, and the cell peaks at 540 nm (green).
- **No loading** — the circuit reading Vout is assumed to draw no current.

What it deliberately does not do

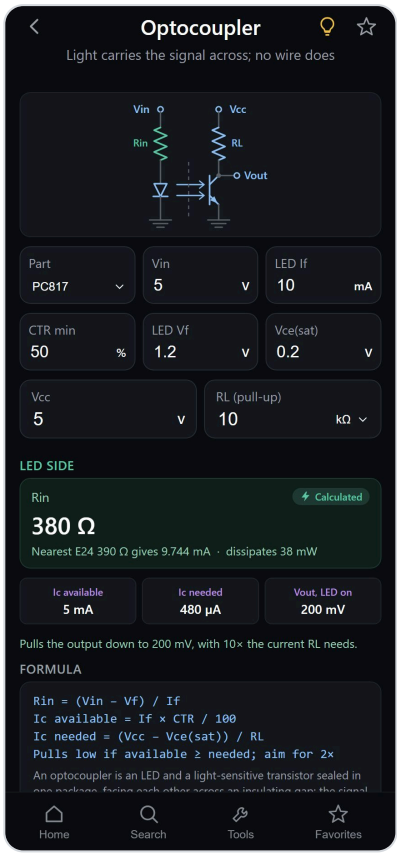
- Photodiodes and phototransistors, which are faster and linear, are not covered.
- Switching thresholds with hysteresis belong to the Comparators tool.

How it was checked

All at 5 V, with the GL5528's middle value, 15 kΩ at 10 lx, γ 0.6: - **On top, 100 lx, R = 10 kΩ:** - LDR 3.768 kΩ, Vout 3.632 V; - 3.33–4 V across the 10–20 kΩ spread; - +1.373 V per ×10 of light, 363.2 µA. - **At the bottom, 10 lx, R = 15 kΩ:** exactly 2.5 V. - **300 lx:** 1.949 kΩ, and 2.532 V with R = 2 kΩ. - **GL5516 at 10 000 lx:** 237.2 Ω, and 4.884 V with R = 10 kΩ.

The review made these changes: - It added the GL55 parts from their datasheet, with the spread shown in the result and on the curve. - It added the choice of LDR on top or at the bottom. - It put Vout first and large. - It enlarged the curve's 8 px labels to 11 px. - It limited the units to real ones and remembers them. - It rewrote the note in plain words, with typical light levels, and added the warning below 1 lx and examples.

Optocoupler



What it computes

For a phototransistor optocoupler driven through a resistor, with a pull-up on the output side: - the input resistor Rin for the LED current wanted, the nearest E24 value, and the power in it; - the collector current the minimum CTR makes available; - the current the pull-up needs to pull the output low; - the output voltage with the LED on; - whether the output really pulls low, and with how much margin.

Source

Data read from the manufacturers' datasheets themselves.

Sharp PC817X series (sheet D2-A03101EN, 2003): - **CTR by rank** — measured at IF = 5 mA, VCE = 5 V, from the collector-current bins:

Rank	CTR
A	80–160 %
B	130–260 %
C	200–400 %
D	300–600 %
Unranked	50–600 %

- **VF** — 1.2 V typical, 1.4 V maximum, at 20 mA.
- **VCE(sat)** — 0.1 V typical, 0.2 V maximum, at IF = 20 mA, IC = 1 mA.
- **Rise / fall time** — 4 / 3 µs typical (VCE 2 V, IC 2 mA, RL 100 Ω).

Vishay 4N35 (document 81181, rev. 1.2, 2010): - **CTR** — at least 100 % at IF = 10 mA, VCE = 10 V. Over −55 to +100 °C it is at least 40 %, 50 % typical. - **VF** — 1.3 V typical, 1.5 V maximum, at 10 mA. - **Switching time** — 10 µs typical (VCC 10 V, IC 2 mA, RL 100 Ω). - **VCE(sat)** — no limit is given. Its curves of saturated CTR are drawn at VCE = 0.4 V, which the tool uses.

What it means

An **optocoupler** is an LED and a light-sensitive transistor sealed in one package, facing each other across an insulating gap. The signal crosses as light, so the two circuits share no wire. They can sit at very different voltages, a microcontroller and mains equipment for example; the PC817 and 4N35 are both rated for 5 kV rms between their sides.

- **Rin** — sets the LED current If, as for any LED: Rin = (Vin − Vf) / If.

- **CTR** (current transfer ratio) — the collector current the transistor can pass for a given LED current, in %. A CTR of 50 % turns 10 mA of LED current into 5 mA available on the output side.
- **Pull-up RL** — from Vcc to the transistor's collector:
- with the LED off, the output reads high;
- with it on, the transistor pulls the output low, provided it can pass the current RL needs: (Vcc − Vce(sat)) / RL.
- **Vce(sat)** — the voltage left across the transistor when it is fully on: 0.2 V for the PC817.
- **Margin** — CTR varies widely between pieces, which is why the PC817 is sold in ranks. It is also lower when the transistor is saturated, and it falls as the LED ages and with temperature.
- Design on the **minimum** CTR, and aim for about **2×** the current RL needs. That 2× is a rule of thumb, not a datasheet figure.
- Below 1× the output stops part way down, between logic levels.

Choosing a part fills in its minimum CTR, Vf and Vce(sat). Typing your own values switches it to Custom.

Why the formulas are these

```
Rin = (Vin − Vf) / If
Ic available = If × CTR / 100
Ic needed = (Vcc − Vce(sat)) / RL
```

If Ic available ≥ Ic needed, the transistor saturates and Vout = Vce(sat). Otherwise it passes only Ic available, and Vout = Vcc − Ic available × RL.

Assumptions and limits

- **CTR at the test current.** CTR changes with If; both datasheets plot it against If. The ranks are specified at 5 mA (PC817) and 10 mA (4N35), so far from those currents read the curve.
- **Speed.** Phototransistor optocouplers are slow, a few microseconds, and a larger RL makes them slower. Fast signals need a logic-output part.
- **The input source** is taken as ideal, and the LED is driven with steady DC.
- **No load** on Vout other than RL.

What it deliberately does not do

- Switching speed is not calculated.
- Triac-output and logic-output optocouplers are not covered.

How it was checked

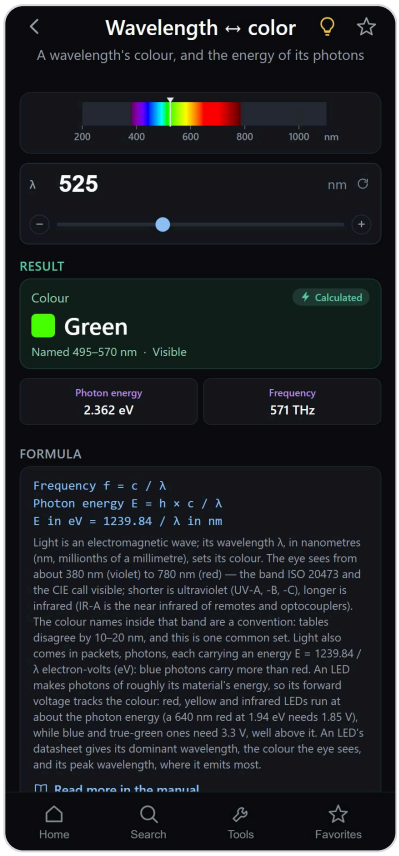
- **PC817, any rank, 5 V, 10 mA, 10 kΩ to 5 V:**
- Rin 380 Ω, nearest E24 390 Ω giving 9.744 mA, 38 mW;
- 5 mA available, 480 µA needed: 10× margin, Vout 200 mV.
- **3.3 V drive at 5 mA:** 420 Ω, 5.2× margin.
- **1 kΩ to 12 V with 2 mA:** needs 11.8 mA but only 1 mA is available, so the output stops at 11 V.

- **4N35 at 10 mA from 5 V:** 370 Ω, nearest E24 360 Ω, Vout 400 mV.
- **1 mA at 50 %:** only 1× over, so the tool flags it.

The review made these changes: - The output check used a fixed 0.3 V saturation. It now takes the part's datasheet Vce(sat) and minimum CTR, from a choice of PC817 ranks or the 4N35. - It added the 2× margin advice. - It put Rin first and large, and draws it green as the calculated part. - It added the nearest E24 value. - It limited the units to real ones, rewrote the note in plain words and added examples.

Wavelength ↔ color

(formerly Light wavelength ↔ color)



What it computes

For a wavelength from 200 nm to 1100 nm, it gives: - the colour name, with a swatch; - the named region it falls in (ultraviolet, visible or infrared, and which part of them); - the energy of one photon, in electron-volts; - the frequency.

It marks the wavelength on a drawn spectrum.

Source

- **Region limits** — the UV divisions follow ISO 20473:2007, for optics and photonics: UV-A 315–380 nm, UV-B 280–315 nm, UV-C 100–280 nm. The visible band is 380–780 nm, and IR-A is 780–1400 nm and IR-B 1400–3000 nm, as the CIE defines them.
- These limits were read from Opsytec's published table of spectral ranges by standard (2024), not from the standards themselves.
- The CIE notes that visible light has no precise limits: the lower one lies between 360 and 400 nm, the upper one between 760 and 830 nm.
- **Colour names** — no standard names the colours inside the visible band. Published tables disagree by 10–20 nm, and some add cyan or indigo. The tool uses one common set: violet from 380 nm, blue 450, green 495, yellow 570, orange 590, red 620 to 780.
- **The examples' LEDs** — taken from their datasheets: Kingbright WP7113SRC/DU, WP7113ZGC and WP7113QBC/D, and Vishay TSAL6200.
- **The swatch** — drawn with Dan Bruton's wavelength-to-RGB approximation, a way to put a colour on screen. It is not colorimetry.

What it means

- **Wavelength** — light is an electromagnetic wave, and its wavelength λ, in nanometres (millionths of a millimetre), sets its colour. The eye sees about 380 nm (violet) to 780 nm (red).
- Shorter than that is **ultraviolet**: UV-A, then UV-B, then UV-C.
- Longer is **infrared**: IR-A is the near infrared that remote controls and optocouplers use.
- **Photon energy** — light comes in packets, photons, each carrying an energy E = 1239.84 / λ electron-volts (eV). Blue photons carry more than red.
- **LEDs** — an LED makes photons of roughly its material's energy, so its forward voltage tracks its colour:
- red, yellow and infrared LEDs run at about the photon energy: a 640 nm red, at 1.94 eV, needs 1.85 V;
- blue and true-green LEDs need about 3.3 V, well above it.
- **Dominant and peak wavelength** — an LED datasheet gives both. The dominant wavelength is the colour the eye sees; the peak is where the LED emits most. Names follow the dominant one.

- **Naming varies** — Kingbright sells its WP7113ID, at 617 nm, as "High Efficiency Red", yet 617 nm is orange in the set used here.

Why the formulas are these

```
f = c / λ,   E = h × f = h × c / λ
```

With h × c = 1239.84 eV·nm, E in eV is 1239.84 divided by λ in nm. With c = 299 792 458 m/s, 525 nm is 571 THz.

Assumptions and limits

- **Single wavelengths.** Real sources emit a band. LEDs are 20–45 nm wide (half-width, from the same datasheets), and white light mixes many wavelengths.
- **Approximate swatch.** Screens cannot show pure spectral colours. The swatch is an approximation, and blank outside 380–780 nm.

What it deliberately does not do

- Colour mixing and colour spaces (RGB, CIE xy) are not covered.
- The eye's sensitivity (lumens) is not calculated.

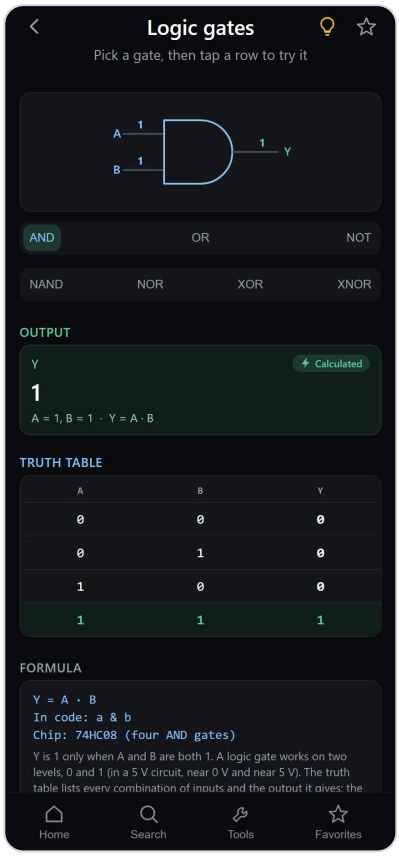
How it was checked

- **525 nm:** green, visible, 2.362 eV, 571 THz.
- **Red LED at 640 nm:** red, 1.937 eV.
- **617 nm:** orange in this set, 2.009 eV.
- **IR LED at 940 nm:** infrared, IR-A, 1.319 eV.
- **760 nm:** still red, since visible light runs to 780 nm.
- **365 nm:** ultraviolet, UV-A.

The review made these changes: - Infrared started at 750 nm; it now starts at 780 nm, the limit ISO 20473 and the CIE use. - It added the named UV and IR regions. - It removed a source attribution ("Britannica") that had never been checked. - It corrected the claim that photon energy is the floor of an LED's forward voltage: a red LED runs slightly below it. - It put the colour first and large, with its swatch. - It enlarged the strip's 9 px labels to 11 px. - It added LED examples from datasheets and rewrote the note in plain words. - It shortened the name to Wavelength ↔ color, which fits on the phone.

Logic gates

(formerly Basic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR))



What it computes

For each of the seven basic gates (AND, OR, NOT, NAND, NOR, XOR and XNOR), it shows: - the gate's symbol; - its truth table; - its Boolean expression; - the same operation written in code; - a common chip that contains it.

Tap a row of the truth table to set the inputs. The symbol then shows the levels on its wires, and the output Y appears large.

Source

- **Symbols** — these are the "distinctive shape" symbols: a D for AND, a shield for OR, a triangle for NOT. Wikipedia's "Logic gate" article says ANSI/IEEE Std 91-1984 (and its supplement 91a-1991) defines both these and the rectangular IEC-style symbols, and that the distinctive shapes come from MIL-STD-806. The standard itself was not read. The app uses the distinctive shapes throughout, as it uses zigzag resistors.
- **Chips** — Texas Instruments' product pages confirm the SN74HC00 (four NAND gates), SN74HC04 (six inverters), SN74HC08 (four AND) and SN74HC32 (four OR), all for 2–6 V. Distributor listings of TI parts confirm the 74HC02 (four NOR) and 74HC86 (four XOR).
- The 74HC7266 (four XNOR) is in a Philips/Nexperia datasheet listed by a distributor. TI's own version is obsolete.

What it means

A logic gate works on two levels, 0 and 1. In a 5 V circuit these are a voltage near 0 V and one near 5 V. The truth table lists every combination of inputs and the output each gives. A bar over an expression means "inverted".

Gate	Output is 1 when…	Expression	In C / Python
AND	A and B are both 1	$Y = A \cdot B$	a & b
OR	A or B, or both, is 1	$Y = A + B$	a | b
NOT	A is 0 (it inverts)	$Y = \bar{A}$	!a / not a
NAND	not both are 1	$Y = \overline{(A \cdot B)}$	!(a & b) / not (a & b)
NOR	both are 0	$Y = \overline{(A + B)}$	!(a | b) / not (a | b)
XOR	A and B differ	$Y = A \oplus B$	a ^ b
XNOR	A and B are the same	$Y = \overline{(A \oplus B)}$	!(a ^ b) / not (a ^ b)

- **NAND and NOR are universal** — any logic, however complex, can be built from either one alone. A NAND with its inputs tied together is a NOT.
- **XOR and AND add two bits** — XOR gives the sum and AND the carry, which makes a half adder.
- **Unused inputs** — on CMOS chips such as the 74HC series, an input left floating can make the gate switch at random and draw extra current. Tie every unused input to 0 V or to the supply.

Why the formulas are these

Each truth table is computed from the gate's function, so a row cannot disagree with the symbol. The code forms use the bitwise operators on the values 0 and 1. In Python, `not` returns True or False rather than 1 or 0.

Assumptions and limits

- **Ideal logic levels.** Real gates have input thresholds, output drive limits and a propagation delay, given on each chip's datasheet. They are not modelled.
- **Two-input versions.** Chips with three or more inputs per gate exist (for example triple 3-input AND); the logic extends the same way.

What it deliberately does not do

- Simplifying a larger expression belongs to Karnaugh map simplification.
- Flip-flops, which remember, have their own tool.

How it was checked

- **AND:** with A = 1, B = 1, Y = 1; tapping the 0 1 row gives 0.
- **XOR:** 1 0 gives 1, and 1 1 gives 0, the sum bit of a half adder.
- **NOT:** A = 1 gives Y = 0.
- **NAND:** with both inputs at 1, Y = 0 (the NOT example).
- **Remembered:** the chosen gate comes back when the tool reopens.

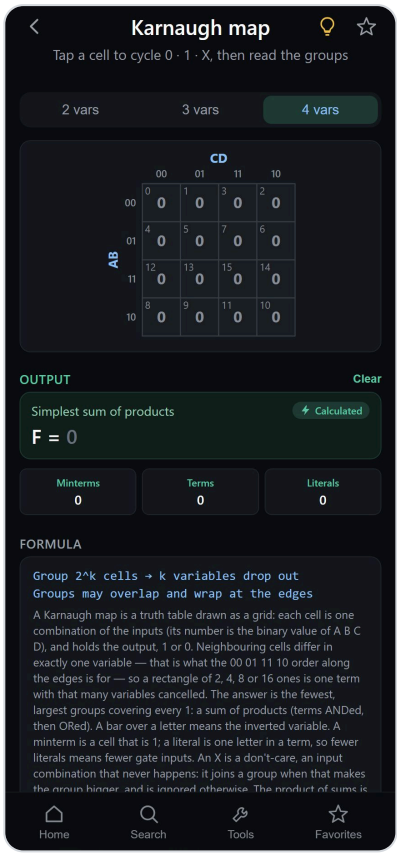
The review made these changes: - It corrected the standard cited ("ANSI/IEEE 315") to IEEE Std 91. - It made the truth table tappable, with the levels shown on the symbol and Y large. - For each gate, it added the

description in plain words, the code form and a common 74HC chip, with the CMOS unused-input advice. - It added examples and remembers the chosen gate. - The tool was renamed Logic gates, on Pierre's choice, to fit on one line.

DIGITAL & LOGIC CIRCUITS › LOGIC GATES

Karnaugh map

(formerly Karnaugh map simplification)



What it computes

Takes a truth table for 2, 3 or 4 variables, where each cell is 0, 1 or a don't-care. It returns: - the minimum sum-of-products expression, with each product term drawn as a group on the map in its own colour; - the minimum product of sums, under the expression; - the exclusive-OR form, when the function is a parity.

Tap a cell to cycle it through 0, 1 and X. The number of variables is remembered, and four examples in the bulb load ready-made maps.

Plain words: - **Cell** — one combination of the inputs. Its small number is the binary value of A B C D, and it holds the output. - **Minterm** — a cell that is 1. - **Literal** — one letter in a term, plain or barred. A bar means the inverted variable. - **Sum of products** — terms ANDed, then ORed together. - **Product of sums** — ORs ANDed together. - **Don't-care (X)** — an input combination that never happens, so its output may be whatever helps.

Source

The map is Karnaugh's (1953), itself a reworking of the Veitch diagram. The minimisation is Quine–McCluskey, which is an exact algorithm rather than a heuristic; the covering step is solved by exhaustive search, which is affordable at four variables and would not be at twenty.

Why the formulas are these

The whole method rests on the ordering along the edges: **00, 01, 11, 10**, not 00, 01, 10, 11. That is Gray code, and it means any two side-by-side cells differ in exactly one variable. When two adjacent cells are both 1, that variable appears once true and once complemented, so it cancels:

$$A \cdot \bar{B} \cdot C + A \cdot B \cdot C = A \cdot C \cdot (\bar{B} + B) = A \cdot C$$

Extend it and a rectangle of 2^k cells cancels k variables. A pair drops one, a block of four drops two, eight drops three. That is the entire trick — the map is just an arrangement that puts algebraically adjacent terms physically next to each other.

The edges wrap. The first and last columns also differ in one variable (00 and 10 differ only in the middle bit), and so do the first and last rows. The map is really a torus. That is why the four corners of a four-variable map form one group of four, and why a group can run off one edge and continue on the other. A torus

rectangle has no single box on a flat sheet, so the tool draws wrapping groups as two or four pieces in the same colour.

Quine–McCluskey. Terms are combined pairwise wherever they differ in one bit, the shared bit becoming a dash; repeat until nothing more combines. What never combined is a **prime implicant** — a group that cannot be made larger. Then a cover must be chosen: a minterm covered by exactly one prime implicant forces that implicant in (it is *essential*), and the rest is a set-cover problem solved here by branch and bound.

The cover is ordered **fewest terms first, then fewest literals**. The second key matters: several covers can tie on term count while differing in how many variables survive, and the answer a textbook prints is the one with fewer letters.

Don't-cares are included when forming groups but never required to be covered. That is where most of the saving usually comes from: an X adjacent to a group lets the group double in size, dropping another variable for free.

Parity is what the map cannot see. A checkerboard of ones has no two adjacent cells, so every group is a single cell and the sum of products comes out at full length — twelve of twelve literals on a three-variable map, nothing saved at all. That function is exclusive-OR:

```
A ⊕ B ⊕ C = 1 whenever an odd number of A, B, C are 1
```

Two XOR gates implement it against four ANDs and an OR for the sum of products. The map cannot find this because XOR is not a sum of products in any small form — its minimal SOP genuinely is the long one. The tool therefore tests, separately from the map, whether the function is the parity of any subset of its variables (or the complement of one) and prints that form when it is.

Assumptions and limits

- Two to four variables. Five and six variable maps exist, drawn as stacked pairs, but they stop being readable and the whole point of the map is that you can see it.
- The result is a minimum, not *the* minimum: several different expressions can tie on both term and literal count, and the tool shows one of them.
- Literal count is a proxy for cost, not a gate count. It ignores that inverters may be free if a signal is already available complemented, and that fan-in is limited in real logic families.
- No account of hazards. A minimal cover can contain a static hazard — a glitch when two inputs change at once — which is removed by adding a redundant group that the minimiser will never choose because it costs a term.

What it deliberately does not do

- **More than four variables**, as above.
- **Hazard-free covers.** Adding the redundant terms is a different objective from minimisation and would make the answers disagree with what a textbook shows.

How it was checked

A second minimiser was written independently for the purpose: enumerate every possible cube, discard those covering a zero, and find the smallest cover by iterative deepening. The two were compared over 3500 random functions at 2, 3 and 4 variables, and 12000 more were checked only for the expression reproducing the truth table at every care position.

That comparison earned its keep. Term counts matched from the start, but 34 of the first 3400 came back with more literals than the reference — the cover search was returning whichever minimum-size cover it reached first, with no tie-break on literals. A correct minimum by term count, and still not the printed answer.

The parity detector was checked exhaustively over all 256 three-variable functions: exactly eight are parity forms (three pairs and one triple, each with its complement), and each one's exclusive-OR form reproduces its truth table.

The product of sums (added in the 2026 review) is the same exact minimisation run on the zeros, keeping the same don't-cares. De Morgan's law then turns each group of zeros into a sum of inverted letters:

$$\bar{F} = \overline{AB} + \ldots \quad \rightarrow \quad F = (A + B)(\ldots)$$

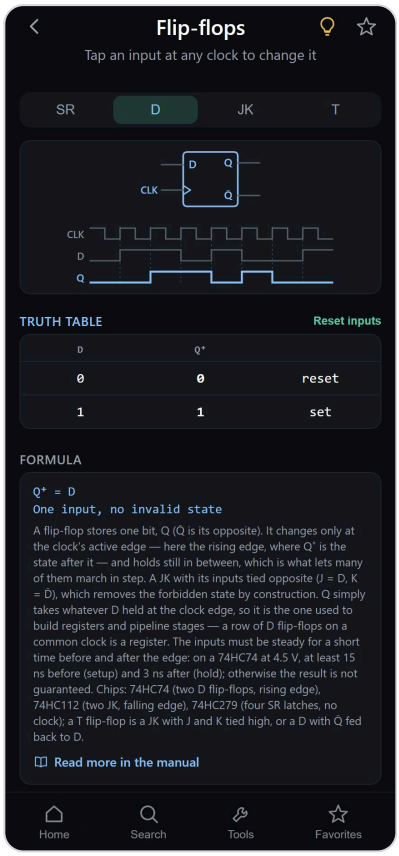
It was checked in the browser over 6000 random functions of 2 to 4 variables, with don't-cares. At all 47,615 positions that are not don't-cares, the product of sums reproduced the truth table.

The examples: - **Majority of three** — AB + AC + BC, with the product of sums (A + B)(A + C)(B + C). - **BCD digit 5 or more** — with codes 10–15 as don't-cares, A + BC + BD. - **4-input parity** — eight terms of four letters, and A ⊕ B ⊕ C ⊕ D. - **The four corners** — they wrap around into the single term $\bar{B}\bar{D}$.

The review made these changes: - It added the product of sums, which an earlier version left out as not worth a second screen. It needs only one line. - It put the answer first and large. - It moved the cells' numbers above the group outlines, at 11 px. - It added plain definitions, examples, and remembers the number of variables. - The tool was renamed Karnaugh map, on Pierre's choice, to fit beside the bulb.

Flip-flops

(formerly Flip-flops (SR, D, JK, T))



What it computes

For the four flip-flop types (SR, D, JK and T), it shows: - the symbol; - the truth table: what the next clock edge does in each input case; - the characteristic equation; - an eight-clock timing diagram, with the output Q computed by running the inputs through the flip-flop.

Tap an input at any clock in the timing diagram to flip it, and Q is recomputed. "Reset inputs" brings back the demonstration sequence. The chosen type is remembered.

Source

The behaviour of each type is textbook logic. The chips and timing figures were read from manufacturers' documents: - **Nexperia 74HC74** (rev. 9, 2 April 2024): - a dual D flip-flop, triggered on the rising clock edge, with asynchronous active-low set and reset; - at 4.5 V and 25 °C, D must be steady at least 15 ns before the edge (setup) and 3 ns after it (hold); - the clock runs up to at least 24 MHz. - **Nexperia 74HC112** — a dual JK flip-flop, triggered on the *falling* edge, with set and reset. - **ST M74HC279** — four SR latches, with no clock.

What it means

A **flip-flop** stores one bit, Q; $\bar{Q}$ is its opposite. It changes only at the clock's active edge, here the rising edge, and holds still in between. That is what lets many of them, such as the bits of a register or a counter, change in step. $\mathbf{Q^+}$ is the state after the next edge.

- **SR** (set / reset):
- S = 1 sets Q to 1, R = 1 resets it to 0, and both at 0 holds it;
- S = R = 1 is forbidden: both outputs are driven the same way, and when the inputs are released together the state is unpredictable. The timing diagram draws that unknown stretch dashed with a "?", until the next set or reset defines Q again;
- on its own, without a clock, the circuit is an SR *latch*.
- **D** (data) — Q takes the value D had at the edge and ignores D in between. One D flip-flop is one bit of a register.
- **JK** — like SR, but J = K = 1 toggles Q instead of being forbidden. It is the general-purpose type: D and T are both a JK with its inputs tied.
- **T** (toggle) — T = 1 flips Q at every edge, so Q is a square wave at half the clock frequency, with a 50 % duty cycle whatever the clock's.
- Chained, n stages divide by 2^n: a ripple counter.
- It is built from a JK with J and K tied high, or a D with $\bar{Q}$ fed back to D.

- **Setup and hold** — the inputs must be steady for a short time before and after the edge, or the result is not guaranteed. A datasheet gives both, as above for the 74HC74.

Why the formulas are these

The characteristic equations follow from the truth tables:

```
SR:  Q⁺ = S + R̄ · Q        (with S · R = 0)
D:   Q⁺ = D
JK:  Q⁺ = J · Q̄ + K̄ · Q
T:   Q⁺ = T ⊕ Q
```

In the timing diagram, each input holds across a whole clock cycle, and Q updates on the rising edge that ends it. Q starts at 0. After SR's forbidden input, Q is unknown. A hold or toggle keeps an unknown unknown, and only a set or reset ends it.

Assumptions and limits

- **One edge for all four.** All four types are drawn edge-triggered on the rising edge. Some chips act on the falling edge (74HC112), and latches are level-sensitive instead.
- **Ideal timing.** Propagation delay, setup and hold violations and metastability are not simulated; the inputs are taken as meeting the setup and hold times.
- **No set or reset pins.** The asynchronous set and reset inputs of real chips are not drawn.

What it deliberately does not do

- Counters and shift registers built from several flip-flops are not simulated.

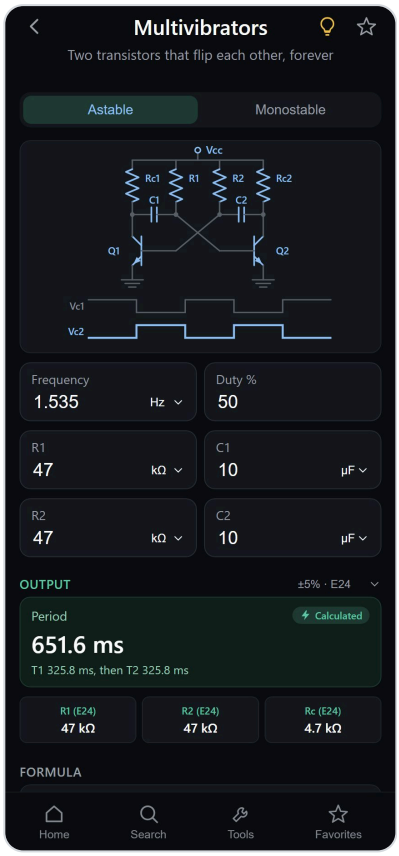
How it was checked

- **D, demonstration inputs:** Q runs 0 0 1 1 0 1 0 0.
- **Tapping D high at the first clock** makes it 0 1 1 1 0 1 0 0, and "Reset inputs" restores it.
- **T held at 1:** 0 1 0 1 0 1 0 1, half the clock frequency.
- **SR with set, hold, S = R = 1, two holds, reset, hold:** 0 1 1 ? ? ? 0 0. Q is unknown from the forbidden input until the reset.

The review made these changes: - It fixed the forbidden SR input. The old diagram quietly kept Q unchanged, as if the state were known; it now shows it unknown until the next set or reset. - It made the inputs tappable, with a reset. - It enlarged the 10 px labels to 11 px. - It added setup and hold times and the chips from Nexperia and ST documents. - It added plain-word explanations and examples, and remembers the type. - The tool was renamed Flip-flops, on Pierre's choice, to fit beside the bulb.

Multivibrators

(formerly Multivibrators (astable/monostable))



What it computes

For the classic two-transistor multivibrator, the tool works in two modes.

Astable (free-running): - the frequency and duty cycle from the two resistor–capacitor pairs, or the resistors from a frequency and duty you type; - the period and its two halves; - the same circuit in standard E-series values, with a suggested collector resistor.

Monostable (one-shot): - the pulse width from R and C, or R from the width you type; - the capacitor's recovery time; - the shortest time between triggers and the highest trigger rate; - the E-series value of R.

Every field works in both directions, and the timing diagrams are drawn to the computed times.

Source

- **The timing** is the standard textbook analysis of the circuit. Each half-cycle is one RC time constant times ln 2, about 0.693 RC, with Vbe and Vce(sat) neglected as the references do.
- **The emitter–base limit** is read from the datasheets: VEBO is 6 V for a 2N3904 (Diodes Inc. MMBT3904, DS30036) and a BC547 (Fairchild, rev. A2), and 5 V for the BC548–BC550 (same Fairchild sheet).

What it means

A **multivibrator** is two transistors that switch each other on and off through capacitors.

- **Astable** — it has no stable state, so it oscillates by itself: an LED flasher, a tone.
- **Monostable** — it has one stable state. A trigger flips it for one timed pulse, and it returns by itself: a one-shot.
- **T1 and T2** — the two halves of the astable's cycle, set by R1·C1 and R2·C2.
- **Duty** — the share of the period spent in the first half, T1.
- **How a half-cycle works** — when one transistor switches on, its collector drops and pulls the other transistor's base, through the coupling capacitor, to about −Vcc. That base then climbs back through its resistor toward +Vcc and switches on as it passes zero, after ln 2 × R × C. The supply voltage cancels out, so the frequency does not depend on it.
- **The catch with high supplies** — the reversed base reaches about −Vcc. A small transistor's emitter–base junction withstands only 5–6 V in reverse (VEBO). On a higher supply it breaks down, and the circuit runs faster than the formula says.
- **The collector resistor Rc** — squeezed from both sides:

- at least R / β (β, or hFE, is the current gain), or the transistor stops switching fully on;
- at most about R / 10, or the collector's rising edge softens.

The tool suggests R / 10 in your E-series. - **Monostable recovery** — after each pulse the capacitor must recharge through Rc, about 5 × Rc × C, before the next trigger does anything useful. Triggering faster gives short pulses, not more of them. The large result is that shortest interval. - **Rb** (monostable) only holds Q1's base low while Q2 is on. It sets no timing, so it is not calculated.

Why the formulas are these

```
T1 = ln 2 × R1 × C1,   T2 = ln 2 × R2 × C2
Frequency = 1 / (T1 + T2),   Duty = T1 / (T1 + T2)
Pulse width = ln 2 × R × C,   Recovery ≈ 5 × Rc × C
Shortest time between triggers = Pulse width + Recovery
```

- **ln 2** — a capacitor charging toward +Vcc from −Vcc crosses zero when half the 2·Vcc swing is done. That takes RC × ln 2.
- **Typing a frequency** — scales both resistors and keeps the duty and the capacitors.
- **Typing a duty** — shares the period between them and keeps the frequency.
- **The capacitors are never rewritten**, since they are the part you have in the drawer.

Assumptions and limits

- **Ideal switches.** Vbe and Vce(sat) are neglected. With them, the base starts at about Vbe − (Vcc − Vce(sat)) and switches on at Vbe rather than 0 V, so each half takes a little longer: about 0.72 RC instead of 0.693 RC on 9 V, and 0.74 RC on 5 V. The real frequency is therefore a few percent lower than calculated, more so on low supplies.
- **The 5 RC recovery** is a rule of thumb (99 % recharged).
- **Capacitor tolerance**, often ±20 % for electrolytics, matters more than the resistors' tolerance for the timing.

What it deliberately does not do

- The 555 timer versions have their own tool.
- Bistable multivibrators, which are flip-flops, have their own tool.

How it was checked

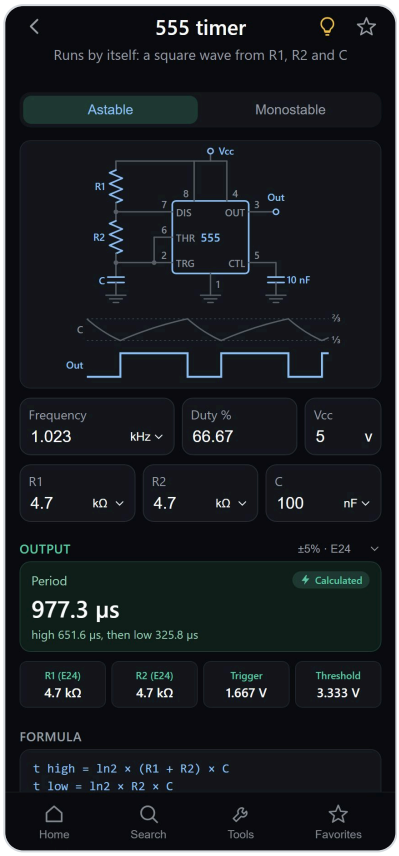
- **47 k / 10 µF on both sides:** period 651.6 ms (T1 = T2 = 325.8 ms), 1.535 Hz, Rc 4.7 kΩ suggested.
- **68 k / 10 µF:** 1.061 Hz.
- **10 k and 100 k with 10 µF:** T1 69.31 ms, T2 693.1 ms, duty 9.091 %.
- **10 k / 68 nF:** 1.061 kHz.
- **Monostable, 68 k / 10 µF with Rc 6.8 k:**
- pulse 471.3 ms;
- recovery 340 ms;

- shortest interval 811.3 ms, up to 1.233 Hz.
- **Typing 2 Hz:** both 10 µF resistors become 36.067 kΩ.

The review made these changes: - It put the period, or the shortest trigger interval, first and large. - It limited the units to real ones (Ω–MΩ, pF–µF, µs–s). - It enlarged the timing labels to 11 px. - It added plain definitions, with the VEBO figures from datasheets. - It added examples, and remembers the mode and the tolerance. - The tool was renamed Multivibrators, on Pierre's choice.

555 timer

(formerly 555 Timer (astable/monostable))



What it computes

The tool covers the 555 timer chip in its two classic circuits.

Astable (free-running): - the frequency and duty cycle from R1, R2 and C, or the resistors from a frequency and duty you type; - the period, with its high and low times; - the two resistors in standard E-series values; - the trigger and threshold voltages, ⅓ and ⅔ of Vcc.

Monostable (one-shot): - the pulse width from R and C, or R from the width you type; - R in standard E-series values; - the trigger and threshold voltages.

Every field works in both directions. The timing diagram, with the capacitor's voltage above the output, is drawn to the computed times. The mode and the tolerance are remembered.

Source

Texas Instruments, **NA555 / NE555 / SA555 / SE555 datasheet, SLFS022K** (revised March 2026), was read for: - **the formulas**: - astable: tH ≈ 0.693 (RA + RB) C, tL ≈ 0.693 RB C, f ≈ 1.44 / ((RA + 2 RB) C), duty (RA + RB) / (RA + 2 RB); - monostable: tw ≈ 1.1 RA C.

RA and RB are R1 and R2 here. - **the levels**: trigger about ⅓ Vcc and threshold about ⅔ Vcc. At 5 V the trigger level is 1.1–2.2 V (1.67 V typical). - **the supply**: 4.5 to 16 V for the NE555. - **the frequency**: TI advises 100 kHz or below, to reduce distortion, and the CMOS TLC555 above that. - **the pulse**: the shortest monostable pulse is about 10 µs. - Trig must be back high at least 10 µs before the pulse ends, or the output stays high. - Retriggering cannot happen during the timing cycle. - **the largest resistors**: the threshold pin draws up to 250 nA (30 nA typical). TI gives the resulting limit as RA + RB ≈ 3.4 MΩ at 5 V and ≈ 10 MΩ at 15 V. - **the smallest resistors**: TI's timing figures and test circuits start at RA = 1 kΩ. No explicit minimum is stated. The discharge switch is specified at 0.4 V maximum while sinking 8 mA, at 5 V. - **the pins**: RESET is tied to Vcc when it is not used. The CONT pin may be decoupled with a capacitor, 0.01 µF in TI's figure.

What it means

A **555** is a timer chip, with two comparators, a latch and a discharge switch inside. The pins in the drawing are: - **TRG**, pin 2, the trigger: a voltage below ⅓ Vcc starts a high output; - **THR**, pin 6, the threshold: a voltage above ⅔ Vcc ends it; - **DIS**, pin 7, the discharge: shorted to ground while the output is low, which empties C; - **CTL**, pin 5, the control voltage: it sets the two levels, and 10 nF to ground keeps them steady; - **pin 4**, reset, tied to Vcc so it never resets.

The modes work like this: - **Astable** — C charges through R1 + R2 up to ⅔ Vcc. The output then goes low, and pin 7 drains C through R2 down to ⅓ Vcc, when the output goes high again. - **Duty** — the share of each

period the output spends high. Since C charges through R1 + R2 but drains through R2 alone, the duty is always above 50 %. - Ask for 50 % or less, and a diode appears across R2 to take it out of the charge path. - Its forward drop is ignored, as datasheets ignore it, so the real duty is a little off. - **Monostable** — the output rests low, with C held empty. A trigger sends the output high, and C charges through R until it reaches ⅔ Vcc, which ends the pulse. - **Vcc cancels out** of every time, because both levels are fractions of it. It only sets the voltages you would probe and the resistor limits.

Why the formulas are these

```
Astable:    t high = ln 2 × (R1 + R2) × C    (with the diode: ln 2 × R1 × C)
            t low  = ln 2 × R2 × C
            Frequency = 1 / (t high + t low)
Monostable: Pulse width = ln 3 × R × C ≈ 1.1 × R × C
```

- **ln 2** — charging from ⅓ toward Vcc, C reaches ⅔ when half the remaining gap is closed, which takes RC × ln 2. Draining from ⅔ toward 0, it reaches ⅓ at half, again RC × ln 2.
- **ln 3** — the monostable starts from an empty C and must reach ⅔ of the way, leaving a third: RC × ln 3 = 1.0986 RC, TI's 1.1 RC.
- **The timing diagram** uses the same curves, so they need no RC. Each stretch is drawn as 1 − ⅔ × 2^−s rising, ⅔ × 2^−s falling, and 1 − 3^−s for the monostable, where s runs from 0 to 1 across it.
- **Typing a frequency** scales both resistors. **Typing a duty** redistributes them. C is never rewritten, since it is the part you have in the drawer.

Assumptions and limits

The tool cautions, without refusing, when: - **R1 (or R) is under 1 kΩ**, where TI's figures start. Pin 7 then sinks Vcc / R1 during every low time, or the whole time a monostable waits; the caution shows that current. - **The charging resistance is too large.** That is R1 + R2, R1 alone with the diode, or the monostable's R, against TI's limit: 3.4 MΩ at 5 V and 10 MΩ at 15 V, joined by a straight line. - The line keeps the charging current at the threshold around 500 nA, twice the pin's 250 nA maximum. - Applying it to the monostable's R is this tool's extension: TI states it for the astable, but it is the same pin sensing the same capacitor. - **The frequency is over 100 kHz**, or **the pulse is under 10 µs**. - **Vcc is outside 4.5–16 V.**

Beyond those: - **The levels are typical.** The real trigger and threshold levels spread, as above, so the real times do too. - **Capacitor tolerance**, often ±20 % for electrolytics, usually matters more than the resistors'. - **The bipolar NE555 is modelled.** CMOS versions such as the TLC555 use the same formulas, but have their own limits.

What it deliberately does not do

- Two-transistor multivibrators have their own tool.
- Switch debouncing has its own tool, Debounce / RC timing.

How it was checked

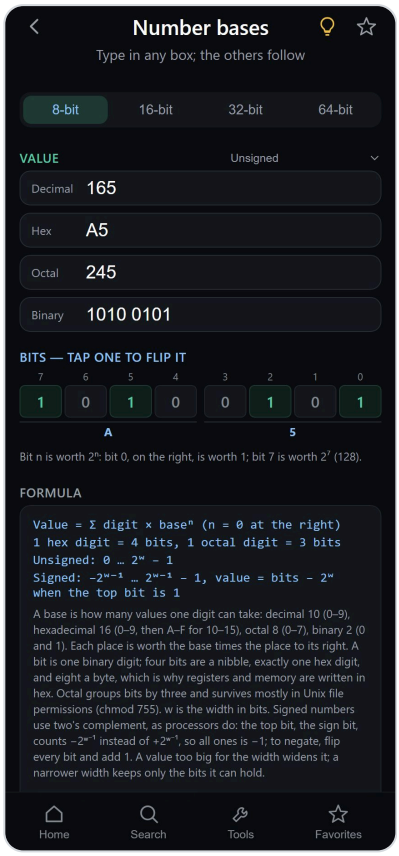
- **4.7 k, 4.7 k, 100 nF:** period 977.3 µs (1.023 kHz), duty 66.67 %, levels 1.667 V and 3.333 V at 5 V.
- **10 k, 680 k, 1 µF:** 949.6 ms (1.053 Hz), high 478.3 ms, duty 50.36 %.
- **1 kHz at 50 % with 10 nF:** the diode switches in, and R1 = R2 = 72.135 kΩ.
- **Monostable, 91 k and 100 µF:** 9.997 s.
- **Monostable, 1 s with 100 nF:** R = 9.1024 MΩ, cautioned at 5 V (over 3.4 MΩ), not at 15 V (under 10 MΩ).
- **R1 = 470 Ω at 5 V:** the caution shows 10.6 mA into pin 7.

The review made these changes: - It put the period, or the pulse width, first and large. - It added the capacitor's voltage to the timing diagram. - It replaced the fixed 10 MΩ caution with TI's supply-dependent limit. The old wording claimed a 1 kΩ minimum "set by datasheets", which TI does not state, and the 1 kΩ caution now says what it rests on. - It added TI's 100 kHz, 10 µs and 4.5–16 V cautions. - It limited the units to real ones (Ω–MΩ, pF–µF, Hz–kHz, µs–s, Vcc in V). - It enlarged the 9–10 px labels to 11 px, and wrote the 10 nF out. - It added plain definitions of every pin, and examples. - It remembers the mode and the tolerance. - The tool was renamed 555 timer, on Pierre's choice.

DIGITAL & LOGIC CIRCUITS › NUMBER SYSTEMS

Number bases

(formerly DEC / HEX / OCT / BIN conversion)



What it computes

One value, shown in decimal, hexadecimal, octal and binary at once. - **Four live boxes.** Type in any of them, and the other three follow. - **A register width:** 8, 16, 32 or 64 bits. - **Unsigned or signed.** Signed numbers are read as two's complement, the way processors store negative numbers: FFF6 in 16 bits is −10. - **One button per bit.** Each is numbered from bit 0 on the right, and tapping one flips it. The hex digit of each group of four bits sits under it.

The width and the signed choice are remembered.

Source

The arithmetic is standard: positional notation and two's complement. No datasheet is involved. The four widths are those of the usual fixed-size integer types (uint8 to uint64, int8 to int64).

What it means

- **A base** is how many values one digit can take:
- decimal, 10 (0–9);
- hexadecimal, 16 (0–9, then A–F for 10–15);
- octal, 8 (0–7);
- binary, 2 (0 and 1).

Each place is worth the base times the place to its right. - **A bit** is one binary digit. Four bits are a **nibble**, which is exactly one hex digit, and eight bits are a **byte**: two hex digits. That is why registers and memory are written in hex. - **Octal** groups bits by three. It survives mostly in Unix file permissions (chmod 755), rarely in electronics. - **Bit 0**, on the right, is the least significant bit (LSB), worth 1. The top bit is the most significant (MSB). - **Two's complement** — in a signed number the top bit, the **sign bit**, counts -2^{w-1} instead of $+2^{w-1}$, where w is the width. - All ones is therefore −1. - To negate a number, flip every bit and add 1. - Adding and subtracting work the same for signed and unsigned values, which is why processors use it.

Why the formulas are these

$$\text{Value} = \Sigma\ \text{digit} \times \text{base}^{n} \qquad (n = 0 \text{ at the right})$$

$$\text{Unsigned: } 0 \ldots 2^{w} - 1$$

$$\text{Signed: } -2^{w-1} \ldots 2^{w-1} - 1, \quad \text{value} = \text{bits} - 2^{w} \text{ when the top bit is 1}$$

- **One hex digit is four bits** because 16 = 2^4, and one octal digit is three bits because 8 = 2^3. So hex and octal are read straight off the binary, four or three bits at a time.
- **A negative decimal** is stored as 2^w plus the value: −10 in 16 bits is 65 536 − 10 = 65 526 = FFF6.

Assumptions and limits

- **Whole numbers only, up to 64 bits.** Every value is an exact big integer, not a JavaScript number, so all 64 bits are exact. Typing more than 64 bits is refused, and the box goes back to the last value.
- **Widening.** A value too big for the chosen width moves up to the next width that holds it (8 → 16 → 32 → 64). Choosing a wider width keeps the value, and a negative one is sign-extended (FFF6 → FFFFFFF6).
- **Narrowing.** Choosing a narrower width keeps only the bits it can hold, as the register would, and the tool says so.
- **No padding.** The boxes show no leading zeros, so 165 in 16 bits reads A5, not 00A5. The bit row shows every bit of the width.

What it deliberately does not do

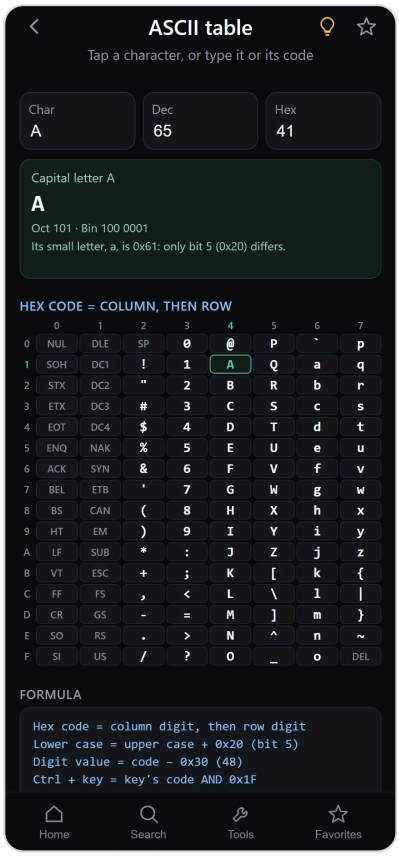
- Characters and their codes have their own tool, the ASCII table.
- Fractions, floating point (IEEE 754), BCD and Gray code are not converted.

How it was checked

- **165:** A5, 245, 1010 0101.
- **FFFFFFFFFFFFFFF:** 18446744073709551615 and 17777777777777777777.
- Before the review this came back as 10000000000000000 in hex, and as 18446744073709552000 in decimal.
- JavaScript numbers hold integers exactly only up to 2^{53}.
- **9007199254740993** (2^{53} + 1): 20000000000001.
- **300** typed at 8 bits widens to 16 bits: 12C, 0001 0010 1100.
- **Signed 8-bit FF:** −1.
- **Signed 32-bit −1:** FFFFFFFF, 37777777777.
- **Signed −10 at 8 bits, widened to 16:** FFF6, still −10.
- **1A5 narrowed to 8 bits:** A5, with a note that the upper bits were dropped.
- **Examples:**
- 0x88 = 136 (bits 7 and 3);
- FFF6 as int16 = −10;
- FFFFFFFF = 4,294,967,295. As a 32-bit millisecond counter, that is 49.7 days before it wraps.

The review made these changes: - It fixed the conversion of values above 2^{53}, which were silently wrong. Values are now exact BigInts. - It replaced the base pills, the input and the four separate results with four live boxes. - It added the register width and signed (two's complement) reading, with a ± key for typing negatives. - It added the tappable bit row, with the hex digit under each nibble and the sign bit marked. - It added plain definitions, examples and remembered settings. - The tool was renamed Number bases, on Pierre's choice.

ASCII table



What it computes

The 128 ASCII characters on one screen, as the classic chart of 8 columns by 16 rows. Tap a character, or type it in one of the three live boxes: - **Char** takes a character, or a control character's name (LF, ESC, XOFF, TAB, SP…); - **Dec** takes the decimal code, 0–127; - **Hex** takes the hex code, 00–7F.

The card shows, for the chosen character: - its name, with the standard mnemonic for a control character; - its codes in octal and in binary, the binary grouped as column and row; - the C escape (\n, \t, \0…) and the Ctrl key that sends it, where there is one; - in plain words, what a control character does, how a letter relates to its other case, or a digit to its value.

Source

RFC 20, ASCII format for Network Interchange (October 1969), was read for this. It is based on USAS X3.4-1968, the US standard ASCII, and gives: - **the mnemonics and full names** of the 33 control characters, NUL to US and DEL. Code 9 is HT, horizontal tabulation, so the tool no longer calls it TAB (it still finds it by that name); - **the definitions** behind each plain-word meaning; - **the chart:** bits b7–b5 give the column (0–7), and b4–b1 the row (0–15).

The modern uses are common knowledge, not from RFC 20: - C's escapes; - XON/XOFF serial flow control on DC1/DC3; - Ctrl+C sending ETX and Ctrl+D sending EOT in a terminal; - CR LF line endings; - ESC starting a terminal control sequence.

What it means

ASCII (American Standard Code for Information Interchange) gives 128 characters a 7-bit code. - **Codes 0–31 and 127 are control characters:** commands for a terminal, printer or modem rather than symbols. RFC 20 groups some of them: - communication controls frame and acknowledge messages: SOH, STX, ETX, EOT, ENQ, ACK, NAK, SYN, ETB; - format effectors move the print position: BS, HT, LF, VT, FF, CR; - information separators split data, from the largest unit to the smallest: FS, GS, RS, US. - **Codes 32–126 are printed characters.** 32, the space, is a printed character that prints nothing. - **Codes above 127 are not ASCII.** Different 8-bit sets give them different characters (Latin-1, Windows-1252…). In UTF-8 they only appear inside multi-byte characters.

Uses worth knowing in electronics: - **CR LF** ends a line on Windows and on many serial devices. Linux and macOS end a line with LF alone. - **XOFF (DC3) and XON (DC1)** pause and resume a serial sender: software flow control, Ctrl+S and Ctrl+Q. - **ESC [** starts a terminal control sequence, which sets colours and moves the cursor. - **SOH, EOT, ACK, NAK and CAN** run XMODEM, a simple serial file-transfer protocol.

Why the formulas are these

```
Hex code      = column digit, then row digit
Lower case    = upper case + 0x20        (bit 5)
Digit value   = code − 0x30              (48)
Ctrl + key    = key's code AND 0x1F      (bits 5 and 6 cleared)
```

- **The chart's columns are the top three bits** and its rows the low four. A column is one hex digit, 0–7, and a row is the other, 0–F. So A, column 4 and row 1, is 0x41.
- **Letters were placed so the two cases differ in one bit.** A to Z run 0x41–0x5A, and a to z run 0x61–0x7A: setting bit 5 lowers the case.
- **Digits were placed so the low four bits are the value.** 0 to 9 run 0x30–0x39, so subtracting 0x30 (or masking with 0x0F) gives the number.
- **Ctrl clears bits 5 and 6.** Ctrl+C is C (0x43) AND 0x1F, which is 3, ETX; Ctrl+[is ESC.

Assumptions and limits

- **US ASCII only.** The national variants of ISO 646 replace a few printed characters, such as # or [, and are not shown.
- **Uses, not rules.** The modern uses in the card say what a code usually does today. A particular device may assign its own meanings.

What it deliberately does not do

- Numbers in other bases have their own tool, Number bases.
- Turning text into bytes, for example "AT\r\n" into 41 54 0D 0A, was left out on Pierre's choice. It could become a tool of its own.

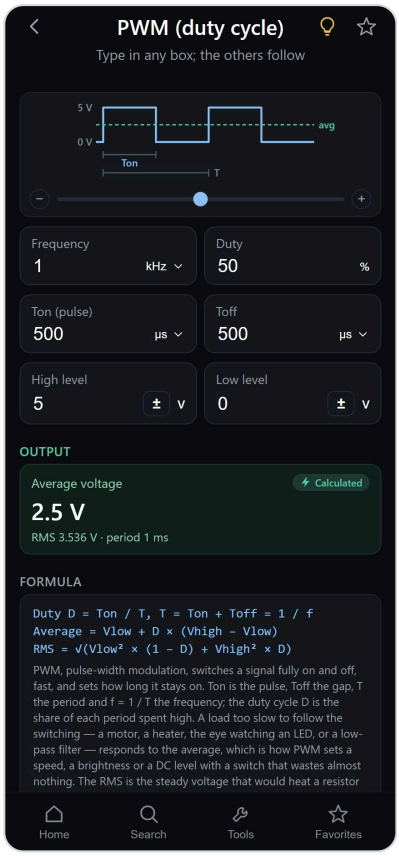
How it was checked

- **A:** 65, 0x41, octal 101, binary 100 0001; column 4, row 1.
- **Code 9:** HT, Horizontal tab (RFC 20's name).
- **LF:** 10, 0x0A, octal 012, binary 000 1010, \n, Ctrl+J.
- **Hex 7E:** 126, the tilde.
- **XOFF typed by name:** 19, DC3.
- **ESC:** octal 033, binary 001 1011, Ctrl+[.
- **127:** DEL.
- **Out of range:** 200 and é are reported as not ASCII.
- **<:** shown as a character, and named less-than sign.
- **Space:** shown as SP.
- **7:** 55; value = 55 − 48 = 7.
- **The chart's height:** constant for all 128 characters. At 375 × 812 it ends 21 px above the tab bar.

The review made these changes: - It replaced the searchable list with RFC 20's chart, and the search with three live boxes. - It named code 9 HT, after the standard, instead of TAB. - It added names and plain meanings for the 33 control codes, C escapes, Ctrl keys, and the case and digit relations. - It escaped <, >

and &, which were inserted as raw HTML. - It added examples: CR LF, XOFF/XON, a digit to its number, upper and lower case.

PWM (duty cycle)



What it computes

For a pulse-width-modulated signal, the tool links six live boxes: - **Frequency** and **Duty**, the usual way to specify a PWM signal; - **Ton** (the pulse) and **Toff** (the gap), the way it is read off a scope; - **High level** and **Low level**, in volts.

From them it computes: - **the average voltage**, shown large; - **the RMS voltage**; - **the period**.

The waveform shows two periods, with the average as a dashed line. The slider under it sets the duty. The high and low levels are remembered.

Source

The formulas follow from the definitions. Two examples were checked against documents: - **The Arduino reference for analogWrite()** (reference-en, read on GitHub), which gives: - the PWM frequency: 490 Hz on most Uno, Nano and Mini pins, and 980 Hz on pins 5 and 6; - the value range: 0 (always off) to 255 (always on). - **An ebm-papst DC fan catalogue page** (speed setting, p. 175). It states a four-wire interface "in accordance with Intel specifications for 12 VDC fans, PWM frequency 25 kHz". Intel's own 4-wire PWM fan specification could not be opened (the copies found refused access), so it was not read.

The hobby-servo figures are the common convention, not a standard: - a pulse every 20 ms (50 Hz); - 1 ms to 2 ms wide, with 1.5 ms at the centre.

What it means

PWM, pulse-width modulation, switches a signal fully on and off, fast, and sets how long it stays on. - **Ton** — the pulse, how long the signal stays high. - **Toff** — the gap, how long it stays low. - **T** — the period, Ton + Toff. **f = 1 / T** is the frequency. - **The duty cycle D** — the share of each period spent high, from 0 % (always off) to 100 % (always on). - **The average** — a load too slow to follow the switching responds to the average. That load can be a motor, a heater, the eye watching an LED, or a low-pass filter. This is how PWM sets a speed, a brightness or a DC level with a switch that wastes almost nothing: it is either fully on or fully off. - **The RMS** — the steady voltage that would heat a resistor the same. With a low level of 0 V, the power into a resistor is D times the full power. - **A negative low level** — an H-bridge switching between +V and −V. The average then goes from −V to +V as the duty goes from 0 to 100 %.

The boxes work like this: - **Typing a frequency** rescales the pulse and keeps the duty. - **Typing a duty** redistributes it and keeps the frequency. - **Ton and Toff** are independent, so two readings off a scope can be typed one after the other.

Why the formulas are these

```
D = Ton / T,   T = Ton + Toff = 1 / f
Average = Vlow + D × (Vhigh − Vlow)
RMS     = √(Vlow² × (1 − D) + Vhigh² × D)
```

- **The average** is the time-weighted mean of the two levels: Vhigh for a share D of the time and Vlow for the rest.
- **The RMS** is the square root of the time-weighted mean of the squares, by definition.

Assumptions and limits

- **Ideal edges.** The rise and fall times are taken as zero. With real edges the average shifts a little, most at very short pulses.
- **Ideal averaging.** The average is what a perfect filter, or an infinitely slow load, would see. How much ripple remains depends on the filter or the load: see the RC filter tool.
- **Frequency is a design choice the tool does not make.** It must be high enough for the load to average: no visible flicker, no audible whine (20 kHz is the usual upper limit of hearing). It must also be low enough for the switching losses to stay small.

What it deliberately does not do

- Timer register values (prescaler, top, compare) are not computed: they differ from one microcontroller family to the next.
- Filtering the PWM into a DC level is the RC filter tool's job.

How it was checked

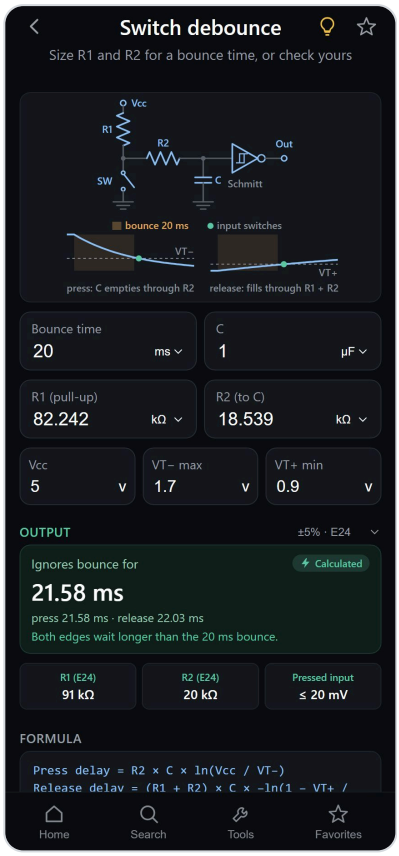
- **1 kHz, 50 %, 5 V / 0 V:** Ton 500 µs, average 2.5 V, RMS 3.536 V, period 1 ms.
- **25 % at 1 kHz:** 250 µs on, 750 µs off, average 1.25 V.
- **2 kHz** with the duty kept at 50 %: 250 µs on.
- **Ton 200 µs, then Toff 800 µs:** 1 kHz and 20 %.
- **150 % typed:** held at 100 %, Toff 0, average 5 V.
- **+12 V / −12 V at 75 %:** average 6 V, RMS 12 V.
- **Examples:**
- servo: 1.5 ms on, 18.5 ms off, 7.5 %;
- analogWrite(128): 490 Hz, 50.2 %, 2.51 V;
- PC fan at 40 %: 16 µs at 25 kHz;
- 12 V motor at 75 %: 9 V.

The review made these changes: - It replaced the two mode pills with six live boxes. - It put the average voltage first and large. - It redrew the waveform: - the pulse in the part blue, not the domain colour; - two periods, with the average dashed in green; - 11 px labels instead of 9 px; - the duty slider under it. - It limited the units to real ones (Hz–MHz, ns–s), and the levels to V with a ± key. - It added plain definitions, examples, and remembered levels.

DIGITAL & LOGIC CIRCUITS › TIMING & INTERFACES

Switch debounce

(formerly Debounce / RC timing)



What it computes

It sizes the RC debouncer from Jack Ganssle's guide: Vcc through R1 to the switch, R2 from the switch to C, and C feeding a Schmitt-trigger input. - **Sizing.** Type the bounce time, C, Vcc and the input's worst-case thresholds (VT− at its highest, VT+ at its lowest), and R1 and R2 are sized so that both the press and the release take the bounce time to change the input. - **Checking.** Type R1 and R2 to check a circuit you have.

The output shows: - **the delays of the circuit built from standard values,** rounded up to the next value of the chosen series. The shorter delay is shown large, with the press and release delays beneath; - **the two resistors** in standard values; - **the pressed input level** caused by input leakage through R2.

Two small graphs show C's voltage after a press, falling through R2, and after a release, rising through R1 + R2. Each is drawn against the bounce window, with a dot where it crosses the threshold: green after the bounce, amber inside it. Vcc, the thresholds and the series are remembered.

Source

- **Jack Ganssle, *A Guide to Debouncing*, part 2** (ganssle.com) was read for:
- the circuit;
- the method: R2 from the press, R1 + R2 from the release, each against the worst-case threshold;
- his worked example: 20 ms, 74AHCT14 at 5 V; with 0.1 µF R2 = 185 kΩ, and with 1 µF R2 = 18 kΩ, R1 + R2 = 101 kΩ, R1 = 82 kΩ;
- the leakage concern;
- his measured bounce times: most switches well under 10 ms, so he designs for 20 ms.

His equations are images on the page and could not be read as text. They were rebuilt from the RC charge law, and reproduce all three of his resistor values. - **TI SN74AHCT14 datasheet (SCLS246T, October 2023)** was read for the thresholds: - VT+ (rising) is 0.9–1.9 V at 4.5 V, and 1.0–2.1 V at 5.5 V; - VT− (falling) is 0.5–1.5 V at 4.5 V, and 0.6–1.7 V at 5.5 V; - Ganssle's 0.9 V and 1.7 V are the extremes; - the input current is ±1 µA. - **Nexperia 74HC_HCT14 datasheet (rev. 10, 29 February 2024)**, table 10, was read for the 74HC14 thresholds: - at 4.5 V, VT+ is 1.7–3.15 V and VT− 0.9–2.0 V; - the input leakage is ±1.0 µA over temperature.

What it means

- **Contact bounce** — a switch's contacts bounce for a few milliseconds when it is pressed or released. A fast input would see every bounce as another press.
- **The RC delay** — C slows the input down. Pressing empties C through R2; releasing fills it through R1 + R2. The input changes only once C crosses its threshold, after the bouncing is over.
- **R1** — the pull-up. It also limits the current through the switch while it is held.
- **R2** — sets the press delay, and limits the current the contacts take from C.
- Without R2 (C straight across the switch), a press registers on the very first contact, and the contacts take C's full discharge.
- The bounce after that first contact is still filtered: an opening is too short for C to recharge past VT+ through R1.
- **A Schmitt-trigger input** — it has two thresholds, VT− for a falling input and VT+ for a rising one, with a gap between them (hysteresis). So a slow, noisy edge switches it only once.
- Ganssle warns never to feed C straight to a microcontroller input, as few have any hysteresis. An input without it does unpredictable things on a slow edge.
- Some microcontrollers do have Schmitt-trigger inputs: check the datasheet, and use its thresholds.
- **Worst case** — thresholds vary between parts and with the supply. The highest VT− and the lowest VT+ give the shortest delays, so those are the values to design with.
- **Pressed input** — the input's leakage (up to 1 µA) flows through R2 while the switch is held, lifting the input up to 1 µA × R2 above ground.
- Ganssle rejected 180 kΩ (0.18 V, against a lowest threshold of 0.5 V) and moved to 1 µF to bring R2 down to 18 kΩ.
- The tool cautions from 0.1 V.

Why the formulas are these

```
Press delay   = R2 × C × ln(Vcc / VT−)
Release delay = (R1 + R2) × C × −ln(1 − VT+ / Vcc)
Sizing:  R2 = t / (C × ln(Vcc / VT−)),   R1 + R2 = t / (C × −ln(1 − VT+ / Vcc))
```

- **Press:** C falls from Vcc toward 0 through R2 alone, following V = Vcc × e^(−t/R2C). It reaches VT− after R2C × ln(Vcc / VT−).
- **Release:** C rises from 0 toward Vcc through R1 + R2, following V = Vcc × (1 − e^(−t/(R1+R2)C)). It reaches VT+ after (R1 + R2)C × −ln(1 − VT+ / Vcc).
- **Bounce makes the real delays longer.** Each opening during a press lets C climb back a little, so these delays are the shortest the circuit can give, which is what the design needs.
- **Typing the bounce time, C or the thresholds** sizes R1 and R2. **Typing R1 or R2** checks them, as typed.
- **The standard values are rounded up,** so the built circuit never waits less than the bounce time. Ganssle rounded to the nearest values (18 k, 82 k): that gives 19.4 ms and 19.8 ms, a little under his 20 ms, which already doubles the bounce he measured.

Assumptions and limits

- **Full charge and discharge.** Each delay starts from C fully charged or fully empty. A press released before C has emptied gives a shorter release delay.
- **Leakage only through R2 is shown.** The released input also sits up to 1 µA × (R1 + R2) below Vcc.
- **Bounce time is the switch's.** Measure it, or use Ganssle's 20 ms. Some switches bounce far longer.

What it deliberately does not do

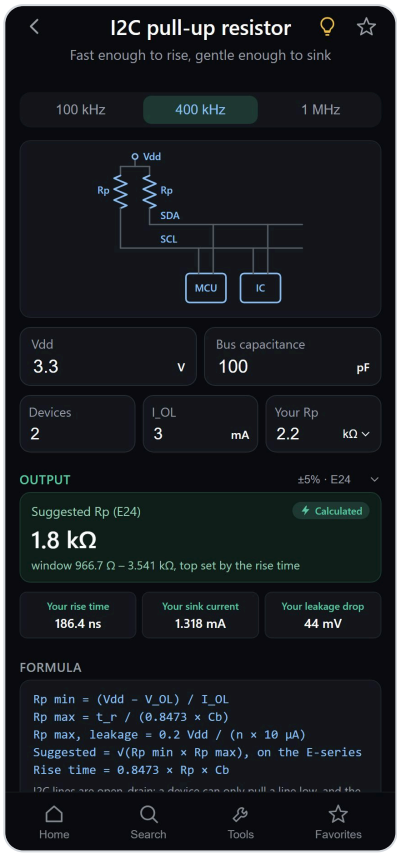
- Firmware debouncing, which Ganssle's guide also covers, is not modelled.
- The RC charge itself, with any threshold, has its own tool: RC charge/discharge.

How it was checked

- **20 ms, 1 µF, 74AHCT14 at 5 V (1.7 V, 0.9 V):**
- exact R2 18.539 kΩ and R1 82.242 kΩ, so R1 + R2 = 100.78 kΩ (Ganssle's 101 kΩ);
- E24, rounded up: 20 kΩ and 91 kΩ;
- delays: press 21.58 ms, release 22.03 ms;
- pressed input ≤ 20 mV.
- **0.1 µF:** R2 = 185.39 kΩ (Ganssle's 185 kΩ). With 200 kΩ (E24) the pressed input sits at 200 mV, and the tool cautions.
- **50 ms:** R2 46.347 kΩ, R1 205.6 kΩ.
- **10 k and 10 k typed with 1 µF:** press 10.79 ms, release 3.969 ms, shorter than 20 ms.
- **74HC14 at 4.5 V (2.0 V, 1.7 V), 1 µF, 20 ms:** 27 kΩ and 18 kΩ (E24), shorter delay 21.35 ms.
- **R2 = 0:** the press delay is 0 s, and the tool says why.

The review made these changes: - It replaced the single-resistor circuit, C straight across the switch, with Ganssle's two-resistor circuit and his worst-case method. - It made Vcc count. It was an input the result never used, because the threshold was a percentage. - It replaced the guessed 50 % threshold with VT− and VT+ in volts, from the datasheet. - It replaced the unsourced bounce-time table with Ganssle's measurements. - It added the two graphs and the leakage check. - It limited the units to real ones (Ω–MΩ, pF–µF, µs–s, volts). - It put the result first and large, added examples, and remembers the input's thresholds. - The tool was renamed Switch debounce, on Pierre's choice.

I2C pull-up resistor



What it computes

For an I²C bus, given the speed mode, the supply, the bus capacitance and the number of devices, it computes: - **the window** of pull-up resistances that work; - **a suggested value** inside it, snapped to a standard E-series and shown large. The result says which limit sets the top of the window: the rise time, or the devices' leakage; - **for the Rp you have:** its rise time, the current it asks a device to sink, and how far leakage pulls the high level down through it.

The mode, the series and Vdd are remembered.

Source

NXP UM10204, I²C-bus specification and user manual, rev. 7.0 (1 October 2021), was read for every limit: - **Section 7.1, pull-up resistor sizing:** - the rise time is measured between 0.3 and 0.7 Vdd, which gives T = 0.8473 × RC; - Rp(max) = t_r / (0.8473 × Cb); - Rp(min) is set by the minimum sink current: 3 mA for Standard and Fast mode, 20 mA for Fast-mode Plus; - portable, low-power designs "can use a value toward the higher end of the range". - **Table 10, the I/O stages:** - V_OL1 is 0.4 V at 3 mA when Vdd > 2 V; - V_OL2 is 0.2 Vdd at 2 mA when Vdd ≤ 2 V, for Fast mode and Fast-mode Plus only; - I_OL at 0.4 V is 3, 3 and 20 mA; - each I/O pin is at most 10 pF. - **Table 11, the bus lines:** - the rise time t_r is at most 1000, 300 and 120 ns; - the bus capacitance Cb is at most 400, 400 and 550 pF; - the high-level noise margin is 0.2 Vdd. - **Section 7.4, input leakage:** each connection may draw up to 10 µA at the high level, and "due to the required noise margin of 0.2 VDD for the HIGH level, this input current limits the maximum value of Rp."

What it means

- **Open-drain.** I²C lines are open-drain: a device can only pull a line low. Nothing drives it high; the pull-up resistor Rp brings it back. That single fact produces the bounds, pulling in opposite directions.
- **I_OL** — the current a device is rated to sink while holding the line at the output-low level V_OL. Too small an Rp asks for more.
- **The rise time t_r** — the time the line takes to climb from 0.3 to 0.7 Vdd. Too large an Rp makes it too slow for the mode.
- **The bus capacitance Cb** — every pin on the bus, up to 10 pF each, plus the wiring. It is the input people are most wrong about, and the whole upper bound scales with it.
- **Leakage** — every device may leak up to 10 µA into the line. Through Rp, that pulls the high level down, and it must not cost more than the 0.2 Vdd noise margin. With many devices this, and not the rise time, sets the top of the window.

Why the formulas are these

```
Rp min          = (Vdd − V_OL) / I_OL        V_OL = 0.4 V, or 0.2 Vdd at 2 V or les
Rp max          = t_r / (0.8473 × Cb)
Rp max, leakage = 0.2 Vdd / (n × 10 µA)
Suggested       = √(Rp min × Rp max), on the E-series
```

- **The lower bound is the pull-down.** When a device asserts the line, its transistor has to hold it below V_OL while the pull-up pushes (Vdd − V_OL) / Rp into it.
- **The upper bound is the rise.** Going up, the line is just an RC: the pull-up charging the bus capacitance. An RC charge reaches 0.3 Vdd at RC × ln(1/0.7), and 0.7 Vdd at RC × ln(1/0.3). The difference is RC × ln(7/3) = 0.8473 RC, the factor the spec prints.
- **The leakage bound** is Ohm's law. n devices × 10 µA through Rp must drop no more than 0.2 Vdd.
- **The suggestion** is the geometric mean of the two bounds: the centre of the window on a logarithmic scale, which is what "equal margin on both sides" means when the ends can differ by a decade. The spec's own advice for low-power designs is the high end instead.
- **Finding Cb.** Measure the rise time with a known Rp, and the same formula gives Cb = t_r / (0.8473 × Rp).
- **When the window closes.** Rp max falls as the bus capacitance rises, while Rp min does not move. At 3.3 V in Fast mode the two cross at about 366 pF. Past that no resistor rises in time without overloading the pull-down. That is a dead end, not a poor choice. The ways out are:
- a shorter bus, or fewer devices;
- a slower mode;
- a bus buffer that splits the bus into segments (UM10204 section 7.2).

Assumptions and limits

- **Bus capacitance is an estimate.** UM10204 gives no capacitance per centimetre of wiring, so the tool does not guess one. Measure it on a long bus, as above.
- **One lumped capacitance and an ideal resistor.** At Fast-mode Plus, with long tracks, the line no longer behaves as one RC.
- **Standard mode at 2 V or less** has no output-low level in UM10204. The tool keeps 0.4 V at 3 mA and says so.
- **I_OL follows the mode and the supply,** unless you type a value: a particular part may be rated differently.

What it deliberately does not do

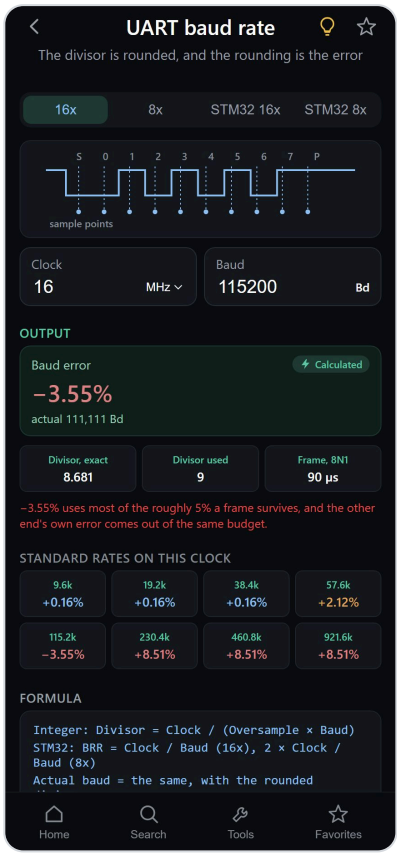
- **Current-source or switched pull-ups** (UM10204 section 7.2.4): a different circuit, with different arithmetic.
- **Bus buffers and repeaters:** they split a bus into segments, each with its own capacitance and resistor. Run the tool once per segment.
- **Level shifters:** a MOSFET level shifter between two voltage domains puts a pull-up on each side, and the two interact.

How it was checked

- **400 kHz, 3.3 V, 100 pF, 2 devices:**
- window 966.7 Ω – 3.541 kΩ, 1.8 kΩ suggested;
- with 2.2 kΩ: rise time 186.4 ns, sink current 1.318 mA, leakage drop 44 mV.
- **100 kHz, same bus:** up to 11.8 kΩ, 3.3 kΩ suggested.
- **100 kHz with 10 devices:** leakage caps Rp at 6.6 kΩ, and 2.4 kΩ is suggested.
- **1.8 V at 400 kHz:** I_OL drops to 2 mA, Rp min is 720 Ω, and 1.6 kΩ is suggested.
- **1 MHz, 3.3 V, 200 pF:** 145–708.1 Ω, 330 Ω suggested.
- **370 pF at 400 kHz and 3.3 V:** no value works.

The review made these changes: - It added the output-low level below 2 V, 0.2 Vdd at 2 mA. The tool used 0.4 V at 3 mA at any supply, which put Rp min too low on a 1.8 V bus (467 Ω instead of 720 Ω). - It added the input-leakage limit (UM10204 section 7.4), with a Devices field. - It replaced the unsourced "a few pF per centimetre of track" with the spec's 10 pF per pin and the measure-the-rise-time method. - It put the suggestion first and large, and named which limit sets the window. - It added examples, remembered the mode, series and Vdd, enlarged the 10 px labels to 11 px, and added plain definitions.

UART baud rate



What it computes

From a peripheral clock and a wanted baud rate, it computes: - **the divisor** the UART needs, and the divisor actually written once it is rounded; - **the rate that divisor gives,** and the resulting **error**, shown large; - **the frame time** for 8N1.

There are two kinds of divider, each at 16x or 8x oversampling: - **16x / 8x:** the integer divider of the AVR, the 16550 family and PIC. - **STM32 16x / 8x:** the finer divider of STM32's USART.

A table gives the error of every standard rate on the same clock; tap one to try it. The drawing shows the sample points walking off the bits as the error grows. The mode and the clock are remembered.

Source

There is no single standard for baud generation: it is per-device. Three sources were read: - **ST's STM32G0 HAL driver** (stm32g0xx_hal_uart.h and .c, on GitHub): - UART_DIV_SAMPLING16 is round(PCLK / baud); - UART_DIV_SAMPLING8 is round(2 × PCLK / baud); - the authorized BRR is UART_BRR_MIN 0x10 (16) to UART_BRR_MAX 0xFFFF. - **The Arduino AVR core** (HardwareSerial.cpp, ArduinoCore-avr on GitHub): - begin() tries U2X, the 8x mode, first; - it falls back to 16x only for 57600 at 16 MHz (for bootloader compatibility), or when the divisor would exceed 4095. - **The error budget** is derived below from the sampling behaviour, not quoted.

What it means

- **A UART** sends bytes one bit at a time, with no clock wire. Both ends agree on a rate, the baud, in bits per second.
- **Its own oscillator** — it has none. It divides whatever clock it is given, and the divisor is a whole number, so the rate comes out a little off.
- **Oversampling** (16x or 8x) — how many clock periods of the divided clock the receiver counts per bit, to find the middle of each bit.
- **8N1** — one start bit, eight data bits, no parity, one stop bit: ten bits per frame.
- **Register names:**
- an AVR writes UBRR = divisor − 1;
- an STM32 writes BRR, which at 16x is the divisor shown. At 8x the HAL computes USARTDIV, shown here, and repacks it into BRR.

Why the formulas are these

```
Integer:  Divisor = Clock / (Oversample × Baud),  Actual = Clock / (Oversample × rc
STM32:    BRR = round(Clock / Baud) at 16x,  round(2 × Clock / Baud) at 8x
          Actual = Clock / BRR,  or 2 × Clock / USARTDIV
Error  = (Actual − Baud) / Baud
```

Integer dividers. At 16 MHz, 115200 baud and 16x, the divisor comes to 8.681. Written as 9, the line runs at 111,111 baud, 3.55 % slow. - That single fact is why 14.7456 MHz crystals exist: 14,745,600 / 16 = 921,600, which divides exactly into every standard rate. 11.0592 MHz and 18.432 MHz are the same trick at other speeds. - **An Arduino Uno does better than that figure.** Its core uses 8x mode first: 16 MHz / (8 × 17) = 117,647 baud, 2.12 % fast.

STM32. STM32's USART divides in whole clock cycles per bit, sixteen times finer than an integer 16x divider. The same 16 MHz at 115200 writes BRR = 139: 115,108 baud, −0.08 %.

How much error a link survives. There is no clock shared between the two ends. The receiver finds the falling edge of the start bit, starts its own counter, and samples each bit at what it believes is the middle. Taking the receiver as the reference, it samples bit *n* at

```
(n + 0.5) × T_rx
```

while the transmitter's bit *n* occupies

```
[ n × T_tx , (n+1) × T_tx ]
```

With r = T_tx / T_rx, the sample lands in the right bit only while

```
n × r  <  n + 0.5  <  (n+1) × r
```

Nothing resynchronises inside a frame, so the worst case is the last bit. For 8N1 that is the stop bit, n = 9:

```
9 r < 9.5      →  r < 1.0556
9.5 < 10 r     →  r > 0.95
```

So the two clocks may differ by about −5.0 % to +5.56 %, usually quoted as ±5 %. - It is slightly asymmetric because the sample point is fixed at mid-bit, while the bit it must land in is what stretches. - That budget covers **both** ends together. Hence 2 % each as the practical rule: it leaves room for the other end, edge jitter and finite rise time. - **The tool's colours:** blue up to 2 %, amber to 3 %, red beyond. The caution says "fails outright" only past 5 %.

The error does **not** accumulate across a message. The receiver re-finds the start edge on every frame, so the drift resets ten bits at a time.

The drawing shows exactly this. The waveform is drawn at the transmitter's real bit width, stretched or squeezed by the error, while the sample marks stay evenly spaced where the receiver puts them. Raise the error, and the marks that fall outside their own bit turn red.

Assumptions and limits

- **8N1 is assumed.** Parity or a second stop bit makes the frame longer and the budget tighter.
- **One end's error.** The receiver is taken as exact. In reality both ends have error and the two add, which is why the caution starts at 2 % rather than 5 %.
- **STM32 as ST's HAL writes it,** read from the G0 driver only; the other STM32 families' drivers were not read. Other vendors' fractional dividers (SAM, nRF and others) have their own resolution, and are not modelled.
- **Oversampling is a divider stage, not noise immunity.** 8x doubles the top speed from a given clock, and halves the margin around each sample.

What it deliberately does not do

- **Auto-baud detection**, where the receiver measures a known character.
- **Anything above the physical layer:** framing errors, break detection, flow control.

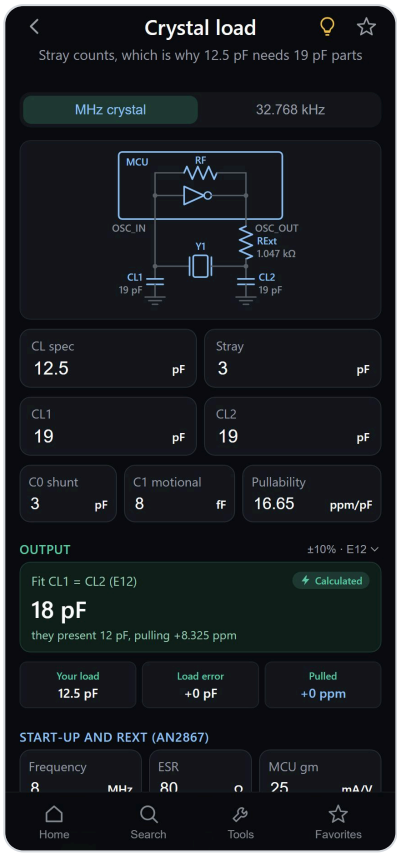
How it was checked

- **16 MHz, 115200, integer 16x:** divisor 8.681, 9 used, 111,111 Bd, −3.55 %.
- **16 MHz at 9600:** 104, +0.16 %.
- **Arduino Uno (8x), 16 MHz at 115200:** 17, 117,647 Bd, +2.12 %.
- **STM32 16x, 16 MHz at 115200:** BRR 139, 115,108 Bd, −0.08 %. At 8x, USARTDIV 278 gives the same.
- **14.7456 MHz at 115200:** exactly 8, 0.00 %.
- **8 MHz at 115200, integer 16x:** 4 used, +8.51 %, fails.
- **STM32 at 1 MHz and 115200:** BRR would be 9, under the HAL's minimum of 16, so the tool refuses it.

The review made these changes: - It added STM32's divider from ST's HAL. The tool modelled only integer dividers, which overstated an STM32's error about fortyfold at 16 MHz and 115200. - It put the error first and large. - It reworded the caution: a single end at 3.55 % does not fail, it uses most of the shared budget; only past 5 % does a frame fail. - It added the Arduino detail (8x first), checked against the core's source, and examples. - It remembers the mode and the clock, and enlarged the 9 px labels to 11 px.

Crystal load

(formerly Crystal load capacitance)



What it computes

For a Pierce oscillator, it computes: - **the two capacitors it needs,** so the crystal sees the load it was cut for. The pair to buy, in the chosen E-series, is shown large, with the load it really presents and how far that pulls the frequency; - **for the pair in the fields:** its load, its error and its pull; - **start-up, after AN2867:** - from the frequency, the crystal's ESR and the MCU's gm, the gain margin, with and without REXt; - REXt's starting value; - from the swing you measure on OSC_IN, the crystal's drive level against its rating, and so whether REXt is needed at all.

The drawing is the oscillator as it is really built: - the MCU, with its inverting amplifier and feedback resistor RF inside; - the OSC_IN and OSC_OUT pins (OSC32_IN and OSC32_OUT for a 32.768 kHz crystal); - the crystal Y1, REXt, and CL1 and CL2, with their values.

The crystal type and the series are remembered.

Source

- **ST AN2867, Oscillator design guide, rev. 10 (DocID15287),** was read in a copy hosted by Raltron (st.com reset the connection):
- **figure 5, "Pierce-oscillator circuitry":** Inv, the internal inverter working as the amplifier; RF, the internal feedback resistor; Q, the crystal; REXt, an external resistor "to limit the inverter output current"; CL1 and CL2, the load capacitors; Cs, the stray, which is the pin capacitance plus the PCB;
- **section 3.3:** CL = CL1 × CL2 / (CL1 + CL2) + Cs, and the worked example: a 15 pF crystal, assuming Cs = 5 pF, needs CL1 = CL2 = 20 pF;
- **section 3.7:** pullability from Cm, C0 and CL;
- **section 3.2, typical RF:** 10–25 MΩ at 32.768 kHz, down to 0.47–5 MΩ at 20 MHz;
- **section 3.4, the gain margin:**
  - gmcrit = 4 × ESR × $(2\pi F)^2$ × $(C0 + CL)^2$, and gm / gmcrit should be at least 5;
  - the worked example: an 8 MHz Fox crystal (C0 7 pF, CL 10 pF, ESR 80 Ω) on an STM32F1 (gm 25 mA/V) gives gmcrit 0.23 mA/V and a margin of 107;
  - its LSE table gives gm from 5 to 25 µA/V across the drive modes of the first family listed;
- **section 3.5.3, REXt:**
  - it limits the drive level, and with CL2 keeps the oscillator off overtones;
  - it is mandatory only if the crystal would dissipate more than its rated drive level, and otherwise "not recommended", 0 Ω;
  - its initial value is the reactance of CL2, 1 / (2πF × CL2): 1326 Ω at 8 MHz with 15 pF;
  - it is then trimmed with a potentiometer, and added to ESR when the margin is rechecked;
- **section 3.5.2, the drive level from a measurement:**

    - the crystal current is the current through CL1, so the swing Vpp measured on CL1 gives DL = ESR × (π F Ctot)² × Vpp² / 2, with Ctot = CL1 + Cs/2 + Cprobe;
    - the probe must add no more than 1 pF;
    - section 3.5.1 gives the other way, a current probe, with I_pp,max = 2√2 × √(DLmax / ESR).
- **The drive-level ratings are Epson's:** 0.5 µW max for the FC-135, and 200 µW max (10 µW recommended) for the FA-238, an MHz crystal.
- **Epson FC-13F / FC-135 specification sheet,** read for the 32.768 kHz values:
- C1 3.4 fF and C0 1.0 pF typical;
- CL 7, 9 or 12.5 pF;
- ±20 ppm at 25 °C.
- **The pulling expression** comes from the crystal's equivalent circuit, the Butterworth–Van Dyke model: a motional arm of L1, C1 and R1 in series, all in parallel with the shunt capacitance C0 of the electrodes and holder.

Why the formulas are these

The two capacitors are in series, not in parallel. This is the step people get backwards. Each one runs from one crystal terminal to ground, so the loop the crystal drives is: out through CL1, along the ground, back through CL2. Two capacitors in series:

```
CL1 × CL2 / (CL1 + CL2)
```

which for an equal pair is just half of one of them. Add the stray capacitance of the two tracks and the two oscillator pins — that sits directly across the same nodes, so it adds on:

```
Load = CL1 × CL2 / (CL1 + CL2) + Stray
```

Turn it round for a symmetric pair and the capacitors come out at

```
CL1 = CL2 = 2 × (CL spec − Stray)
```

A crystal marked 12.5 pF on a board with 3 pF of stray therefore wants **19 pF** parts, not 25 pF and not 12.5 pF. Both of those are common mistakes, and the second is the worse one.

Why the load changes the frequency at all. A crystal has two resonances. At series resonance the motional arm is purely resistive; a little above it the arm looks inductive and resonates with everything capacitive across it — C0 and whatever load the circuit adds. The parallel-resonant frequency is

```
fp = fs × [ 1 + C1 / (2 × (C0 + CL)) ]
```

A crystal is cut and trimmed so that this lands on the marked frequency **for one particular CL**. Present a different load and it lands somewhere else, which is the whole reason the number is on the datasheet.

Differentiating that with respect to CL gives the sensitivity, which is what the tool reports as pullability:

```
d(Δf/f) / dCL = − C1 / (2 × (C0 + CL)²)
```

The sign is negative: **more load, lower frequency**. For a typical MHz part with C1 = 8 fF and C0 = 3 pF at CL = 12.5 pF that comes to about 16.6 ppm/pF, so fitting 22 pF where 19 pF was wanted — 1.5 pF too much load — pulls it roughly 25 ppm slow. Epson's FC-135 watch crystal (C1 3.4 fF, C0 1.0 pF) pulls 9.3 ppm/pF at 12.5 pF. In its 7 pF version it pulls 26.6 ppm/pF: the lower the load, the steeper the slope, so a low-load crystal wants the tightest stray.

Where the inputs come from, and what they are called

Three of the inputs are easy to look for in the wrong place.

- **C0** — on the crystal datasheet, as `C0`, "shunt capacitance" or "static capacitance". Often given only as a maximum; that is usable.
- **C1** — the motional capacitance, `C1` on datasheets that give it, after the Butterworth–Van Dyke model. **Many do not.** When it is missing, look for a pullability figure in ppm/pF, or a "frequency versus load capacitance" curve, and type that into Pullability instead: the tool works C1 back from it. C1 is the one that stays fixed when the load changes — it is the crystal — while pullability is its slope at whatever load is chosen, so changing CL spec recomputes pullability, not C1.
- **Stray** — **never** on the crystal datasheet, because it is not the crystal. It is the capacitance of the two oscillator pins, sometimes listed in the MCU datasheet, plus the two tracks. ST's AN2867 assumes 5 pF in its worked example; take that as an order of magnitude, not a value to copy.

The load capacitors are called **CL1 and CL2** rather than C1 and C2, which is how MCU application notes label them and which keeps C1 free for its datasheet meaning. Every capacitance here only ever comes in one unit — pF for load, stray and shunt, fF for motional — so the fields show the unit fixed rather than offering a picker.

When CL says "Series". Some datasheets offer, or list, a series-resonant cut: the crystal is trimmed to its marked frequency with no load at all. A Pierce oscillator cannot present no load, so the part runs above its marking by the whole parallel-resonance offset,

```
Δf/f = C1 / (2 × (C0 + Load))
```

which is not a trim but a few hundred ppm: 8 fF against 3 pF + 12.5 pF is +258 ppm. Type 0 in CL spec to see it. The fix is to order the part with a load capacitance, or to use it in a circuit that runs it at series resonance.

Assumptions and limits

- **Stray is an estimate, and it dominates the mistakes.** It is a few picofarads for the pins plus short tracks (AN2867's example takes 5 pF), but a long or guarded layout can be well outside that. If the frequency matters, measure.
- **The presets are starting points.**
- The 32.768 kHz preset is the Epson FC-135's typical values.
- The MHz preset is illustrative: no manufacturer table was found to quote for C0 and C1.
- Use your crystal's datasheet: a wrong C1 scales the pullability in proportion.

- The pulling figure is a **slope taken at the specified load**, so it is exact for small errors and increasingly optimistic for large ones, since the true curve flattens as CL grows.
- Sizing assumes a **symmetric pair**. Asymmetric capacitors are legitimate and the tool will report the load they present, but it will not propose them.

What it deliberately does not do

- **RF is drawn, not sized.** It is inside the MCU on the parts AN2867 covers. Its section 3.2 gives typical values, from 10–25 MΩ at 32.768 kHz to 0.47–5 MΩ at 20 MHz.
- **The drive level comes from your measurement,** not from a model. Type the peak-to-peak swing you read on OSC_IN.
- The Vpp defaults (1 V, 0.5 V) only illustrate a reading.
- A probe heavier than 1 pF both loads the oscillator and inflates the figure: a 10 pF probe reads twice the drive of a 1 pF one at 8 MHz here.
- **Temperature and ageing.** The load error here is a fixed offset; drift over temperature is the crystal's cut and belongs with the ppm tooling.

How it was checked

Recomputed by hand: 12.5 pF specified with 3 pF stray gives 19 pF capacitors and a presented load of exactly 12.5 pF, 0 ppm. Fitting 22 pF instead presents 14 pF, 1.5 pF over, and pulls −24.97 ppm against 16.65 × 1.5 = 24.98 by hand. Fitting 33 pF presents 19.5 pF and pulls −116.5 ppm against 16.65 × 7 = 116.6. The watch preset (FC-135) returns 9.328 ppm/pF, against 3.4 fF / (2 × (1.0 + 12.5 pF)^2) = 9.33 by hand.

Further checks: - **AN2867's example** (15 pF, 5 pF stray) gives 20 pF each, the same as the note. - **The E12 pair for 12.5 pF with 3 pF stray** is 18 pF, which presents 12 pF and pulls +8.325 ppm. - **The FC-135's 7 pF version** needs 8 pF parts and pulls 26.56 ppm/pF. - **A Series crystal with 18 pF parts** runs +266.7 ppm fast. - **Start-up reproduces AN2867:** - C0 7 pF, CL 10 pF, 8 MHz, ESR 80 Ω and gm 25 mA/V give a margin of 107; - REext with CL2 = 15 pF is 1326 Ω. - **At the defaults** (8 MHz, C0 3 pF, CL 12.5 pF, 19 pF parts): - REext 1.047 kΩ; - margin 129 without REext, 9.135 with it. - **Drive level at 8 MHz** (19 pF parts, 3 pF stray, 1 pF probe, ESR 80 Ω): - 1 Vpp gives 11.68 µW, 5.84 % of the FA-238's 200 µW, so REext is 0 Ω; - 5 Vpp gives 292 µW, over the rating, so REext is needed; - a 10 pF probe reads 23.5 µW for the same 1 Vpp. - **The FC-135 at 0.5 Vpp:** 42.86 nW, against its 0.5 µW rating. - **The FC-135 at 15 µA/V:** - margin 6.934; - with REext (255.6 kΩ) it would fall to 1.491, so the tool says to fit 0 Ω; - at the lowest drive, 5 µA/V, the margin is 2.311, too low.

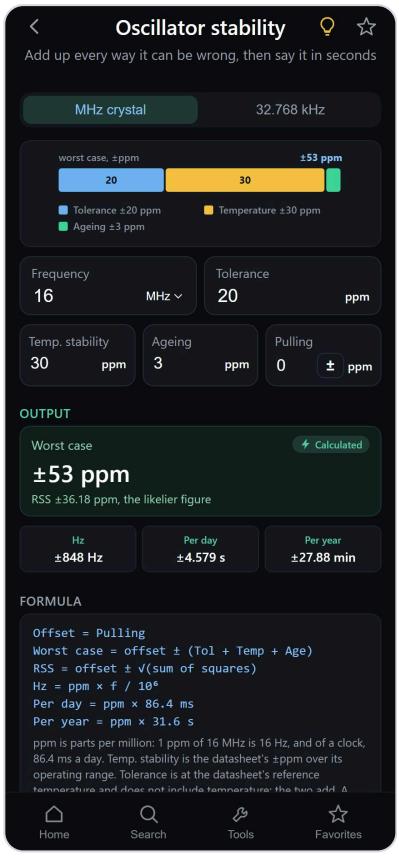
The pullability was wrong by twelve orders of magnitude on first write: C1 / (2(C0+CL)^2) is *per farad*, not dimensionless, because the numerator is first order in capacitance and the denominator is second. It needs multiplying by one picofarad to become per-pF. The hand check is what caught it.

The review made these changes: - It redrew the schematic as the real Pierce oscillator of AN2867 figure 5, on Pierre's request. - It replaced the 32.768 kHz preset (C0 1.5 pF, C1 2.5 fF, from no source) with the Epson FC-135's typical values, and corrected the claim that a watch crystal has "a tenth the motional capacitance" and pulls "around 6 ppm/pF". - It replaced the "2–5 pF" stray range, which AN2867 does not state in what was read, with its worked example's 5 pF. - It put the pair to buy first and large, with what it pulls. - It added the start-up check and REext's starting value from AN2867, after Pierre asked why REext was drawn but not calculated. - It added the drive-level check from a scope reading, so the tool says whether REext is needed at

all. - It added examples, remembers the crystal type and the series, and was renamed Crystal load, on Pierre's choice.

Oscillator stability

(formerly Oscillator stability (ppm → Hz))



What it computes

The total frequency error budget of a crystal oscillator — tolerance, temperature, ageing and load pulling combined — as a worst case and as a root-sum-square estimate, then expressed as hertz and as time gained or lost per day and per year. The worst case is shown large, the RSS beneath it. For a 32.768 kHz crystal, the parabola's coefficient B and turnover T0 are fields, taken from the crystal's datasheet. The crystal type is remembered.

For a single ppm figure applied to a single value, the **PPM converter** under Tools does that; this tool is the budget that produces the figure.

Source

The contributions and their names are the ones on crystal datasheets: frequency tolerance at a reference temperature, frequency stability over the operating temperature range, and ageing, usually per first year. The reference is 20 °C on some datasheets and 25 °C on others — one Petermann-Technik page quotes both for the same part — so the field is labelled Tolerance without a temperature.

The 32.768 kHz temperature curve is a parabola, whose two numbers come from the crystal's datasheet. **Epson's FC-135 specification sheet,** read for this review, gives: - B −0.04 ppm/°C^2 max, and a turnover of 25 ± 5 °C; - a tolerance of ±20 ppm at 25 °C; - ageing of ±3 ppm in the first year.

These are the tool's defaults. The −0.034 ppm/°C^2 often quoted as typical (±0.006) has −0.04 as its worst end.

Why the formulas are these

Two kinds of error, kept apart. Tolerance, temperature stability and ageing are *limits*: the part is somewhere within ± each of them, and which way it sits is not known. The tuning fork's temperature drift and the load pulling are *offsets*: they have a known sign and they move the whole window. Mixing the two kinds is how budgets end up either too pessimistic or wrong-signed, so the tool computes

```
Offset     = Drift + Pulling
Worst case = Offset ± (Tolerance + Temperature + Ageing)
```

and reports the extreme of that window with its sign. A window of −69 to −23 ppm is a clock that can only lose time, and writing it as ±69 ppm would throw that away.

Worst case and RSS. The worst case adds every limit at full size in the same direction. It is what datasheets call *overall stability*, and it is the right figure to design against when failure is not an option. If the contributions are independent, all of them sitting at their extremes together is unlikely, and the root-sum-square

```
RSS = Offset ± √(Tolerance² + Temperature² + Ageing²)
```

is the likelier spread. Offsets are added directly, never squared: they are not random.

The tuning-fork parabola. A 32.768 kHz crystal is a quartz tuning fork, and its frequency falls off either side of a turnover temperature:

```
Δf/f = B × (T − T0)²        (FC-135: B −0.04 ppm/°C² max, T0 25 °C)
```

It only ever goes down. - **With the FC-135's worst-case B:** −25 ppm at 0 °C (2.2 seconds a day), and −81 ppm at −20 °C (7 seconds a day). - **With the typical −0.034:** −21 ppm and −69 ppm. That is why a watch or a data logger left outdoors in winter runs slow, and why the tool asks for the temperature the fork will actually see rather than a spec. MHz crystals are AT-cut plates, whose curve is a cubic set by the cut angle; their datasheets fold it into one ±ppm figure over the range, which is what the MHz pill asks for.

Into time. A day is 86 400 s, so 1 ppm is 86.4 ms a day, and a year of 31 557 600 s makes 1 ppm about 31.6 s a year. Twenty ppm is 1.73 s a day or 10.5 minutes a year.

Assumptions and limits

- **Ageing is not linear.** It is fastest in the first months and slows roughly logarithmically, so ten years is much less than ten times the first-year figure. Enter the total you expect over the period, not a rate.
- **The parabola uses one B and one T0.** The datasheet gives B as a maximum and T0 with a tolerance (±5 °C on the FC-135). Typing the worst B, and the turnover furthest from your temperature, gives the worst case. Near the turnover it barely matters; at the extremes it does.
- The MHz temperature figure is taken as a symmetric ± limit, which is how it is specified, even though the real AT-cut curve is not symmetric.
- Pulling is carried in as a signed figure from the crystal load tool and is treated as exact.
- **Tolerance never includes temperature on a crystal datasheet.** The temperature stability is a separate line, often a separate ordering table, and the two add: ±20 ppm tolerance with ±30 ppm over −40/+85 °C is up to ±50 ppm before ageing. A packaged **XO or TCXO** is different: its "frequency stability" is often one all-inclusive figure covering tolerance, temperature, supply and load, sometimes ageing too. Read the footnote; if it is all-inclusive, enter it as Tolerance and leave the rest at zero.

What it deliberately does not do

- **Compensated oscillators.** A TCXO or an RTC with digital trimming cancels most of the temperature term; its datasheet gives a residual stability, which can be entered as Temp. stability on the MHz pill.
- **Short-term stability and jitter.** Phase noise and Allan deviation are a different question from where the average frequency sits.

- **Protocol limits.** Whether a budget meets USB, CAN or Ethernet timing needs those standards' own tolerances and belongs with the tools for each.

How it was checked

By hand, MHz defaults: 20 + 30 + 3 = 53 ppm worst case; √(400 + 900 + 9) = 36.18 ppm RSS; 53 ppm of 16 MHz is 848 Hz; 53 × 86.4 ms = 4.579 s a day; 53 × 31.56 s = 27.88 minutes a year. With −25 ppm of pulling the window becomes −78 to +28 ppm and the worst case reads −78, signed.

32.768 kHz at 0 °C, with the FC-135's B of −0.04: - drift $-0.04 \times 25^2 = -25$ ppm; - with ±23 ppm of limits the window is −48 to −2 ppm, worst −48 ppm.

At −10 °C it is −49 ppm of drift, −72 ppm worst, 6.221 s a day slow. A typed B of −0.034 at −10 °C gives −41.65 ppm of drift, −64.65 ppm worst. At 25 °C only the limits remain: ±23 ppm, 12.1 minutes a year. Switching pills resets pulling, which on first build carried over from one family to the other.

The review made these changes: - It made B and the turnover fields, defaulting to the FC-135 sheet. The tool hard-coded −0.034 ppm/°C^2 about 25 °C, which is a typical value and not the datasheet's worst case. - It put the worst case first and large. - It enlarged the 9–10 px chart labels to 11 px. - It added a plain definition of ppm, examples, and the remembered type. - It was renamed Oscillator stability, on Pierre's choice.

PLL settings

(formerly PLL multiplication factor)



What it computes

Every combination of input divider M, feedback multiplier N and output divider that a chip's PLL allows, walked exhaustively, with the ones that land closest to a target frequency listed first — each with its error in ppm and the frequencies at the phase detector and the VCO, so the limits can be seen being respected. The chosen setting is shown large, with its error and its internal frequencies; tap another row to choose it. Presets cover the STM32F1, F4, G0 and G4 and the RP2040 and RP2350; Generic takes any integer-N PLL's limits from its manual. The chip is remembered.

Source

The limits are taken from the vendors' datasheets and code rather than from summaries, which disagree with each other:

- **STM32F4** — ST's HAL driver (`stm32f4xx_hal_rcc.h`, `stm32f4xx_hal_rcc_ex.h`): PLLM 2–63, PLLN 50–432, PLLP 2, 4, 6 or 8, PLLQ 2–15, and the VCO input between 1 and 2 MHz. The VCO output window of 100–432 MHz and the 168 MHz system clock of the F405/407 are from RM0090.
- **STM32F1** (the F103 of the Blue Pill) — DS5319 rev 18, table 27: PLL input 1–25 MHz, PLL output 16–72 MHz. The F1 has no output divider: its PLL output is the system clock. From the F1 HAL: PLLMUL 2–16, the crystal divided by 1 or 2 on its way in (PLLXTPRE), and USB clocked by the PLL output divided by 1 or 1.5 — so USB works only with the PLL at 48 or 72 MHz.
- **STM32G4** (G474) — DS12288 rev 2, table 49: PLL input 2.66–16 MHz, VCO 96–344 MHz, and the R output that clocks the core up to 150 MHz, or 170 MHz with the regulator in Range 1 boost mode (section 3 of the same datasheet). Dividers from the G4 HAL: PLLM 1–16, PLLN 8–127, PLLR 2, 4, 6 or 8.
- **STM32G0** (G071) — DS12232 rev 4, table 43: PLL input 2.66–16 MHz, VCO 96–344 MHz, R output 12–64 MHz. Dividers from the G0 HAL: PLLM 1–8, PLLN 8–86, PLLR 2–8.
- **RP2040** — the Pico SDK's own calculator, `vcocalc.py`: reference divider 1–63 with at least 5 MHz after it, FBDIV 16–320, VCO 750–1600 MHz, and two post-dividers each 1–7.
- **The RP2040's rated clock** is from the Pico SDK 2.1.1 release notes: it is now "certified to run at a system clock of 200Mhz when using a regulator voltage of at least 1.15 volts". The SDK raises the voltage itself when needed, and keeps 125 MHz as the default. Before, 133 MHz was the most that was supported.
- **RP2350** — its datasheet, section 8.6, lists the RP2040's limits again (at least 5 MHz after REFDIV, VCO 750–1600 MHz, FBDIV 16–320, post-dividers 1–7) and gives 150 MHz as the most the RP2350 supports.

The STM32 datasheets were read in the copies distributors host (RS and DigiKey), because st.com refused the download; later revisions exist and were not compared. ST's own code disagrees with these tables: its LL utils files check the VCO from 64 MHz on both families and the G0's PLL input at 4–8 MHz, and the G4's reference manual (RM0440 rev 3) gave a 64–344 MHz VCO and a 2.66–8 MHz input. Asked about it on ST's community forum in April 2020, an ST technical moderator answered that the datasheet's PLL characteristics are the correct values and that RM0440 would drop its numbers, so the tables are used here.

Why the formulas are these

An integer-N PLL has one loop and three dividers:

```
PFD = Input / M
VCO = PFD × N
Out = VCO / P
```

The input is divided down to the **phase-frequency detector** (PFD) rate. The VCO runs at N times that, because the loop compares the VCO divided by N against the PFD reference and steers the VCO until they match. The output is the VCO divided down again. Each stage has a window it must stay inside: the PFD because the detector and its filter are designed for a range, the VCO because the oscillator only tunes over a range. That is what makes this a search rather than a division — the obvious M and N often put one of the stages outside its window.

The STM32F1 has no output divider, so there Out = PFD × N, and its window is the PLL output's. The output divider is P on the F4, R on the G0 and G4, and two dividers in series on the RP2040 and RP2350.

Ranking. Rows are ordered by error first. Among equally good rows:

- on the STM32F1 and F4, a row that also gives USB exactly 48 MHz comes first, since USB will not work on anything else;
- then the **higher PFD**. ST's manual recommends a 2 MHz VCO input on the F4 to limit jitter, and in general a faster comparison lets the loop correct the VCO more often and filter less, which is quieter;
- then the higher VCO, quieter for the same reason. It costs current: the G0 and G4 datasheets give their PLL 200 µA typical at a 96 MHz VCO and 520 µA at 344 MHz, so pick a lower-VCO row from the table when power matters more.

That is why 8 MHz to 168 MHz on the STM32F4 comes out as M 4, N 168, P 2, Q 7 (PFD 2 MHz, VCO 336 MHz, USB 48 MHz) ahead of the equally exact M 8, N 336, which runs the detector at 1 MHz.

The RP2040's and RP2350's two post-dividers multiply, so the tool offers each distinct product once, as the pair with the larger first divider — the SDK prefers that split for power.

Notes above a plain rating. The RP2040 above 133 MHz needs its core regulator at 1.15 V or more, and the G4 above 150 MHz needs Range 1 boost mode (`PWR_REGULATOR_VOLTAGE_SCALE1_BOOST` in the HAL); the tool says so under the result. Above the rating itself it says the part is not rated for it.

Assumptions and limits

- **Integer-N only.** Many modern PLLs add a fractional part to N, which can hit almost any target but trades it for spurs and depends on each vendor's modulator. Not modelled.
- The preset limits are for the named parts. Other members of a family can differ — the F401, for one, needs its VCO in 192–432 MHz, and the F105/F107 multiply by 4–9 or 6.5 after a 1–16 prescaler — and the G0 and G4 presets come from the G071 and G474 datasheets, so check another member's own table, or use Generic with the right reference manual when in doubt.
- The G0 and G4 windows are Range 1's. Range 2, the low-power range, narrows the VCO to 96–128 MHz and the system clock to 26 MHz on the G4 and 16 MHz on the G0; not modelled.
- Lock time, loop-filter design and phase noise are not addressed; the tool answers only which settings are legal and how close they land.
- On Generic, the search stops at three million combinations to stay responsive and says so; narrow the ranges for a complete answer.

What it deliberately does not do

- **Vendor clock trees beyond the PLL** — AHB and APB prescalers, peripheral clock muxes, flash wait states for the chosen frequency. Those are the next steps after this one, and every family arranges them differently.
- **The G0's and G4's other PLL outputs.** P and Q feed peripherals; the tool sets R, the system clock. The G4 can clock USB from its separate 48 MHz oscillator, HSI48, instead of the PLL.
- **Other presets.** The Generic pill covers any integer-N PLL given its manual; a preset is only worth adding for a part used often enough to save the reading.

How it was checked

Against the configurations the vendors ship: 8 MHz to 168 MHz on the STM32F4 gives M 4, N 168, P 2, Q 7 with USB at exactly 48 MHz, and from a 25 MHz crystal M 25, N 336, P 2, Q 7 — the standard setting for that crystal. 180 MHz is flagged as above the part's 168 MHz rating and as leaving USB at 45 MHz.

On the STM32F1, 8 MHz to 72 MHz gives M 1, N 9 with USB at 72 / 1.5 = 48 MHz, the setting of ST's own F1 USB examples; at 64 MHz the tool warns that USB cannot get its 48 MHz. On the STM32G4, 16 MHz (HSI16) to 170 MHz gives M 4, N 85, R 2 with the boost-mode note — the setting of ST's NUCLEO-G474RE examples, which switch boost mode on — and 150 MHz gives M 4, N 75, R 2 without it. On the STM32G0, 16 MHz to 64 MHz gives M 1, N 20, R 5 (VCO 320 MHz).

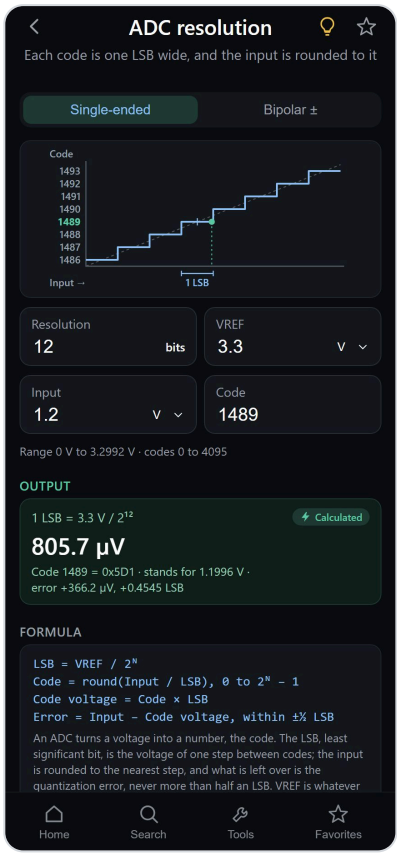
On the RP2040, 12 MHz to 125 MHz gives VCO 1500 MHz with post-dividers 6 and 2, and 133 MHz gives VCO 1596 MHz with 6 and 2 — both exactly the Pico SDK's own default settings. 200 MHz gives VCO 1200 MHz with 6 and 1, and the note about the 1.15 V core voltage; 250 MHz is flagged as above the certified 200 MHz. On the RP2350, 12 MHz to 150 MHz gives VCO 1500 MHz with 5 and 2, the SDK's default in `clocks.h`; 200 MHz is flagged as above its 150 MHz. The full RP2040 search takes about 3 ms.

The review made these changes: - It raised the RP2040's rating from 133 MHz to the 200 MHz certified in Pico SDK 2.1.1, with the voltage condition shown above 133 MHz. - It put the chosen setting first and large. - It enlarged the 9–10 px diagram labels to 11 px, and moved the VCO frequency clear of the USB branch. - It added plain definitions of the PFD, the VCO and the dividers, examples, and the remembered chip. - It was renamed PLL settings, on Pierre's choice. - It added the STM32F1, G0 and G4 and the RP2350, on Pierre's

choice, with limits read from their datasheets, the chips split over two rows of pills. - It fixed a chip switch that carried the RP2040's 1.15 V note over to the next chip. - Without a USB branch, the diagram drops its empty top band, which keeps Generic's result above the tab bar with the second row of pills.

ADC resolution

(formerly ADC resolution / quantization)



What it computes

For an ideal N-bit converter and its reference: the size of one step (the LSB), shown large since it is the resolution, then the code a given input produces, the voltage that code stands for, and the quantization error between the two, in volts and in LSB. It works both ways: type a voltage to get the code, or type a code — say, one read off a register — to get the voltage. The range of inputs and codes is shown under the fields. The reference and the input can be typed in mV or V; the mode and both units are remembered.

Two pills cover the two ways converters are built:

- **Single-ended** — 0 to VREF, codes 0 to $2^N - 1$. Every MCU ADC (AVR, STM32, RP2040, ESP32) works this way.
- **Bipolar ±** — −FSR to +FSR, codes -2^{N-1} to $2^{N-1} - 1$ in two's complement. Differential converters such as the ADS1115 and the HX711 work this way.

Examples: an Arduino Uno on its 5 V reference, an LM35 temperature sensor on the Uno's 1.1 V reference, an STM32 at 12 bits on 3.3 V, the ADS1115 at its default ±2.048 V, and the HX711 load-cell converter at 24 bits.

Source

- The ideal transfer function is the one in IEEE Std 1241 and the one MCU datasheets measure their errors against. The ATmega328P datasheet (Atmel-42735B, section 28.6) defines offset error as the deviation of the first transition from its ideal place "at 0.5 LSB", gain error against a last transition 1.5 LSB below full scale, the quantization error as always ±0.5 LSB, and ±0.5 LSB as the ideal absolute accuracy. The same chip is specified at ±2 LSB absolute accuracy, and its internal reference at 1.0 V minimum, 1.1 V typical, 1.2 V maximum.
- Arduino's analogReference() page gives the Uno's DEFAULT reference as 5 V and INTERNAL as 1.1 V on the ATmega328P.
- The bipolar pill follows the TI ADS1115 datasheet (SBAS444E): the full-scale range is set by the PGA to ±6.144, ±4.096, ±2.048 (the power-up default), ±1.024, ±0.512 or ±0.256 V; Table 7-1 gives the LSB as 125 µV at ±4.096 V and 62.5 µV at ±2.048 V; output is binary two's complement, clipping at 7FFFh and 8000h.
- The HX711 datasheet (Avia Semiconductor) gives its full-scale input as ±0.5(AVDD/GAIN), ±20 mV at gain 128 with a 5 V AVDD, 24-bit two's complement output from 800000h to 7FFFFFh, and 50 nV rms input noise at gain 128 and 10 samples a second.
- The LM35 is 10 mV/°C (TI SNIS159H).

Why the formulas are these

The step. N bits give 2^N codes. Spread over the reference:

```
Single-ended:  LSB = VREF / 2^N
Bipolar:       LSB = 2 × FSR / 2^N
```

The bipolar span is twice FSR because it runs from −FSR to +FSR. The ADS1115 datasheet writes its Equation 4 as LSB = FSR / 2^{16}, but its own table (125 µV at ±4.096 V) only works with FSR as the whole 8.192 V span — the tool asks for the ± figure, as the datasheet's register table quotes it, and doubles it.

The code. Code k stands for k × LSB and covers half an LSB either side, so the ideal converter rounds:

```
Code = round(Input / LSB)
```

The first transition, 0 to 1, is at ½ LSB. That is why the quantization error is ±½ LSB rather than 0 to −1 LSB: the steps are centred on the ideal straight line, and the error is the sawtooth between the line and the staircase. The drawing shows eight codes around the input at scale, with the dashed ideal line through the step centres.

The top of the range. The highest code is $2^N - 1$, not 2^N, so the largest voltage a code stands for is VREF − 1 LSB: 3.2992 V for 12 bits on 3.3 V. An input at VREF itself reads the top code, a whole LSB short.

Two's complement. A bipolar converter's register holds negative codes as 2^N + code, so −1 reads FFFFh and −FS reads 8000h. The hex in the result shows the register as it would be read, the Code field the signed value it means.

Dividing by $2^N - 1$. Arduino's ReadAnalogVoltage example computes code × (5.0 / 1023.0), and ST's LL macro `__LL_ADC_CALC_DATA_TO_VOLTAGE` divides by `__LL_ADC_DIGITAL_SCALE`, 4095 at 12 bits. That treats the top code as exactly VREF, stretching the scale: the reading is k / (2^N − 1) instead of k / 2^N of VREF, high by up to one LSB near the top. Small beside a real ADC's errors, but a systematic one, and it is why a 3.3 V input "reads 3.3 V" on those examples and 3.2992 V here.

Resolution is not accuracy. A finer LSB only helps while it stays above the noise and the reference's tolerance. The HX711's 2.33 nV step sits under 50 nV rms of noise, about 21 LSB, so its last four bits or so are noise; SNR estimation turns noise into effective bits.

Assumptions and limits

- **An ideal converter.** Offset, gain error, INL, DNL and noise are not modelled; on a real MCU ADC they add up to several LSB, far more than the quantization shown here (the ATmega328P's ±2 LSB, for one). Their datasheet figures are in LSB, which is what this tool gives the size of.
- **VREF is taken as exact.** On most MCUs it is the supply, so the supply's own tolerance scales every reading. A 1% regulator is 41 LSB at 12 bits. An internal reference is no cure by itself: the ATmega328P's 1.1 V is 1.0 to 1.2 V from part to part, ±9 %, so measure it once and use that figure.
- **Inputs past the range** read the end code; the tool says so and the error then is the overshoot, not quantization. On the ADS1115 the ±4.096 and ±6.144 V settings are scaling only — the pins

themselves never go past VDD + 0.3 V or below GND − 0.3 V.
- **Single-ended on a bipolar part.** The ADS1115 measuring one input against ground uses only codes 0000h to 7FFFh, so it is a 15-bit converter in that mode. Model it on the bipolar pill with 16 bits, and read only the positive half.

What it deliberately does not do

- **SNR, ENOB, oversampling.** The noise that quantization adds, and what averaging buys back, is the next tool, SNR estimation.
- **DAC.** The reverse conversion has its own tool.
- **Offset-binary and sign-magnitude codes.** Some bipolar converters output these instead of two's complement; they are rare enough on current parts to leave out.

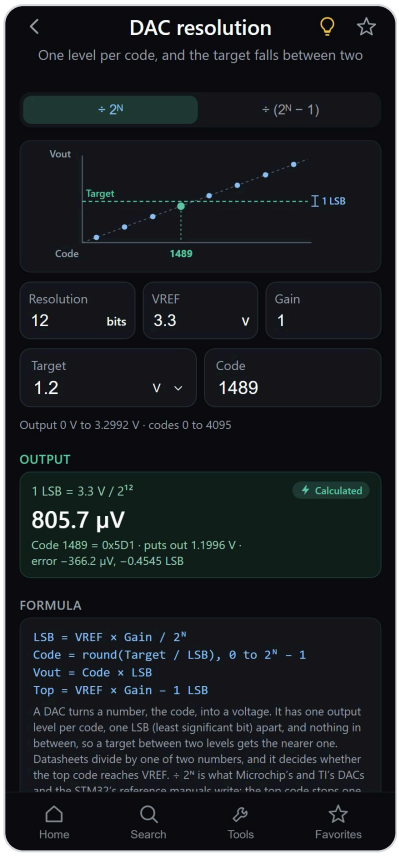
How it was checked

By hand, single-ended 12 bits on 3.3 V: LSB = 3.3 / 4096 = 805.66 µV; 1.2 V / 805.66 µV = 1489.45, code 1489 = 0x5D1, code voltage 1.19963 V, error +366.2 µV = +0.4545 LSB. Code 4095 gives 3.29919 V; 4 V reads 4095 and is flagged as over range.

The examples: 3.3 V on the Uno's 5 V reference reads 676 (4.883 mV steps, −0.16 LSB); an LM35 at 25 °C, 250 mV, reads 233 on the 1.1 V reference, in steps of 1.074 mV, 0.11 °C, against 0.49 °C on 5 V; −1 V on the ADS1115 at ±2.048 V reads −16000, 0xC180; 5 mV on the HX711 at ±19.53 mV reads 2147484, 0x20C49C, in steps of 2.328 nV.

The review made these changes: - It was renamed ADC resolution, on Pierre's choice; the old name still finds it in search. - The LSB became the large result, with the code, its voltage and the error on the line under it. - The reference and input fields take mV or V, remembered with the mode. - The drawing's labels went from 9–10 px to 11 px, with a left margin that grows with the code's digits. - It added the five examples, checked against their datasheets. - In formula search, the cards of renamed tools had lost their link to the tool; they now find a tool by its old name too.

DAC resolution



What it computes

For an N-bit DAC, its reference and its output gain: the step between neighbouring output levels (the LSB), shown large since it is the resolution, then the code to write for a target voltage, the voltage that code really puts out, and how far that is from the target, in volts and in LSB. It works both ways — type the voltage you want, or type a code to see what it outputs. The output range is shown under the fields. The target can be typed in mV or V; the divisor pill and the unit are remembered.

Examples: an MCP4725 on 5 V, an MCP4822 at ×2, an STM32 DAC set to 0.7 V, the ESP32's 8-bit DAC and the SAMD21's 10-bit DAC (the Arduino Zero's).

Source

Divided by 2^N, the top code one LSB under VREF:

- **Microchip MCP4725** (DS22039D): Equation 4-1 gives the ideal LSB as VREF / 2^n, which equals (full-scale − zero-scale) / (2^n − 1); VREF is VDD on this part.
- **Microchip MCP4802/4812/4822** (DS20002249B): VOUT = VREF × D / 4096 with the GA bit at 1 (×1), and 2 × VREF × D / 4096 with GA at 0 (×2), from the internal 2.048 V reference; its table gives 0.5 mV per step at ×1 and 1 mV at ×2.
- **TI DAC7311** (SBAS442D): Equation 1, VOUT = AVDD × D / 2^n, read in TI's HTML datasheet viewer.
- **STM32.** On ST's community forum, a user quotes RM0444 (STM32G0) as dividing by 4096, and in a 2017 thread on the STM32F373, whose reference manual also writes 4096, ST engineer K.Oku explains why: the DAC "is kind of R-2R implementation", and its top code "has one resistor (corresponds 1LSB) under the VREF". RM0444 itself was not read; st.com refused the download. The STM32G071 datasheet (DS12232 rev 4, table 59) gives the output as 0 to VREF+ with the buffer off and 0.2 V to VREF+ − 0.2 V with it on.

Divided by 2^N − 1, the top code at VREF:

- **SAMD21** (Atmel-42181, the Arduino Zero's microcontroller): VDAC = DATA / 0x3FF × VREF.
- **ESP32** (technical reference manual, section 31.4.3): DACn_OUT = VDD3P3_RTC × PDACn_DAC / 255, with VDD3P3_RTC typically 3.3 V.

And ST's software, against its own manuals: the STM32G0 HAL's comment writes DAC_OUTx = VREF+ × DOR / 4095, with the example (3.3 × 868) / 4095 = 0.7 V, and its LL driver's `DAC_DIGITAL_SCALE_12BITS` is 4095, which `__LL_DAC_CALC_VOLTAGE_TO_DATA` multiplies by.

Why the formulas are these

One level per code. A DAC puts out one voltage for each code and nothing in between:

```
Vout = Code × LSB
```

so a target that is not exactly a level gets the nearer one, at most half an LSB away. That is why the picture is points, not a staircase: eight levels around the target at scale, the target a line between two of them, the chosen one in green.

Two divisors, and why it matters. How big the step is depends on what the datasheet divides by:

```
÷ 2ᴺ:        LSB = VREF × Gain / 2ᴺ          top code = VREF × Gain − 1 LSB
÷ (2ᴺ − 1):  LSB = VREF × Gain / (2ᴺ − 1)    top code = VREF × Gain exactly
```

A resistor string or R-2R ladder makes 2^N equal steps between ground and VREF, of which the top code uses $2^N - 1$, so the output stops one LSB short: that is the $\div 2^N$ datasheets, the STM32 among them. The SAMD21 and ESP32 documents divide by $2^N - 1$ instead. The difference is under one LSB, but it is systematic, largest near the top of the range. Pick the pill your part's datasheet uses.

ST's 4095. ST's HAL and LL code divide by 4095, so a code they compute is up to one LSB low near the top on a DAC that, by ST's manuals, divides by 4096. At 0.7 V on 3.3 V, the HAL comment's 868 puts out 0.69932 V; 869, the nearer level on the 4096 scale, puts out 0.70012 V.

An earlier version of this tool took the STM32 to be ÷ 4095, on a forum user's bench readings from a NUCLEO-G071RB. Those readings cannot decide it: on the 2.5 V reference used there, the two divisors give the same code, or codes one LSB (0.61 mV) apart, at the 400, 800 and 1200 mV targets, yet the readings differ by about 4 mV, six or seven LSB, so something else changed between the two runs.

Gain. Many DACs follow the ladder with an amplifier. On the MCP4822 the GA bit picks ×1 or ×2 from its internal 2.048 V reference, which is how a 2.048 V reference makes a 4.095 V range at 1 mV a step.

Assumptions and limits

- **The output cannot reach the rails.** The MCP4822's amplifier swings from 10 mV to VDD − 40 mV, and its accuracy holds only between those. The STM32 DAC with its output buffer on spans 0.2 V to VREF+ − 0.2 V. The top and bottom few codes on the tool's range are therefore not really available on those parts, and with ×2 gain the range can exceed VDD, which the output will not follow either.
- **An ideal DAC.** Offset, gain error, INL and DNL are not modelled; they are specified in LSB, which is what this tool gives the size of.
- **VREF is taken as exact.** When it is VDD, as on the MCP4725, the supply's tolerance scales every output.

What it deliberately does not do

- **Bipolar DACs** (±VREF output with offset-binary or two's-complement codes) are rare on current general-purpose parts; the bipolar pill on the ADC tool covers the arithmetic if needed.

- **PWM as a DAC.** Resolution there is set by the timer and the filter, not a ladder; it is a different calculation.
- **SNR and ENOB** are the next tool.

How it was checked

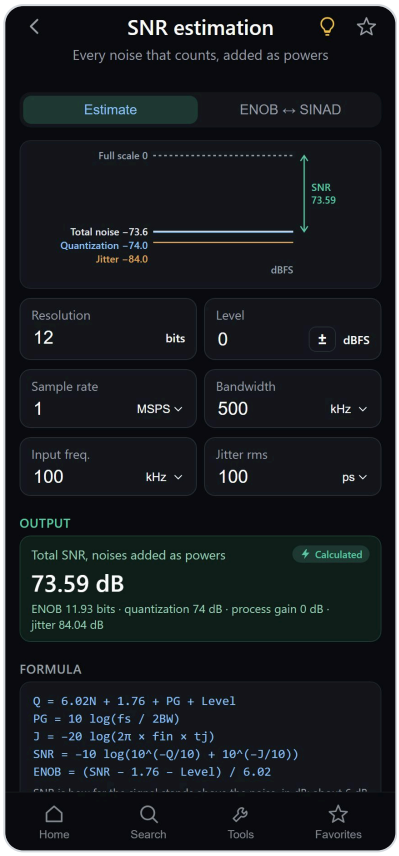
÷ 2^N, 12 bits, 3.3 V: LSB 805.66 µV; 1.2 V gives code 1489 (0x5D1), output 1.19963 V, error −366.2 µV = −0.4545 LSB. Code 4095 outputs 3.29919 V, one LSB short of VREF.

÷ $(2^N - 1)$: LSB 3.3 / 4095 = 805.86 µV; code 4095 outputs exactly 3.3 V; 1.2 V gives code 1489, output 1.19993 V.

The examples: the MCP4725 on 5 V puts 3.3 V at code 2703, 0xA8F, 0.44 mV low; the MCP4822 at ×2 steps exactly 1 mV, so 3 V is code 3000; the STM32 at 0.7 V gets 869, +122 µV; the ESP32's 8 bits ÷ 255 step 12.94 mV, so 1.5 V is code 116 at 1.501 V; the SAMD21's 10 bits ÷ 1023 put code 310 at exactly 1 V.

The review made these changes: - The STM32 moved from the ÷ $(2^N - 1)$ side to ÷ 2^N, after reading ST's engineer's answer and its manuals' 4096; the second pill now rests on the SAMD21 and ESP32 documents, and the forum bench test is no longer cited as evidence. Analog Devices is no longer named: none of its datasheets was read. - The LSB became the large result, with the code, its output and the error on the line under it. - The target field takes mV or V, remembered with the pill. - The drawing's labels went from 9–10 px to 11 px. - It added the five examples, the first cases (13), and in every tool's examples a value now stays on one line with its unit.

SNR estimation



What it computes

Two things, one per pill.

- **Estimate** — the signal-to-noise ratio an ADC can reach from the noises that can be calculated: quantization, the process gain of oversampling, and sampling-clock jitter. Each is shown on its own, then added as noise powers into one SNR and expressed as an effective number of bits (ENOB).
- **ENOB ↔ SINAD** — the conversion between the two figures ADC datasheets quote, both ways, with how many of the converter's bits noise and distortion take away.

The total SNR is the large result on Estimate, with the ENOB it is worth and its three terms on the line under it; on ENOB ↔ SINAD the effective bits are large, with the bits lost and the noise floor. The pill and the units of the sample rate, bandwidth, input frequency and jitter are remembered.

The picture is a level diagram in dBFS: full scale at the top, the signal, and each noise floor as a line. The gap from the signal down to the total floor is the SNR, and the highest noise line is the one worth fixing.

Examples: 16-bit audio at 44.1 kHz, ×16 oversampling, 1 ps of jitter at 100 MHz, a signal at −20 dBFS, and the ADCs of the RP2040 and the STM32G0.

Source

Walt Kester's Analog Devices tutorials:

- **MT-001**, "Taking the Mystery out of the Infamous Formula, SNR = 6.02N + 1.76 dB": the ideal SNR of an N-bit converter for a full-scale sine measured over dc to fs/2, and the correction 10 log(fs / 2BW) it calls process gain.
- **MT-003**, "Understand SINAD, ENOB, SNR, THD, THD + N, and SFDR": ENOB from SINAD, and its Equation 2, which adds the level below full scale so ENOB is normalised to full scale.
- **MT-007**, "Aperture Time, Aperture Jitter, Aperture Delay Time": the SNR limit set by jitter, and that the total jitter is the root-sum-square of the sampling clock's and the ADC's own aperture jitter.

analog.com refused a re-read of these during the review, so the formulas were checked again elsewhere:

- **TI's SLYT379**, part 1 of an Analog Applications Journal series on clock jitter: its Equation 2 is SNR = −20 log(2π × fIN × tJitter), its Equation 1 adds the quantization, thermal and jitter SNRs as noise powers, and its Equation 3 combines the clock's and the ADC's aperture jitter as a root-sum-square.
- Wikipedia's article on ENOB quotes MT-003's ENOB = (SINAD − 1.76) / 6.02.

- National Semiconductor's note on aperture time and jitter (TI SNAA018) agrees that doubling the sample rate and filtering gains 3 dB. Its jitter example does not agree with the formula: it gives 73 dB for 10 ps at 10 MHz, where −20 log(2π × 10 MHz × 10 ps) is 64 dB, because it divides by the full-scale range where the sine's rms value belongs (20 log 2√2 = 9 dB).

Datasheet figures:

- **RP2040**, section 4.9.3, table 566, typical, measured at 250 ksps with the input spanning codes 1 to 4094: SNR 61.5 dB, THD −55 dB, SFDR 59.9 dB, SINAD 54.0 dB, ENOB 8.7. These are the ENOB pill's defaults.
- **STM32G071**, DS12232 rev 4, table 58: SINAD 63 dB typical (62.5 dB minimum) and ENOB 10.2 (10.1) at VDDA = VREF+ = 3 V and 25 °C.

Why the formulas are these

Quantization. Rounding to the nearest code leaves an error spread evenly over ±½ LSB, whose rms value is q / √12. A full-scale sine has an rms value of 2^N q / (2√2). Their ratio, in dB, is

```
SNR = 6.02N + 1.76 dB
```

— 6.02 dB per bit, which is 20 log 2, plus a constant from the shape of a sine. 12 bits: 74.0 dB. 16 bits: 98.1 dB.

Level. The quantization noise stays where it is when the signal gets smaller, so the SNR falls one dB for every dB below full scale. A 12-bit ADC reading a signal at −20 dBFS has 54 dB of SNR, the same as an ideal 8.7-bit one. That is the most common way resolution is wasted.

Process gain. Quantization noise is spread evenly from dc to fs/2. Filter the result digitally down to a bandwidth BW and only the share of the noise inside BW remains:

```
PG = 10 log(fs / 2BW)
```

Four times the bandwidth needed is 6.02 dB, one extra bit. MT-001's example: a 65 MSPS ADC with channels 30 kHz wide gains 30.3 dB, 65 dB becoming 95.3 dB.

Jitter. If the instant of sampling wanders by tj rms, the error it causes is the signal's slope times that wander. For a full-scale sine at fin the result is independent of amplitude:

```
SNR(jitter) = −20 log(2π × fin × tj)
```

It depends on the input frequency, not the sample rate. 100 ps is harmless at 1 kHz (124 dB) and ruinous at 100 MHz (24 dB). MT-007 notes that 14-bit performance at 100 MHz needs under 0.1 ps; the tool gives 84 dB at exactly 0.1 ps, just under the 86 dB of 14 bits.

Adding them. Uncorrelated noises add as powers:

```
SNR = −10 log(10^(−Q/10) + 10^(−J/10))
```

so the total is always a little worse than the worse of the two, and a term 10 dB better than the other barely counts. When jitter is the lower one, the tool says so — more bits will not help.

What ENOB means. ENOB, the *effective number of bits*, answers the question "how many bits is this converter really worth?". The resolution on the datasheet, 12 bits say, only counts the codes the ADC can output. A real ADC also adds its own noise and distortion, so the last few bits are buried in them: they change from reading to reading without telling you anything about the signal. ENOB is the resolution of a perfect converter that would be exactly as noisy as the real one. A 12-bit ADC with an ENOB of 8.7 gives, on a changing signal, the same quality as a flawless 8.7-bit ADC; the bottom 3.3 bits are noise. ENOB is not a whole number, because it is worked back from a measured ratio, not counted.

What SINAD means. SINAD, the *signal-to-noise-and-distortion ratio*, is that measured ratio: a pure sine is fed in, and the power of the sine is compared with the power of everything else in the output — noise and the harmonics that distortion adds. SNR is the same measurement leaving the harmonics out, which is why a datasheet's SNR is always the higher of the two. Because a perfect N-bit converter has an SINAD of 6.02N + 1.76 dB, the two figures are the same fact in different units, and datasheets quote either.

ENOB from the ratio. The SNR equation solved for N, with the level term of MT-003:

```
ENOB = (SNR − 1.76 − Level) / 6.02
```

On the ENOB pill SINAD takes the place of SNR, as the datasheets do. The RP2040's 54.0 dB gives 8.68 bits, and its datasheet says 8.7: of its 12 bits, 3.3 are lost to noise and distortion. Its noise alone, an SNR of 61.5 dB, would leave 9.9; its THD of −55 dB — its INL and DNL errors (the DNL spikes are erratum RP2040-E11) — costs more than the noise. The STM32G071's 63 dB gives 10.17 bits, its datasheet's 10.2.

Assumptions and limits

- **Oversampling needs noise.** Process gain assumes the quantization error is random. A clean, slow signal on a quiet ADC can give the same code every time, and averaging a constant gives nothing. MT-001 warns about this correlation; the cure is noise or dither of about an LSB at the input.
- **Thermal noise and distortion are not estimated.** They are the ADC's own and only its datasheet knows them. The Estimate pill gives the ceiling the architecture allows; the ENOB pill gives what the part actually does.
- **No noise shaping.** Delta-sigma converters push quantization noise out of band and gain far more than 10 log per octave of oversampling. Their datasheets give SNR directly.
- **SINAD at other levels.** MT-003's level correction assumes the noise does not change with level. Distortion usually drops at lower levels, so ENOB normalised from a −6 dBFS measurement can read better than at full scale.

What it deliberately does not do

- **THD, SFDR, THD + N.** Read from the datasheet, not calculated.

- **Noise-free resolution and effective resolution** from rms input noise, the measure for dc and slow signals. MT-003 warns not to confuse them with ENOB; they are a different calculation from different data.

How it was checked

Default Estimate: 12 bits, 1 MSPS, 500 kHz, 0 dBFS, 100 kHz, 100 ps. Q = 6.02 × 12 + 1.76 = 74.0 dB; J = −20 log($2\pi \times 10^{5} \times 10^{-10}$) = 84.04 dB; total 73.59 dB; ENOB 11.93 bits.

MT-001's process-gain example: 65 MSPS and 30 kHz give 30.35 dB. MT-007's jitter example: 14 bits, 100 MHz, 0.1 ps give J = 84.04 dB, total 81.91 dB, with the jitter warning. 40 MHz of bandwidth at 65 MSPS is refused as past fs/2. With no jitter at −1 dBFS, 14 bits give 85.04 dB and an ENOB of 14.0, normalised.

ENOB pill: SINAD 54.0 dB at 0 dBFS gives 8.678 bits — the RP2040's 8.7 — with 3.32 bits lost; ENOB 10 gives SINAD 61.96 dB.

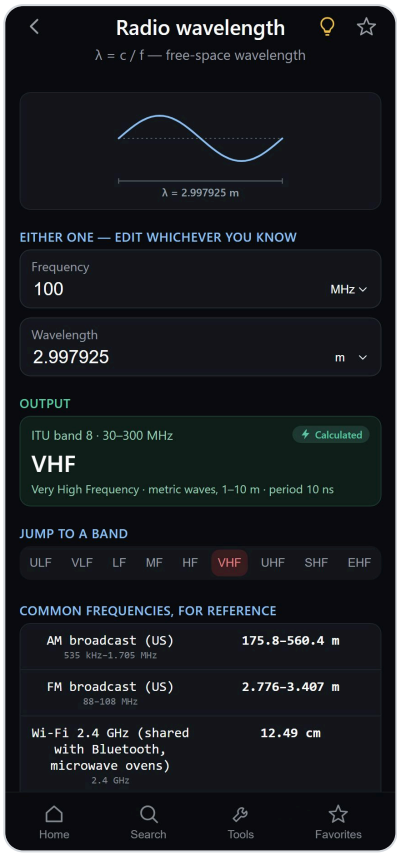
The examples: 16 bits over the whole band with no jitter, 98.08 dB; 12 bits keeping 31.25 kHz of a 1 MSPS band, process gain 12.04 dB, total 86.04 dB, 14 bits; 14 bits at 250 MSPS with 1 ps at 100 MHz, jitter 64.04 dB, total 64.01 dB, 10.34 bits, with the warning; 12 bits at −20 dBFS, 54 dB and an ENOB still of 12; the STM32G0's 63 dB, 10.17 bits.

The review made these changes: - The total SNR and the ENOB became the large results, with their terms on the line under them. - Rates, frequencies and jitter are kept in hertz and seconds, so a remembered unit only changes how a field shows them; the pill and units are remembered. - The Estimate fields went from three to a row to two: at 375 px the sample rate's unit was cut to "MS…". - The diagram's labels went from 8–9 px to 11 px, with values to one decimal. - It added the six examples, the first cases (12) and a formula-search card.

RF & MICROWAVE › ANTENNAS & PROPAGATION

Radio wavelength

(formerly Wavelength (RF), then Wavelength ↔ frequency)



What it computes

A frequency and its free-space wavelength, each from the other — type either — with the period of one cycle and the ITU band the frequency falls in: its symbol and number, its full name, and the name ITU gives its wavelengths (metric waves for VHF, decimetric for UHF). The band is the large result, since the wavelength already sits in its field, and the drawing carries the wavelength's value. A row of buttons jumps to the middle of each named band, and a table gives the wavelengths of common allocations.

The units are remembered. When typing one field leaves the other unreadable in its unit — 100 Hz is 2 997 925 000 mm — that field moves to the unit that reads best, without changing the remembered choice.

Examples: FM radio at 100 MHz, AM radio at 1 MHz, Wi-Fi at 2.4 GHz, GPS L1, a 1 m wavelength typed as such, and exactly 30 MHz, on a band edge.

Source

- **c = 299 792 458 m/s**, exact: the metre has been defined from the speed of light since 1983.
- **The bands** are ITU-R Recommendation V.431-8 (08/2015), table 1, which follows No. 2.1 of the Radio Regulations: band N runs from 0.3 × 10ᴺ to 3 × 10ᴺ Hz, "lower limit exclusive, upper limit inclusive", with the symbols ULF (band 3) to EHF (band 11) and a metric name for each band's wavelengths, from hectokilometric to decimillimetric waves. Band 12, 300 GHz to 3 THz, has no symbol in the table.
- **The reference rows** are US allocations from the FCC: the AM broadcast band, 535 to 1705 kHz (47 CFR 73.14); the FM broadcast band, 88 to 108 MHz (73.310); and Wi-Fi's 6 GHz band, 5.925 to 7.125 GHz (15.407), all read through Cornell's copy of the CFR. GPS L1, 1575.42 MHz, is 154 times the 10.23 MHz reference that IS-GPS-200 section 3.3.1.1 specifies: the change notice read here gives the 10.23 MHz, and the multiplier comes from secondary sources.

Why the formulas are these

```
λ = c / f
f = c / λ
T = 1 / f
```

A wave travels at c and repeats every period T, so one cycle covers c × T, which is the wavelength. In free space only: in a cable or on a PCB the wave is slower, by the velocity factor, and so its wavelength is shorter.

Bands by decade. ITU's bands are decades of frequency, and since λ = c/f each is a decade of wavelength too: band N spans 10^(8−N) to 10^(9−N) m, which is where the metric names come from — VHF, 30 to 300 MHz, is 1 to 10 m, the metric waves. The decades are nominal: c is not quite 3 × 10^8 m/s, so 30 MHz is 9.993 m rather than 10.

Edges. Each band includes its upper limit and excludes its lower one, so exactly 30 MHz is HF, band 7, and anything above it is VHF.

Assumptions and limits

- **Free space.** Air is close enough to it for this purpose; coax, twisted pair and PCB traces are not, and need their velocity factor.
- **ITU bands 3 to 12 only.** Below 300 Hz and above 3 THz the tool shows no band. V.431 also lists URSI's extension to lower bands and bands 13 to 15 up to 3000 THz, which reach the infrared; light has its own tool, Wavelength ↔ color.
- **The reference table is the US's.** Other regions allocate these bands differently.

What it deliberately does not do

- **Antenna lengths.** A quarter-wave or half-wave antenna is a fraction of this wavelength, shortened further by end effects: that is the Antenna length tool's job when it is built.
- **Velocity factor.** Its own future tool.
- **Band plans.** Amateur, cellular and microwave allocations are reference tools of their own.

How it was checked

100 MHz gives 2.997925 m: VHF, ITU band 8, metric waves 1–10 m, period 10 ns. Exactly 30 MHz gives HF and 9.993082 m, and 30.001 MHz gives VHF. A 1 m wavelength gives 299.7925 MHz. GPS L1 keeps its decimals: 1575.42 MHz, 19.02937 cm. The EHF button lands on the band's geometric middle, 94.86833 GHz. 1 kHz is ULF, hectokilometric waves, 100–1000 km; 500 GHz is band 12; 100 Hz is outside the bands, its wavelength shown as 2997.925 km.

The review made these changes: - It was renamed Radio wavelength, on Pierre's choice: with the examples bulb, "Wavelength ↔ frequency" wrapped. - Band edges now follow ITU, the upper limit in the band: the tool had put exactly 30 MHz in VHF. - ULF (band 3) and band 12 were added, with each band's number and ITU wavelength name. - The AM row read 525 kHz, which is not the US band's edge; it is now the FCC's 535–1705 kHz, and the rows say US. - The band and the period became one result; the wave on the drawing went from the domain colour to part blue and carries the wavelength's value. - Frequency and wavelength are kept in hertz and metres with remembered units, shown to seven figures so GPS's .42 survives. - It added six examples and the first cases (11).